

CM3005 Data Science Midterm Coursework

Predicting Brazilian E-commerce Freight Costs Using Multiple Linear Regression

Module: CM3005 Data Science

Project: Data Science Midterm Coursework

Dataset: Brazilian E-commerce Public Dataset by Olist

Programming Language: Python

Development Environment: Jupyter Notebook

Project Aim

This project investigates whether the freight cost of Brazilian e-commerce order items can be predicted using product characteristics, geographical distance, product category, order value and temporal information.

The project applies data acquisition, cleaning, sanitisation, statistical analysis, visualisation, multiple linear regression, cross-validation and feature engineering. The final objective is to compare a baseline regression model with a feature-engineered model and determine whether the additional variables improve freight-cost prediction.

1. Domain-Specific Area and Project Objectives

The domain-specific area selected for this project is freight-cost estimation in business-to-consumer e-commerce and last-mile logistics. Last-mile delivery is the stage in which goods move from a seller, fulfilment point or distribution facility to the final customer. It is operationally difficult because delivery costs are affected by geographical dispersion, parcel characteristics, order composition and the density of deliveries. Previous research describes the last mile as one of the most expensive and least efficient parts of the logistics chain and shows that its cost is influenced by several measurable characteristics (Gevaers, Van de Voorde and Vanelslander, 2014). The continued growth of consumer-driven e-commerce has therefore increased the importance of reliable planning and cost-estimation methods (Lim, Jin and Srail, 2018).

The main objective of this project is to predict the freight value charged for an individual product item in a Brazilian e-commerce order. The prediction uses information available at or close to the time of purchase, including product price, weight, length, height and width; seller-to-customer distance; product category; customer and seller state; purchase timing; and order-level characteristics. A second objective is to identify which factors have the strongest relationship with freight cost. A third objective is to test whether domain-informed feature engineering and regularisation improve predictive accuracy.

Multiple linear regression is appropriate because the target, `freight_value`, is continuous and may be influenced simultaneously by several numerical and categorical predictors. Multiple regression can estimate the combined relationship between the target and multiple explanatory variables while providing interpretable coefficients and measurable prediction error (James et al., 2023). It also provides a transparent baseline against which feature-engineered and regularised models can be compared.

The project contributes a reproducible workflow for integrating relational e-commerce data, calculating geographical distance, preventing target leakage and evaluating unseen orders through group-aware validation. In practice, such a model could support preliminary freight quotation, cost planning, budgeting and the identification of unusually expensive deliveries. Although the model is developed using Brazilian marketplace data, the overall method could be transferred to other e-commerce settings after retraining with local product, customer and logistics information.

2. Dataset Description and Suitability

This project uses the Brazilian E-Commerce Public Dataset by Olist, obtained from Kaggle. The dataset contains approximately 100,000 orders placed between 2016 and 2018 across multiple Brazilian marketplaces and includes information about customers, sellers, products, order status, prices, freight charges, payments, reviews and geographical locations (Olist, n.d.).

The downloaded dataset consists of nine CSV files connected through relational identifiers. The main files contain 99,441 customers, 99,441 orders, 112,650 order-item records, 32,951 products, 3,095 sellers, 103,886 payment records, 99,224 reviews, 1,000,163 geolocation observations and 71 product-category translations. The tables are linked through keys such as `order_id`, `customer_id`, `product_id`, `seller_id` and ZIP-code prefix. This relational structure creates meaningful preprocessing work because the source is not provided as one analysis-ready First Normal Form table.

The dataset contains several data types. Identifiers, city names, state codes, product categories and order status are categorical or textual. Price, freight value, product weight, dimensions, payment values and coordinates are numerical. Purchase, approval, dispatch, delivery and review fields are timestamp variables that require conversion to datetime format. The raw files also contain missing product information, duplicated geolocation observations and legitimate one-to-many relationships.

The dataset fits the project because `freight_value` is a continuous numerical target suitable for regression, while the other tables provide multiple potential predictors. These include price, weight, dimensions, parcel volume, seller-to-customer distance, category, regional location, order size and purchase timing. Multiple linear regression is designed to model a continuous response using several explanatory variables (James et al., 2023). Payment values, reviews and post-delivery outcomes are excluded from the predictors because they may introduce target leakage or information unavailable when freight cost is estimated.

After cleaning and controlled relational merging, the complete analytical population contains 112,650 order items. A reproducible sample of 9,000 rows is used for modelling, while its distribution is compared with the full cleaned population to confirm that the sample remains representative.

```
In [1]: # Standard library imports
from pathlib import Path
import math
import warnings

# Data manipulation
import numpy as np
import pandas as pd

# Data visualisation
import matplotlib.pyplot as plt

# Statistical analysis
from scipy import stats

# Scikit-learn preprocessing
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import (
    OneHotEncoder,
    PolynomialFeatures,
    StandardScaler
)

# Scikit-learn modelling
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.model_selection import (
    KFold,
    cross_validate,
    train_test_split
)
```

```

# Scikit-learn evaluation
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

# Display and warning configuration
warnings.filterwarnings("ignore")

pd.set_option("display.max_columns", 100)
pd.set_option("display.max_rows", 100)
pd.set_option("display.float_format", lambda value: f"{value:,.3f}")

# Ensure plots appear inside the Jupyter notebook
%matplotlib inline

```

```

In [2]: # =====
# PROJECT CONFIGURATION
# =====

# Main project folders
PROJECT_DIRECTORY = Path.cwd()
RAW_DATA_DIRECTORY = PROJECT_DIRECTORY / "data" / "raw"
PROCESSED_DATA_DIRECTORY = PROJECT_DIRECTORY / "data" / "processed"
FIGURES_DIRECTORY = PROJECT_DIRECTORY / "figures"

# Create output folders if they do not already exist
PROCESSED_DATA_DIRECTORY.mkdir(parents=True, exist_ok=True)
FIGURES_DIRECTORY.mkdir(parents=True, exist_ok=True)

# Reproducibility settings
RANDOM_STATE = 42

# The coursework recommends using fewer than 10,000 observations
SAMPLE_SIZE = 9_000

# Test data will contain 20% of the final modelling dataset
TEST_SIZE = 0.20

# Five-fold cross-validation will be used
NUMBER_OF_FOLDS = 5

# Display the configured paths
print("Project directory:")
print(PROJECT_DIRECTORY)

print("\nRaw data directory:")
print(RAW_DATA_DIRECTORY)

print("\nProcessed data directory:")
print(PROCESSED_DATA_DIRECTORY)

print("\nFigures directory:")
print(FIGURES_DIRECTORY)

```

Project directory:
/Users/zimmeljavedvirk/Desktop/datasciencemid

Raw data directory:
/Users/zimmeljavedvirk/Desktop/datasciencemid/data/raw

Processed data directory:
/Users/zimmeljavedvirk/Desktop/datasciencemid/data/processed

Figures directory:
/Users/zimmeljavedvirk/Desktop/datasciencemid/figures

```
In [3]: # =====
# REQUIRED DATASET FILES
# =====

required_files = {
    "customers": "olist_customers_dataset.csv",
    "geolocation": "olist_geolocation_dataset.csv",
    "order_items": "olist_order_items_dataset.csv",
    "payments": "olist_order_payments_dataset.csv",
    "reviews": "olist_order_reviews_dataset.csv",
    "orders": "olist_orders_dataset.csv",
    "products": "olist_products_dataset.csv",
    "sellers": "olist_sellers_dataset.csv",
    "category_translation": "product_category_name_translation.csv"
}

print("Dataset file check")
print("=" * 70)

all_files_available = True

for dataset_name, file_name in required_files.items():
    file_path = RAW_DATA_DIRECTORY / file_name
    file_exists = file_path.exists()

    status = "FOUND" if file_exists else "MISSING"

    print(
        f"{dataset_name:<25} "
        f"{status:<10} "
        f"{file_path}"
    )

    if not file_exists:
        all_files_available = False

print("=" * 70)

if all_files_available:
    print("All nine dataset files were found successfully.")
else:
    print(
        "One or more dataset files are missing. "
        "Check the data/raw folder and filenames."
```

)

Dataset file check

=====		
customers	FOUND	/Users/zimmeljavedvirk/Desktop/datasciencemid/data/raw/olist_customers_dataset.csv
geolocation	FOUND	/Users/zimmeljavedvirk/Desktop/datasciencemid/data/raw/olist_geolocation_dataset.csv
order_items	FOUND	/Users/zimmeljavedvirk/Desktop/datasciencemid/data/raw/olist_order_items_dataset.csv
payments	FOUND	/Users/zimmeljavedvirk/Desktop/datasciencemid/data/raw/olist_order_payments_dataset.csv
reviews	FOUND	/Users/zimmeljavedvirk/Desktop/datasciencemid/data/raw/olist_order_reviews_dataset.csv
orders	FOUND	/Users/zimmeljavedvirk/Desktop/datasciencemid/data/raw/olist_orders_dataset.csv
products	FOUND	/Users/zimmeljavedvirk/Desktop/datasciencemid/data/raw/olist_products_dataset.csv
sellers	FOUND	/Users/zimmeljavedvirk/Desktop/datasciencemid/data/raw/olist_sellers_dataset.csv
category_translation	FOUND	/Users/zimmeljavedvirk/Desktop/datasciencemid/data/raw/product_category_name_translation.csv
=====		

All nine dataset files were found successfully.

3. Data Preparation: Acquisition, Cleaning and 1NF Construction

The Brazilian E-commerce Public Dataset by Olist is distributed across nine CSV files. Each file represents a different business entity, including customers, sellers, products, orders, order items, payments, reviews and geographical locations.

The files are loaded separately before any cleaning or merging is performed. This preserves the original relational structure and allows the quality, dimensions, data types, missing values and duplicate records of each table to be examined.

```
In [4]: # =====
# LOAD THE RAW OLIST DATASETS
# =====

def load_olist_datasets(data_directory, file_mapping):
    """
    Load all Olist CSV files into a dictionary of Pandas DataFrames.

    Parameters
    -----
    data_directory : pathlib.Path
        Directory containing the raw CSV files.

    file_mapping : dict
        Dictionary mapping short dataset names to CSV filenames.

    Returns
    -----
    dict
        Dictionary containing one DataFrame for each CSV file.
    """

    loaded_datasets = {}

    for dataset_name, file_name in file_mapping.items():
        file_path = data_directory / file_name

        loaded_datasets[dataset_name] = pd.read_csv(
            file_path,
            low_memory=False
```

```
)

print(
    f"Loaded {dataset_name:<25} "
    f"{loaded_datasets[dataset_name].shape[0]:>10,} rows × "
    f"{loaded_datasets[dataset_name].shape[1]:>2} columns"
)

return loaded_datasets

# Load all nine raw datasets
datasets = load_olist_datasets(
    data_directory=RAW_DATA_DIRECTORY,
    file_mapping=required_files
)
```

Loaded customers 99,441 rows × 5 columns
Loaded geolocation 1,000,163 rows × 5 columns
Loaded order_items 112,650 rows × 7 columns
Loaded payments 103,886 rows × 5 columns
Loaded reviews 99,224 rows × 7 columns
Loaded orders 99,441 rows × 8 columns
Loaded products 32,951 rows × 9 columns
Loaded sellers 3,095 rows × 4 columns
Loaded category_translation 71 rows × 2 columns

```
In [5]: # =====
# CREATE DESCRIPTIVE DATAFRAME VARIABLES
# =====

customers_df = datasets["customers"].copy()
geolocation_df = datasets["geolocation"].copy()
order_items_df = datasets["order_items"].copy()
payments_df = datasets["payments"].copy()
reviews_df = datasets["reviews"].copy()
orders_df = datasets["orders"].copy()
products_df = datasets["products"].copy()
sellers_df = datasets["sellers"].copy()
category_translation_df = datasets["category_translation"].copy()

print("DataFrame variables created successfully.")
```

DataFrame variables created successfully.

3.1 Initial Dataset Audit

Before preprocessing, each table is examined for its dimensions, memory usage, duplicate rows and missing values. This audit provides evidence of the dataset's original condition and identifies the preprocessing operations required later in the project.

```
In [6]: # =====
# DATASET-LEVEL QUALITY AUDIT
# =====

def create_dataset_audit(dataframes):
```

```
"""
```

```
Produce a dataset-level quality report.
```

```
The report includes:
```

- number of rows
- number of columns
- duplicate rows
- missing cells
- missing-cell percentage
- memory usage

```
"""
```

```
audit_records = []
```

```
for dataset_name, dataframe in dataframes.items():  
    total_cells = dataframe.shape[0] * dataframe.shape[1]  
    missing_cells = int(dataframe.isna().sum().sum())  
    duplicate_rows = int(dataframe.duplicated().sum())
```

```
    missing_percentage = (  
        missing_cells / total_cells * 100  
        if total_cells > 0  
        else 0  
    )
```

```
    memory_megabytes = (  
        dataframe.memory_usage(deep=True).sum()  
        / (1024 ** 2)  
    )
```

```
    audit_records.append(  
        {  
            "dataset": dataset_name,  
            "rows": dataframe.shape[0],  
            "columns": dataframe.shape[1],  
            "duplicate_rows": duplicate_rows,  
            "missing_cells": missing_cells,  
            "missing_percentage": missing_percentage,  
            "memory_mb": memory_megabytes  
        }  
    )
```

```
audit_df = pd.DataFrame(audit_records)
```

```
return audit_df.sort_values(  
    by="rows",  
    ascending=False  
) .reset_index(drop=True)
```

```
dataset_audit_df = create_dataset_audit(datasets)
```

```
dataset_audit_df
```

Out [6]:

	dataset	rows	columns	duplicate_rows	missing_cells	missing_percentage	memory_mb
0	geolocation	1000163	5	261831	0	0.000	146.086
1	order_items	112650	7	0	0	0.000	39.427
2	payments	103886	5	0	0	0.000	17.815
3	customers	99441	5	0	0	0.000	29.621
4	orders	99441	8	0	4908	0.617	58.969
5	reviews	99224	7	0	145903	21.006	42.747
6	products	32951	9	0	2448	0.825	6.795
7	sellers	3095	4	0	0	0.000	0.659
8	category_translation	71	2	0	0	0.000	0.010

In [7]:

```
# Save the initial audit for reproducibility and later reporting
dataset_audit_path = (
    PROCESSED_DATA_DIRECTORY
    / "initial_dataset_audit.csv"
)

dataset_audit_df.to_csv(
    dataset_audit_path,
    index=False
)

print(f"Initial audit saved to:\n{dataset_audit_path}")
```

Initial audit saved to:
/Users/zimmeljavedvirk/Desktop/datasciencemid/data/processed/initial_dataset_audit.csv

3.2 Column-Level Quality Audit

A more detailed audit is performed for every variable. The number of unique values, missing observations, missing-value percentage and detected data type are recorded. This helps identify variables requiring type conversion, imputation, removal or further investigation.

In [8]:

```
# =====
# COLUMN-LEVEL QUALITY AUDIT
# =====

def create_column_audit(dataframes):
    """
    Create a detailed quality report for every column
    in every dataset.
    """

    column_records = []

    for dataset_name, dataframe in dataframes.items():
        number_of_rows = len(dataframe)
```

```
for column_name in dataframe.columns:
    missing_count = int(
        dataframe[column_name].isna().sum()
    )

    missing_percentage = (
        missing_count / number_of_rows * 100
        if number_of_rows > 0
        else 0
    )

    column_records.append(
        {
            "dataset": dataset_name,
            "column": column_name,
            "data_type": str(
                dataframe[column_name].dtype
            ),
            "unique_values": dataframe[
                column_name
            ].nunique(dropna=True),
            "missing_values": missing_count,
            "missing_percentage": missing_percentage
        }
    )

return pd.DataFrame(column_records)
```

```
column_audit_df = create_column_audit(datasets)
```

```
column_audit_df.head(20)
```

Out [8]:

	dataset	column	data_type	unique_values	missing_values	missing_percentage
0	customers	customer_id	str	99441	0	0.000
1	customers	customer_unique_id	str	96096	0	0.000
2	customers	customer_zip_code_prefix	int64	14994	0	0.000
3	customers	customer_city	str	4119	0	0.000
4	customers	customer_state	str	27	0	0.000
5	geolocation	geolocation_zip_code_prefix	int64	19015	0	0.000
6	geolocation	geolocation_lat	float64	717363	0	0.000
7	geolocation	geolocation_lng	float64	717615	0	0.000
8	geolocation	geolocation_city	str	8011	0	0.000
9	geolocation	geolocation_state	str	27	0	0.000
10	order_items	order_id	str	98666	0	0.000
11	order_items	order_item_id	int64	21	0	0.000
12	order_items	product_id	str	32951	0	0.000
13	order_items	seller_id	str	3095	0	0.000
14	order_items	shipping_limit_date	str	93318	0	0.000
15	order_items	price	float64	5968	0	0.000
16	order_items	freight_value	float64	6999	0	0.000
17	payments	order_id	str	99440	0	0.000
18	payments	payment_sequential	int64	29	0	0.000
19	payments	payment_type	str	5	0	0.000

In [9]:

```
# Display variables that contain at least one missing value
columns_with_missing_values_df = (
    column_audit_df[
        column_audit_df["missing_values"] > 0
    ]
    .sort_values(
        by=["dataset", "missing_percentage"],
        ascending=[True, False]
    )
    .reset_index(drop=True)
)

columns_with_missing_values_df
```

Out [9]:

	dataset	column	data_type	unique_values	missing_values	missing_percentage
0	orders	order_delivered_customer_date	str	95664	2965	2.982
1	orders	order_delivered_carrier_date	str	81018	1783	1.793
2	orders	order_approved_at	str	90733	160	0.161
3	products	product_category_name	str	73	610	1.851
4	products	product_name_lenght	float64	66	610	1.851
5	products	product_description_lenght	float64	2960	610	1.851
6	products	product_photos_qty	float64	19	610	1.851
7	products	product_weight_g	float64	2204	2	0.006
8	products	product_length_cm	float64	99	2	0.006
9	products	product_height_cm	float64	102	2	0.006
10	products	product_width_cm	float64	95	2	0.006
11	reviews	review_comment_title	str	4527	87656	88.342
12	reviews	review_comment_message	str	36159	58247	58.703

In [10]:

```
column_audit_path = (
    PROCESSED_DATA_DIRECTORY
    / "initial_column_audit.csv"
)

column_audit_df.to_csv(
    column_audit_path,
    index=False
)

print(f"Column audit saved to:\n{column_audit_path}")
```

Column audit saved to:
/Users/zimmeljavedvirk/Desktop/datasciencemid/data/processed/initial_column_audit.csv

In [11]:

```
# =====
# PREVIEW ALL RAW DATASETS
# =====

for dataset_name, dataframe in datasets.items():
    print("\n" + "=" * 80)
    print(f"DATASET: {dataset_name.upper()}")
    print(f"SHAPE: {dataframe.shape}")
    print("=" * 80)

    display(dataframe.head(3))
```

```
=====
DATASET: CUSTOMERS
SHAPE: (99441, 5)
=====
```

	customer_id	customer_unique_id	customer_zip_code_prefix	customer_city	customer_state
0	06b8999e2fba1a1fbc88172c00ba8bc7	861eff4711a542e4b93843c6dd7febb0	14409	franca	SP
1	18955e83d337fd6b2def6b18a428ac77	290c77bc529b7ac935b93aa66c333dc3	9790	sao bernardo do campo	SP
2	4e7b3e00288586ebd08712fdd0374a03	060e732b5b29e8181a18229c7b0b2b5e	1151	sao paulo	SP

=====

DATASET: GEOLLOCATION

SHAPE: (1000163, 5)

=====

	geolocation_zip_code_prefix	geolocation_lat	geolocation_lng	geolocation_city	geolocation_state
0	1037	-23.546	-46.639	sao paulo	SP
1	1046	-23.546	-46.645	sao paulo	SP
2	1046	-23.546	-46.643	sao paulo	SP

=====

DATASET: ORDER_ITEMS

SHAPE: (112650, 7)

=====

	order_id	order_item_id	product_id	seller_id	shipping_limit_date	price	freight_value
0	00010242fe8c5a6d1ba2dd792cb16214	1	4244733e06e7ecb4970a6e2683c13e61	48436dade18ac8b2bce089ec2a041202	2017-09-19 09:45:35	58.900	13.290
1	00018f77f2f0320c557190d7a144bdd3	1	e5f2d52b802189ee658865ca93d83a8f	dd7ddc04e1b6c2c614352b383efe2d36	2017-05-03 11:05:13	239.900	19.930
2	000229ec398224ef6ca0657da4fc703e	1	c777355d18b72b67abbef9df44fd0fd	5b51032eddd242adc84c38acab88f23d	2018-01-18 14:48:30	199.000	17.870

=====

DATASET: PAYMENTS

SHAPE: (103886, 5)

=====

	order_id	payment_sequential	payment_type	payment_installments	payment_value
0	b81ef226f3fe1789b1e8b2acac839d17	1	credit_card	8	99.330
1	a9810da82917af2d9aefd1278f1dcfa0	1	credit_card	1	24.390
2	25e8ea4e93396b6fa0d3dd708e76c1bd	1	credit_card	1	65.710

=====

DATASET: REVIEWS

SHAPE: (99224, 7)

=====

	review_id	order_id	review_score	review_comment_title	review_comment_message	review_creation_date	review_answer_timestamp
0	7bc2406110b926393aa56f80a40eba40	73fc7af87114b39712e6da79b0a377eb	4	NaN	NaN	2018-01-18 00:00:00	2018-01-18 21:46
1	80e641a11e56f04c1ad469d5645fdfde	a548910a1c6147796b98fdf73dbeba33	5	NaN	NaN	2018-03-10 00:00:00	2018-03-11 03:05
2	228ce5500dc1d8e020d8d1322874b6f0	f9e4b658b201a9f2ecdecbb34bed034b	5	NaN	NaN	2018-02-17 00:00:00	2018-02-18 14:36

=====
DATASET: ORDERS
SHAPE: (99441, 8)
=====

	order_id	customer_id	order_status	order_purchase_timestamp	order_approved_at	order_delivered_carrier_date	order_delivered_c
0	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-10-02 10:56:33	2017-10-02 11:07:15	2017-10-04 19:55:00	2017-
1	53cdb2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef	delivered	2018-07-24 20:41:37	2018-07-26 03:24:27	2018-07-26 14:31:00	2018-(
2	47770eb9100c2d0c44946d9cf07ec65d	41ce2a54c0b03bf3443c3d931a367089	delivered	2018-08-08 08:38:49	2018-08-08 08:55:23	2018-08-08 13:50:00	2018-(

=====
DATASET: PRODUCTS
SHAPE: (32951, 9)
=====

	product_id	product_category_name	product_name_lenght	product_description_lenght	product_photos_qty	product_weight_g	product_length_cm	product_h
0	1e9e8ef04dbcff4541ed26657ea517e5	perfumaria	40.000	287.000	1.000	225.000	16.000	
1	3aa071139cb16b67ca9e5dea641aaa2f	artes	44.000	276.000	1.000	1,000.000	30.000	
2	96bd76ec8810374ed1b65e291975717f	esporte_lazer	46.000	250.000	1.000	154.000	18.000	

=====
DATASET: SELLERS
SHAPE: (3095, 4)
=====

	seller_id	seller_zip_code_prefix	seller_city	seller_state
0	3442f8959a84dea7ee197c632cb2df15	13023	campinas	SP
1	d1b65fc7debc3361ea86b5f14c68d2e2	13844	mogi guacu	SP
2	ce3ad9de960102d0677a81f5d0bb7b2d	20031	rio de janeiro	RJ

=====
DATASET: CATEGORY_TRANSLATION
SHAPE: (71, 2)
=====

	product_category_name	product_category_name_english
0	beleza_saude	health_beauty
1	informatica_acessorios	computers_accessories
2	automotivo	auto

3.3 Relational Integrity and Key Analysis

The Olist dataset is relational rather than a single modelling table. The files are connected through identifiers such as order_id , customer_id , product_id and seller_id .

Before merging the tables, candidate primary keys, composite keys and foreign-key relationships are examined. This is necessary because several tables contain one-to-many relationships. For

example, one order may contain multiple products and multiple payment records. An uncontrolled merge could therefore multiply rows and produce invalid descriptive statistics and model results.

```
In [12]: # =====
# CANDIDATE PRIMARY-KEY AND COMPOSITE-KEY AUDIT
# =====

def audit_candidate_key(
    dataframe,
    dataset_name,
    key_columns,
    key_purpose
):
    """
    Examine whether one or more columns can uniquely identify rows.

    Parameters
    -----
    dataframe : pandas.DataFrame
        Dataset being examined.

    dataset_name : str
        Descriptive dataset name.

    key_columns : list
        Column or columns forming the candidate key.

    key_purpose : str
        Explanation of the role of the key.

    Returns
    -----
    dict
        Candidate-key quality statistics.
    """
    missing_key_rows = int(
        dataframe[key_columns].isna().any(axis=1).sum()
    )

    unique_key_combinations = int(
        dataframe[key_columns]
        .drop_duplicates()
        .shape[0]
    )

    duplicate_excess_rows = int(
        dataframe.duplicated(
            subset=key_columns,
            keep="first"
        ).sum()
    )

    all_duplicated_key_rows = int(
        dataframe.duplicated(
```

```
        subset=key_columns,
        keep=False
    ).sum()
)

is_unique_and_complete = (
    missing_key_rows == 0
    and unique_key_combinations == len(dataframe)
)

return {
    "dataset": dataset_name,
    "key_columns": " + ".join(key_columns),
    "key_purpose": key_purpose,
    "total_rows": len(dataframe),
    "unique_key_combinations": unique_key_combinations,
    "missing_key_rows": missing_key_rows,
    "duplicate_excess_rows": duplicate_excess_rows,
    "all_duplicated_key_rows": all_duplicated_key_rows,
    "valid_unique_key": is_unique_and_complete
}

candidate_key_specifications = [
    (
        customers_df,
        "customers",
        ["customer_id"],
        "Candidate customer primary key"
    ),
    (
        sellers_df,
        "sellers",
        ["seller_id"],
        "Candidate seller primary key"
    ),
    (
        products_df,
        "products",
        ["product_id"],
        "Candidate product primary key"
    ),
    (
        orders_df,
        "orders",
        ["order_id"],
        "Candidate order primary key"
    ),
    (
        order_items_df,
        "order_items",
        ["order_id", "order_item_id"],
        "Composite key identifying an item within an order"
    ),
    (
        payments_df,
```

```
        "payments",
        ["order_id", "payment_sequential"],
        "Composite key identifying a payment within an order"
    ),
    (
        reviews_df,
        "reviews",
        ["review_id"],
        "Candidate review identifier"
    ),
    (
        category_translation_df,
        "category_translation",
        ["product_category_name"],
        "Candidate Portuguese category key"
    )
]
```

```
candidate_key_records = []
```

```
for (
    dataframe,
    dataset_name,
    key_columns,
    key_purpose
) in candidate_key_specifications:

    candidate_key_records.append(
        audit_candidate_key(
            dataframe=dataframe,
            dataset_name=dataset_name,
            key_columns=key_columns,
            key_purpose=key_purpose
        )
    )
```

```
candidate_key_audit_df = pd.DataFrame(
    candidate_key_records
)
```

```
candidate_key_audit_df
```

Out[12]:

	dataset	key_columns	key_purpose	total_rows	unique_key_combinations	missing_key_rows	duplicate_excess_rows	all_duplicated_key_rows	valid_unique_key
0	customers	customer_id	Candidate customer primary key	99441	99441	0	0	0	True
1	sellers	seller_id	Candidate seller primary key	3095	3095	0	0	0	True
2	products	product_id	Candidate product primary key	32951	32951	0	0	0	True
3	orders	order_id	Candidate order primary key	99441	99441	0	0	0	True
4	order_items	order_id + order_item_id	Composite key identifying an item within an order	112650	112650	0	0	0	True
5	payments	order_id + payment_sequential	Composite key identifying a payment within an ...	103886	103886	0	0	0	True
6	reviews	review_id	Candidate review identifier	99224	98410	0	814	1603	False
7	category_translation	product_category_name	Candidate Portuguese category key	71	71	0	0	0	True

3.4 Foreign-Key Integrity

Foreign-key integrity is examined to determine whether child-table identifiers have matching records in their respective parent tables. Missing parent records could cause observations to disappear during inner joins or produce incomplete values during left joins.

In [13]:

```
# =====  
# FOREIGN-KEY INTEGRITY AUDIT  
# =====  
  
def audit_foreign_key(  
    child_dataframe,  
    child_key,  
    parent_dataframe,  
    parent_key,  
    relationship_name  
):  
    """  
    Check whether values in a child-table foreign key exist  
    in the corresponding parent-table key.
```

```
#####

child_values = child_dataframe[child_key].dropna()
parent_values = set(
    parent_dataframe[parent_key]
    .dropna()
    .unique()
)

unmatched_mask = ~child_values.isin(parent_values)

unmatched_rows = int(unmatched_mask.sum())

unmatched_unique_values = int(
    child_values[unmatched_mask].nunique()
)

number_of_child_values = len(child_values)

match_rate = (
    (
        number_of_child_values - unmatched_rows
    )
    / number_of_child_values
    * 100
    if number_of_child_values > 0
    else np.nan
)

return {
    "relationship": relationship_name,
    "child_key": child_key,
    "parent_key": parent_key,
    "child_non_missing_rows": number_of_child_values,
    "child_unique_keys": child_values.nunique(),
    "unmatched_rows": unmatched_rows,
    "unmatched_unique_keys": unmatched_unique_values,
    "match_rate_percentage": match_rate
}

foreign_key_specifications = [
    (
        orders_df,
        "customer_id",
        customers_df,
        "customer_id",
        "orders → customers"
    ),
    (
        order_items_df,
        "order_id",
        orders_df,
        "order_id",
        "order_items → orders"
    ),
]
```

```
(
    order_items_df,
    "product_id",
    products_df,
    "product_id",
    "order_items → products"
),
(
    order_items_df,
    "seller_id",
    sellers_df,
    "seller_id",
    "order_items → sellers"
),
(
    payments_df,
    "order_id",
    orders_df,
    "order_id",
    "payments → orders"
),
(
    reviews_df,
    "order_id",
    orders_df,
    "order_id",
    "reviews → orders"
),
(
    products_df,
    "product_category_name",
    category_translation_df,
    "product_category_name",
    "products → category translation"
),
(
    customers_df,
    "customer_zip_code_prefix",
    geolocation_df,
    "geolocation_zip_code_prefix",
    "customers → geolocation"
),
(
    sellers_df,
    "seller_zip_code_prefix",
    geolocation_df,
    "geolocation_zip_code_prefix",
    "sellers → geolocation"
)
]

foreign_key_records = []

for (
    child_dataframe,
```

```
child_key,
parent_dataframe,
parent_key,
relationship_name
) in foreign_key_specifications:

    foreign_key_records.append(
        audit_foreign_key(
            child_dataframe=child_dataframe,
            child_key=child_key,
            parent_dataframe=parent_dataframe,
            parent_key=parent_key,
            relationship_name=relationship_name
        )
    )

foreign_key_audit_df = pd.DataFrame(
    foreign_key_records
)

foreign_key_audit_df
```

Out[13]:

	relationship	child_key	parent_key	child_non_missing_rows	child_unique_keys	unmatched_rows	unmatched_unique_keys	match_rate_percentage
0	orders → customers	customer_id	customer_id	99441	99441	0	0	100.000
1	order_items → orders	order_id	order_id	112650	98666	0	0	100.000
2	order_items → products	product_id	product_id	112650	32951	0	0	100.000
3	order_items → sellers	seller_id	seller_id	112650	3095	0	0	100.000
4	payments → orders	order_id	order_id	103886	99440	0	0	100.000
5	reviews → orders	order_id	order_id	99224	98673	0	0	100.000
6	products → category translation	product_category_name	product_category_name	32341	73	13	2	99.960
7	customers → geolocation	customer_zip_code_prefix	geolocation_zip_code_prefix	99441	14994	278	157	99.720
8	sellers → geolocation	seller_zip_code_prefix	geolocation_zip_code_prefix	3095	2246	7	7	99.774

3.5 Relationship Multiplicity

Several Olist tables contain repeated business identifiers because one order can contain multiple order items, payment transactions or review records. The multiplicity of these identifiers is measured before merging to prevent accidental many-to-many joins.

```

In [14]: # =====
# ONE-TO-MANY RELATIONSHIP MULTIPLICITY
# =====

def create_multiplicity_record(
    dataframe,
    dataset_name,
    grouping_key
):
    """
    Summarise how many rows occur for each business identifier.
    """

    rows_per_key = (
        dataframe
        .groupby(grouping_key, dropna=False)
        .size()
    )

    return {
        "dataset": dataset_name,
        "grouping_key": grouping_key,
        "total_rows": len(dataframe),
        "unique_keys": dataframe[grouping_key].nunique(
            dropna=True
        ),
        "keys_with_multiple_rows": int(
            (rows_per_key > 1).sum()
        ),
        "mean_rows_per_key": rows_per_key.mean(),
        "median_rows_per_key": rows_per_key.median(),
        "95th_percentile_rows": rows_per_key.quantile(0.95),
        "maximum_rows_per_key": rows_per_key.max()
    }

multiplicity_specifications = [
    (
        order_items_df,
        "order_items",
        "order_id"
    ),
    (
        payments_df,
        "payments",
        "order_id"
    ),
    (
        reviews_df,
        "reviews",
        "order_id"
    ),
    (
        geolocation_df,
        "geolocation",

```

```
        "geolocation_zip_code_prefix"
    )
]

multiplicity_records = []

for (
    dataframe,
    dataset_name,
    grouping_key
) in multiplicity_specifications:

    multiplicity_records.append(
        create_multiplicity_record(
            dataframe=dataframe,
            dataset_name=dataset_name,
            grouping_key=grouping_key
        )
    )

relationship_multiplicity_df = pd.DataFrame(
    multiplicity_records
)

relationship_multiplicity_df
```

Out[14]:

	dataset	grouping_key	total_rows	unique_keys	keys_with_multiple_rows	mean_rows_per_key	median_rows_per_key	95th_percentile_rows	maximum_rows_per_key
0	order_items	order_id	112650	98666	9803	1.142	1.000	2.000	2
1	payments	order_id	103886	99440	2961	1.045	1.000	1.000	29
2	reviews	order_id	99224	98673	547	1.006	1.000	1.000	3
3	geolocation	geolocation_zip_code_prefix	1000163	19015	17972	52.599	29.000	180.000	1146

```
In [15]: # =====
# GEOLOCATION DUPLICATION ANALYSIS
# =====

geolocation_exact_duplicates = int(
    geolocation_df.duplicated().sum()
)

geolocation_unique_zip_prefixes = int(
    geolocation_df[
        "geolocation_zip_code_prefix"
    ].nunique()
)

geolocation_duplicated_zip_rows = int(
    geolocation_df.duplicated(
        subset=["geolocation_zip_code_prefix"],
        keep=False
    )
```

```

    ).sum()
)

print("Geolocation quality summary")
print("=" * 60)

print(
    f"Total geolocation rows: "
    f"{len(geolocation_df):,}"
)

print(
    f"Exact duplicate rows: "
    f"{geolocation_exact_duplicates:,}"
)

print(
    f"Unique ZIP-code prefixes: "
    f"{geolocation_unique_zip_prefixes:,}"
)

print(
    f"Rows belonging to repeated ZIP prefixes: "
    f"{geolocation_duplicated_zip_rows:,}"
)

print("=" * 60)

```

Geolocation quality summary

```

=====
Total geolocation rows: 1,000,163
Exact duplicate rows: 261,831
Unique ZIP-code prefixes: 19,015
Rows belonging to repeated ZIP prefixes: 999,120
=====

```

```

In [16]: # =====
# SAVE RELATIONAL-INTEGRITY REPORTS
# =====

candidate_key_audit_path = (
    PROCESSED_DATA_DIRECTORY
    / "candidate_key_audit.csv"
)

foreign_key_audit_path = (
    PROCESSED_DATA_DIRECTORY
    / "foreign_key_audit.csv"
)

relationship_multiplicity_path = (
    PROCESSED_DATA_DIRECTORY
    / "relationship_multiplicity.csv"
)

```

```
candidate_key_audit_df.to_csv(
    candidate_key_audit_path,
    index=False
)

foreign_key_audit_df.to_csv(
    foreign_key_audit_path,
    index=False
)

relationship_multiplicity_df.to_csv(
    relationship_multiplicity_path,
    index=False
)

print("Relational-integrity reports saved successfully:")
print(candidate_key_audit_path)
print(foreign_key_audit_path)
print(relationship_multiplicity_path)
```

Relational-integrity reports saved successfully:
/Users/zimmeljavedvirk/Desktop/datasciencemid/data/processed/candidate_key_audit.csv
/Users/zimmeljavedvirk/Desktop/datasciencemid/data/processed/foreign_key_audit.csv
/Users/zimmeljavedvirk/Desktop/datasciencemid/data/processed/relationship_multiplicity.csv

Initial Relational-Audit Interpretation

The relational audit confirmed that the Olist files are connected through customer, order, product and seller identifiers. The customers, sellers, products and orders tables each contain complete and unique identifiers, while the composite keys in the order-items and payments tables also uniquely identify their rows. These properties permit controlled many-to-one joins when constructing the analytical dataset.

However, several important data-quality risks were identified. Orders may contain multiple items, payment transactions and review records. Directly joining these one-to-many tables could therefore create many-to-many relationships and artificially multiply observations. The final analytical table will use one order item as its unit of analysis. Payments and reviews will not be directly merged into the primary regression table unless they are first aggregated to an appropriate level.

The review identifier is not fully unique, demonstrating that the reviews table requires deduplication or a defined record-selection rule. The category-translation table is unique but does not cover two product categories, so untranslated categories will retain their original names rather than being removed.

The geolocation table contains 261,831 exact duplicate rows and multiple observations for most ZIP-code prefixes. It must therefore be cleaned and aggregated to one representative coordinate per ZIP-code prefix before being joined to customers and sellers. A small number of customer and seller ZIP-code prefixes are absent from the geolocation table, so geographical fallback rules will be considered instead of deleting these records automatically.

All later merges will use the Pandas `validate` argument to enforce the expected relationship type and detect unintended row multiplication.

4. Data Preparation and Sanitisation

The raw Olist files require several preprocessing operations before they can be combined into a regression-ready dataset. The preparation process includes preserving immutable copies of the raw data, sanitising textual values, converting timestamp strings to datetime objects, treating exact duplicate rows, identifying invalid numerical values, translating product categories and consolidating repeated geographical observations.

Missing values are not removed automatically. Each missing value is considered according to its meaning and intended use. For example, missing delivery dates may represent cancelled or undelivered orders rather than data-entry errors. Similarly, missing numerical product measurements will be handled inside the machine-learning pipeline to avoid information leakage between the training and testing datasets.

```
In [17]: # =====
# CREATE CLEANED WORKING COPIES
# =====

def sanitise_string_columns(dataframe):
    """
    Strip leading and trailing whitespace from textual columns
    and convert empty strings into missing values.

    Parameters
    -----
    dataframe : pandas.DataFrame
        Input dataset.

    Returns
    -----
    pandas.DataFrame
        Dataset with sanitised textual columns.
    """

    sanitised_dataframe = dataframe.copy(deep=True)

    string_columns = sanitised_dataframe.select_dtypes(
        include=["object", "string"]
    ).columns

    for column_name in string_columns:
        sanitised_dataframe[column_name] = (
            sanitised_dataframe[column_name]
            .astype("string")
            .str.strip()
            .replace(r"^\s*$", pd.NA, regex=True)
        )

    return sanitised_dataframe

# Preserve the original raw DataFrames and create separate cleaned copies
cleaned_datasets = {
    dataset_name: sanitise_string_columns(dataframe)
    for dataset_name, dataframe in datasets.items()
}

# Create convenient cleaned DataFrame variables
customers_clean_df = cleaned_datasets["customers"]
geolocation_clean_df = cleaned_datasets["geolocation"]
order_items_clean_df = cleaned_datasets["order_items"]
payments_clean_df = cleaned_datasets["payments"]
reviews_clean_df = cleaned_datasets["reviews"]
orders_clean_df = cleaned_datasets["orders"]
products_clean_df = cleaned_datasets["products"]
```

```

sellers_clean_df = cleaned_datasets["sellers"]
category_translation_clean_df = cleaned_datasets[
    "category_translation"
]

# Standardise Brazilian state codes
customers_clean_df["customer_state"] = (
    customers_clean_df["customer_state"].str.upper()
)

sellers_clean_df["seller_state"] = (
    sellers_clean_df["seller_state"].str.upper()
)

geolocation_clean_df["geolocation_state"] = (
    geolocation_clean_df["geolocation_state"].str.upper()
)

# Standardise product category naming
products_clean_df["product_category_name"] = (
    products_clean_df["product_category_name"].str.lower()
)

category_translation_clean_df["product_category_name"] = (
    category_translation_clean_df[
        "product_category_name"
    ].str.lower()
)

print("Cleaned working copies created successfully.")
print("The original raw DataFrames remain unchanged.")

```

Cleaned working copies created successfully.
The original raw DataFrames remain unchanged.

4.1 Datetime Conversion

The raw timestamp variables were initially imported as strings. They are converted into Pandas datetime objects so that temporal features, durations and chronological validity checks can later be calculated correctly.

Invalid timestamp strings are converted to missing datetime values using `errors="coerce"`. A conversion audit records whether any originally populated values failed to parse.

```

In [18]: # =====
# DATETIME CONVERSION AND AUDIT
# =====

datetime_column_mapping = {
    "orders": [
        "order_purchase_timestamp",
        "order_approved_at",
        "order_delivered_carrier_date",
        "order_delivered_customer_date",
        "order_estimated_delivery_date"
    ],
    "order_items": [

```

```
        "shipping_limit_date"
    ],
    "reviews": [
        "review_creation_date",
        "review_answer_timestamp"
    ]
}

def convert_datetime_columns(
    dataframe,
    dataset_name,
    datetime_columns
):
    """
    Convert selected columns to datetime and record parsing failures.
    """

    converted_dataframe = dataframe.copy()
    conversion_records = []

    for column_name in datetime_columns:
        original_non_missing = int(
            converted_dataframe[column_name].notna().sum()
        )

        converted_values = pd.to_datetime(
            converted_dataframe[column_name],
            format="%Y-%m-%d %H:%M:%S",
            errors="coerce"
        )

        converted_non_missing = int(
            converted_values.notna().sum()
        )

        parsing_failures = (
            original_non_missing - converted_non_missing
        )

        converted_dataframe[column_name] = converted_values

        conversion_records.append(
            {
                "dataset": dataset_name,
                "column": column_name,
                "original_non_missing": original_non_missing,
                "converted_non_missing": converted_non_missing,
                "parsing_failures": parsing_failures,
                "final_data_type": str(
                    converted_dataframe[column_name].dtype
                )
            }
        )

    return converted_dataframe, conversion_records
```

```
datetime_conversion_records = []

orders_clean_df, records = convert_datetime_columns(
    dataframe=orders_clean_df,
    dataset_name="orders",
    datetime_columns=datetime_column_mapping["orders"]
)
datetime_conversion_records.extend(records)

order_items_clean_df, records = convert_datetime_columns(
    dataframe=order_items_clean_df,
    dataset_name="order_items",
    datetime_columns=datetime_column_mapping["order_items"]
)
datetime_conversion_records.extend(records)

reviews_clean_df, records = convert_datetime_columns(
    dataframe=reviews_clean_df,
    dataset_name="reviews",
    datetime_columns=datetime_column_mapping["reviews"]
)
datetime_conversion_records.extend(records)

datetime_conversion_audit_df = pd.DataFrame(
    datetime_conversion_records
)

datetime_conversion_audit_df
```

Out[18]:

	dataset	column	original_non_missing	converted_non_missing	parsing_failures	final_data_type
0	orders	order_purchase_timestamp	99441	99441	0	datetime64[us]
1	orders	order_approved_at	99281	99281	0	datetime64[us]
2	orders	order_delivered_carrier_date	97658	97658	0	datetime64[us]
3	orders	order_delivered_customer_date	96476	96476	0	datetime64[us]
4	orders	order_estimated_delivery_date	99441	99441	0	datetime64[us]
5	order_items	shipping_limit_date	112650	112650	0	datetime64[us]
6	reviews	review_creation_date	99224	99224	0	datetime64[us]
7	reviews	review_answer_timestamp	99224	99224	0	datetime64[us]

In [19]:

```
# =====
# CHRONOLOGICAL VALIDITY CHECKS
# =====

chronological_checks = {
    "approval_before_purchase": (
        orders_clean_df["order_approved_at"]
        < orders_clean_df["order_purchase_timestamp"]
    )
}
```

```
),
"carrier_before_purchase": (
    orders_clean_df["order_delivered_carrier_date"]
    < orders_clean_df["order_purchase_timestamp"]
),
"customer_delivery_before_purchase": (
    orders_clean_df["order_delivered_customer_date"]
    < orders_clean_df["order_purchase_timestamp"]
),
"customer_delivery_before_carrier": (
    orders_clean_df["order_delivered_customer_date"]
    < orders_clean_df["order_delivered_carrier_date"]
),
"estimated_delivery_before_purchase": (
    orders_clean_df["order_estimated_delivery_date"]
    < orders_clean_df["order_purchase_timestamp"]
)
)
}

chronological_audit_records = []

for check_name, condition in chronological_checks.items():
    chronological_audit_records.append(
        {
            "validity_check": check_name,
            "affected_rows": int(
                condition.fillna(False).sum()
            ),
            "affected_percentage": (
                condition.fillna(False).mean() * 100
            )
        }
    )

chronological_audit_df = pd.DataFrame(
    chronological_audit_records
)

chronological_audit_df
```

Out[19]:

	validity_check	affected_rows	affected_percentage
0	approval_before_purchase	0	0.000
1	carrier_before_purchase	166	0.167
2	customer_delivery_before_purchase	0	0.000
3	customer_delivery_before_carrier	23	0.023
4	estimated_delivery_before_purchase	0	0.000

4.2 Exact-Duplicate Treatment

Exact duplicate rows can distort summary statistics and increase the influence of repeated observations. Exact duplicates are removed from the cleaned working copies while the number of

affected records is documented.

This operation is particularly important for the geolocation file, which contains many repeated observations. Business-key duplicates are not removed automatically because repeated order, payment or review identifiers may represent legitimate one-to-many relationships.

```
In [20]: # =====
# EXACT-DUPLICATE REMOVAL
# =====

cleaned_dataframe_mapping = {
    "customers": customers_clean_df,
    "geolocation": geolocation_clean_df,
    "order_items": order_items_clean_df,
    "payments": payments_clean_df,
    "reviews": reviews_clean_df,
    "orders": orders_clean_df,
    "products": products_clean_df,
    "sellers": sellers_clean_df,
    "category_translation": category_translation_clean_df
}

duplicate_removal_records = []
deduplicated_dataframes = {}

for dataset_name, dataframe in cleaned_dataframe_mapping.items():

    rows_before = len(dataframe)
    exact_duplicates = int(dataframe.duplicated().sum())

    deduplicated_dataframe = (
        dataframe
        .drop_duplicates()
        .reset_index(drop=True)
    )

    rows_after = len(deduplicated_dataframe)

    duplicate_removal_records.append(
        {
            "dataset": dataset_name,
            "rows_before": rows_before,
            "exact_duplicates_removed": exact_duplicates,
            "rows_after": rows_after,
            "percentage_removed": (
                exact_duplicates / rows_before * 100
                if rows_before > 0
                else 0
            )
        }
    )

    deduplicated_dataframes[
        dataset_name
    ] = deduplicated_dataframe
```

```
duplicate_removal_audit_df = pd.DataFrame(  
    duplicate_removal_records  
)  
  
duplicate_removal_audit_df
```

Out[20]:

	dataset	rows_before	exact_duplicates_removed	rows_after	percentage_removed
0	customers	99441	0	99441	0.000
1	geolocation	1000163	261831	738332	26.179
2	order_items	112650	0	112650	0.000
3	payments	103886	0	103886	0.000
4	reviews	99224	0	99224	0.000
5	orders	99441	0	99441	0.000
6	products	32951	0	32951	0.000
7	sellers	3095	0	3095	0.000
8	category_translation	71	0	71	0.000

```
In [21]: # Update cleaned variables using the deduplicated DataFrames  
customers_clean_df = deduplicated_dataframes["customers"]  
geolocation_clean_df = deduplicated_dataframes["geolocation"]  
order_items_clean_df = deduplicated_dataframes["order_items"]  
payments_clean_df = deduplicated_dataframes["payments"]  
reviews_clean_df = deduplicated_dataframes["reviews"]  
orders_clean_df = deduplicated_dataframes["orders"]  
products_clean_df = deduplicated_dataframes["products"]  
sellers_clean_df = deduplicated_dataframes["sellers"]  
category_translation_clean_df = deduplicated_dataframes[  
    "category_translation"  
]  
  
print("Cleaned DataFrame variables updated successfully.")
```

Cleaned DataFrame variables updated successfully.

4.3 Numerical Sanitisation

Numerical columns are explicitly converted using `pd.to_numeric`. Values that cannot be interpreted numerically are converted to missing values. Business rules are then used to identify physically impossible or invalid observations.

Zero or negative product weight and dimensions are considered invalid because they cannot represent a physical parcel. These values are converted to missing values rather than replaced immediately. Their imputation will later be learned from the training data only.

```
In [22]: # =====  
# NUMERICAL CONVERSION  
# =====
```

```
numerical_column_mapping = {
    "products": [
        "product_name_lenght",
        "product_description_lenght",
        "product_photos_qty",
        "product_weight_g",
        "product_length_cm",
        "product_height_cm",
        "product_width_cm"
    ],
    "order_items": [
        "order_item_id",
        "price",
        "freight_value"
    ],
    "payments": [
        "payment_sequential",
        "payment_installments",
        "payment_value"
    ],
    "geolocation": [
        "geolocation_zip_code_prefix",
        "geolocation_lat",
        "geolocation_lng"
    ]
}

def convert_numeric_columns(
    dataframe,
    dataset_name,
    numerical_columns
):
    """
    Convert selected columns to numerical data and audit
    values that fail conversion.
    """

    converted_dataframe = dataframe.copy()
    conversion_records = []

    for column_name in numerical_columns:

        original_non_missing = int(
            converted_dataframe[column_name].notna().sum()
        )

        converted_values = pd.to_numeric(
            converted_dataframe[column_name],
            errors="coerce"
        )

        converted_non_missing = int(
            converted_values.notna().sum()
        )
```

```
        conversion_failures = (
            original_non_missing - converted_non_missing
        )

        converted_dataframe[column_name] = converted_values

        conversion_records.append(
            {
                "dataset": dataset_name,
                "column": column_name,
                "original_non_missing": original_non_missing,
                "converted_non_missing": converted_non_missing,
                "conversion_failures": conversion_failures,
                "final_data_type": str(
                    converted_dataframe[column_name].dtype
                )
            }
        )

    return converted_dataframe, conversion_records


numeric_conversion_records = []

products_clean_df, records = convert_numeric_columns(
    dataframe=products_clean_df,
    dataset_name="products",
    numerical_columns=numerical_column_mapping["products"]
)
numeric_conversion_records.extend(records)

order_items_clean_df, records = convert_numeric_columns(
    dataframe=order_items_clean_df,
    dataset_name="order_items",
    numerical_columns=numerical_column_mapping["order_items"]
)
numeric_conversion_records.extend(records)

payments_clean_df, records = convert_numeric_columns(
    dataframe=payments_clean_df,
    dataset_name="payments",
    numerical_columns=numerical_column_mapping["payments"]
)
numeric_conversion_records.extend(records)

geolocation_clean_df, records = convert_numeric_columns(
    dataframe=geolocation_clean_df,
    dataset_name="geolocation",
    numerical_columns=numerical_column_mapping["geolocation"]
)
numeric_conversion_records.extend(records)

numeric_conversion_audit_df = pd.DataFrame(
    numeric_conversion_records
)
```

numeric_conversion_audit_df

Out[22]:

	dataset	column	original_non_missing	converted_non_missing	conversion_failures	final_data_type
0	products	product_name_lenght	32341	32341	0	float64
1	products	product_description_lenght	32341	32341	0	float64
2	products	product_photos_qty	32341	32341	0	float64
3	products	product_weight_g	32949	32949	0	float64
4	products	product_length_cm	32949	32949	0	float64
5	products	product_height_cm	32949	32949	0	float64
6	products	product_width_cm	32949	32949	0	float64
7	order_items	order_item_id	112650	112650	0	int64
8	order_items	price	112650	112650	0	float64
9	order_items	freight_value	112650	112650	0	float64
10	payments	payment_sequential	103886	103886	0	int64
11	payments	payment_installments	103886	103886	0	int64
12	payments	payment_value	103886	103886	0	float64
13	geolocation	geolocation_zip_code_prefix	738332	738332	0	int64
14	geolocation	geolocation_lat	738332	738332	0	float64
15	geolocation	geolocation_lng	738332	738332	0	float64

In [23]:

```
# =====  
# INVALID NUMERICAL VALUE AUDIT  
# =====  
  
invalid_value_rules = [  
    (  
        "products",  
        "product_weight_g",  
        products_clean_df["product_weight_g"] <= 0,  
        "Product weight must be greater than zero"  
    ),  
    (  
        "products",  
        "product_length_cm",  
        products_clean_df["product_length_cm"] <= 0,  
        "Product length must be greater than zero"  
    ),  
    (  
        "products",  
        "product_height_cm",  
        products_clean_df["product_height_cm"] <= 0,  
        "Product height must be greater than zero"  
    ),  
    (  

```

```
        "products",
        "product_width_cm",
        products_clean_df["product_width_cm"] <= 0,
        "Product width must be greater than zero"
    ),
    (
        "order_items",
        "price",
        order_items_clean_df["price"] < 0,
        "Product price cannot be negative"
    ),
    (
        "order_items",
        "freight_value",
        order_items_clean_df["freight_value"] < 0,
        "Freight cost cannot be negative"
    ),
    (
        "payments",
        "payment_value",
        payments_clean_df["payment_value"] < 0,
        "Payment value cannot be negative"
    )
]

invalid_value_records = []

for (
    dataset_name,
    column_name,
    condition,
    rule_description
) in invalid_value_rules:

    invalid_value_records.append(
        {
            "dataset": dataset_name,
            "column": column_name,
            "invalid_values": int(
                condition.fillna(False).sum()
            ),
            "rule": rule_description
        }
    )

invalid_numerical_values_df = pd.DataFrame(
    invalid_value_records
)

invalid_numerical_values_df
```

Out [23]:

	dataset	column	invalid_values	rule
0	products	product_weight_g	4	Product weight must be greater than zero
1	products	product_length_cm	0	Product length must be greater than zero
2	products	product_height_cm	0	Product height must be greater than zero
3	products	product_width_cm	0	Product width must be greater than zero
4	order_items	price	0	Product price cannot be negative
5	order_items	freight_value	0	Freight cost cannot be negative
6	payments	payment_value	0	Payment value cannot be negative

In [24]:

```
# Product measurements that are zero or negative are physically invalid
physical_measurement_columns = [
    "product_weight_g",
    "product_length_cm",
    "product_height_cm",
    "product_width_cm"
]

for column_name in physical_measurement_columns:
    invalid_measurement_mask = (
        products_clean_df[column_name] <= 0
    )

    products_clean_df.loc[
        invalid_measurement_mask,
        column_name
    ] = np.nan

print("Invalid physical product measurements converted to missing values.")

display(
    products_clean_df[
        physical_measurement_columns
    ].isna().sum().to_frame(
        name="missing_values_after_sanitisation"
    )
)
```

Invalid physical product measurements converted to missing values.

missing_values_after_sanitisation	
product_weight_g	6
product_length_cm	2
product_height_cm	2
product_width_cm	2

4.4 Geolocation Cleaning and Aggregation

The geolocation table contains repeated coordinates for the same ZIP-code prefix and several coordinates outside plausible Brazilian geographical boundaries. Directly merging this table would create extensive row multiplication.

Coordinates outside the defined Brazilian boundary are excluded from aggregation. The median latitude and longitude are then calculated for each ZIP-code prefix. The median is preferred to the mean because it is less sensitive to remaining coordinate outliers.

The most frequently occurring city and state are retained for each ZIP-code prefix, and an observation count is recorded as an indicator of coordinate support.

```
In [25]: # =====
# GEOLOCATION COORDINATE VALIDATION
# =====

# Slightly expanded geographical limits surrounding Brazil
BRAZIL_MIN_LATITUDE = -35.5
BRAZIL_MAX_LATITUDE = 6.5
BRAZIL_MIN_LONGITUDE = -75.5
BRAZIL_MAX_LONGITUDE = -32.0

valid_latitude_mask = geolocation_clean_df[
    "geolocation_lat"
].between(
    BRAZIL_MIN_LATITUDE,
    BRAZIL_MAX_LATITUDE
)

valid_longitude_mask = geolocation_clean_df[
    "geolocation_lng"
].between(
    BRAZIL_MIN_LONGITUDE,
    BRAZIL_MAX_LONGITUDE
)

valid_coordinate_mask = (
    valid_latitude_mask
    & valid_longitude_mask
)

invalid_geolocation_df = geolocation_clean_df.loc[
    ~valid_coordinate_mask
].copy()

geolocation_valid_df = geolocation_clean_df.loc[
    valid_coordinate_mask
].copy()

geolocation_coordinate_audit_df = pd.DataFrame(
    {
        "measurement": [
            "Rows after exact-duplicate removal",
            "Valid Brazilian coordinates",
            "Coordinates outside accepted bounds",
            "Unique ZIP prefixes before coordinate filtering",
        ]
    })
```

```
        "Unique ZIP prefixes after coordinate filtering"
    ],
    "value": [
        len(geolocation_clean_df),
        len(geolocation_valid_df),
        len(invalid_geolocation_df),
        geolocation_clean_df[
            "geolocation_zip_code_prefix"
        ].nunique(),
        geolocation_valid_df[
            "geolocation_zip_code_prefix"
        ].nunique()
    ]
}

)

geolocation_coordinate_audit_df
```

Out[25]:

	measurement	value
0	Rows after exact-duplicate removal	738332
1	Valid Brazilian coordinates	738307
2	Coordinates outside accepted bounds	25
3	Unique ZIP prefixes before coordinate filtering	19015
4	Unique ZIP prefixes after coordinate filtering	19011

```
In [26]: # =====
# AGGREGATE GEOLOCATION BY ZIP-CODE PREFIX
# =====

def first_mode(series):
    """
    Return the most frequent non-missing value in a Series.
    If multiple modes exist, return the first sorted mode.
    """

    modes = series.dropna().mode()

    if modes.empty:
        return pd.NA

    return modes.iloc[0]

geolocation_by_zip_df = (
    geolocation_valid_df
    .groupby(
        "geolocation_zip_code_prefix",
        as_index=False
    )
    .agg(
        zip_latitude=(
```

```
        "geolocation_lat",
        "median"
    ),
    zip_longitude=(
        "geolocation_lng",
        "median"
    ),
    zip_city=(
        "geolocation_city",
        first_mode
    ),
    zip_state=(
        "geolocation_state",
        first_mode
    ),
    zip_observation_count=(
        "geolocation_lat",
        "size"
    )
)

# Confirm that each ZIP-code prefix is now unique
assert geolocation_by_zip_df[
    "geolocation_zip_code_prefix"
].is_unique

print("Geolocation aggregation complete.")
print(
    f"Raw valid geolocation rows: "
    f"{len(geolocation_valid_df):,}"
)
print(
    f"Aggregated ZIP-code rows: "
    f"{len(geolocation_by_zip_df):,}"
)

display(geolocation_by_zip_df.head())
```

Geolocation aggregation complete.
Raw valid geolocation rows: 738,307
Aggregated ZIP-code rows: 19,011

	geolocation_zip_code_prefix	zip_latitude	zip_longitude	zip_city	zip_state	zip_observation_count
0	1001	-23.550	-46.634	sao paulo	SP	11
1	1002	-23.548	-46.635	sao paulo	SP	6
2	1003	-23.549	-46.635	sao paulo	SP	11
3	1004	-23.550	-46.635	sao paulo	SP	14
4	1005	-23.550	-46.636	sao paulo	SP	13

4.5 Product-Category Translation

Product categories are stored in Portuguese, while a separate translation table provides English names. A controlled many-to-one left join is used so that no product records are lost.

Where an English translation is unavailable, the original Portuguese category is retained. Products with no category are assigned the label `unknown`. This preserves all product records and prevents missing categorical values from being silently discarded.

```
In [27]: # =====
# PRODUCT CATEGORY TRANSLATION
# =====

products_enriched_df = products_clean_df.merge(
    category_translation_clean_df,
    how="left",
    on="product_category_name",
    validate="many_to_one"
)

missing_translation_before_fallback = int(
    products_enriched_df[
        "product_category_name_english"
    ].isna().sum()
)

# Use the original Portuguese category when translation is unavailable
products_enriched_df[
    "product_category_name_english"
] = (
    products_enriched_df[
        "product_category_name_english"
    ]
    .fillna(
        products_enriched_df[
            "product_category_name"
        ]
    )
    .fillna("unknown")
)

missing_translation_after_fallback = int(
    products_enriched_df[
        "product_category_name_english"
    ].isna().sum()
)

category_translation_audit_df = pd.DataFrame(
    {
        "measurement": [
            "Total products",
            "Missing English category before fallback",
            "Missing English category after fallback",
            "Final unique English category labels"
        ],
        "value": [
            len(products_enriched_df),
            missing_translation_before_fallback,
            missing_translation_after_fallback,
```

```
        products_enriched_df[
            "product_category_name_english"
        ].nunique()
    ]
}
)

category_translation_audit_df
```

Out[27]:

	measurement	value
0	Total products	32951
1	Missing English category before fallback	623
2	Missing English category after fallback	0
3	Final unique English category labels	74

In [28]:

```
# =====
# SAVE DATA-CLEANING AUDIT OUTPUTS
# =====

datetime_conversion_audit_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "datetime_conversion_audit.csv",
    index=False
)

chronological_audit_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "chronological_validity_audit.csv",
    index=False
)

duplicate_removal_audit_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "duplicate_removal_audit.csv",
    index=False
)

numeric_conversion_audit_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "numeric_conversion_audit.csv",
    index=False
)

invalid_numerical_values_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "invalid_numerical_values.csv",
    index=False
)

geolocation_coordinate_audit_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "geolocation_coordinate_audit.csv",
```

```
        index=False
    )

    invalid_geolocation_df.to_csv(
        PROCESSED_DATA_DIRECTORY
        / "invalid_geolocation_coordinates.csv",
        index=False
    )

    geolocation_by_zip_df.to_csv(
        PROCESSED_DATA_DIRECTORY
        / "geolocation_by_zip.csv",
        index=False
    )

    category_translation_audit_df.to_csv(
        PROCESSED_DATA_DIRECTORY
        / "category_translation_audit.csv",
        index=False
    )

    print("All cleaning audit files saved successfully.")
```

All cleaning audit files saved successfully.

Data-Cleaning Interpretation

The cleaning process preserved the original raw datasets while creating separate working copies for preprocessing. Textual values were stripped of unnecessary whitespace, timestamp variables were converted into datetime objects and numerical variables were explicitly validated.

Exact duplicate removal had its greatest effect on the geolocation table. However, repeated business identifiers in the order-items, payments and reviews tables were preserved because they may represent legitimate one-to-many relationships rather than exact duplication.

Physical product measurements equal to or below zero were treated as invalid and converted to missing values. These measurements have not yet been imputed because imputation before the train-test split could transfer information from the testing data into the training process.

Geographical coordinates were checked against expanded Brazilian boundaries. Implausible coordinates were excluded, and the remaining observations were aggregated using median coordinates to produce one unique record per ZIP-code prefix. This prevents row multiplication during later customer and seller joins.

Product category translations were attached through a validated many-to-one merge. Missing translations were replaced with the corresponding Portuguese category, while products without any category were assigned the label `unknown`. Consequently, product records were preserved rather than removed solely because of missing category information.

5. Construction of the Analytical Dataset

The regression model will use one order item as its unit of analysis. Therefore, each row in the final dataset will represent one product item sold within one customer order.

The order-items table forms the base table because it contains the target variable, `freight_value`. Orders, customers, sellers, products and geographical information are then attached through controlled many-to-one joins.

Payments and reviews are excluded from the primary regression table. Both contain repeated order identifiers, and directly merging them could multiply order-item rows. Payment values may also include freight charges, creating target leakage if used to predict `freight_value`.

All joins use Pandas relationship validation, and row counts are audited before and after every merge.

```
In [29]: # =====
# CREATE GEOGRAPHICAL FALLBACK TABLES
# =====

geolocation_city_state_df = (
    geolocation_valid_df
    .groupby(
        [
            "geolocation_city",
            "geolocation_state"
        ],
        as_index=False
    )
    .agg(
        city_state_latitude=(
            "geolocation_lat",
            "median"
        ),
        city_state_longitude=(
            "geolocation_lng",
            "median"
        ),
        city_state_observation_count=(
            "geolocation_lat",
            "size"
        )
    )
)

geolocation_state_df = (
    geolocation_valid_df
    .groupby(
        "geolocation_state",
        as_index=False
    )
    .agg(
        state_latitude=(
            "geolocation_lat",
            "median"
        ),
        state_longitude=(
            "geolocation_lng",
            "median"
        ),
        state_observation_count=(
            "geolocation_lat",
            "size"
        )
    )
)

# Validate uniqueness before later many-to-one joins
assert not geolocation_city_state_df.duplicated()
```

```

subset=[
    "geolocation_city",
    "geolocation_state"
]
).any()

assert geolocation_state_df[
    "geolocation_state"
].is_unique

print(
    "City-state fallback records:",
    f"{len(geolocation_city_state_df):,}"
)

print(
    "State fallback records:",
    f"{len(geolocation_state_df):,}"
)

```

City-state fallback records: 8,457
 State fallback records: 27

```

In [30]: # =====
# CUSTOMER GEOGRAPHICAL ENRICHMENT
# =====

customer_zip_lookup_df = (
    geolocation_by_zip_df[
        [
            "geolocation_zip_code_prefix",
            "zip_latitude",
            "zip_longitude",
            "zip_observation_count"
        ]
    ]
    .rename(
        columns={
            "geolocation_zip_code_prefix":
                "customer_zip_code_prefix",
            "zip_latitude":
                "customer_zip_latitude",
            "zip_longitude":
                "customer_zip_longitude",
            "zip_observation_count":
                "customer_zip_observation_count"
        }
    )
)

customer_city_lookup_df = (
    geolocation_city_state_df
    .rename(
        columns={
            "geolocation_city":
                "customer_city",

```

```
        "geolocation_state":
            "customer_state",
        "city_state_latitude":
            "customer_city_latitude",
        "city_state_longitude":
            "customer_city_longitude",
        "city_state_observation_count":
            "customer_city_observation_count"
    }
)
)

customer_state_lookup_df = (
    geolocation_state_df
    .rename(
        columns={
            "geolocation_state":
                "customer_state",
            "state_latitude":
                "customer_state_latitude",
            "state_longitude":
                "customer_state_longitude",
            "state_observation_count":
                "customer_state_observation_count"
        }
    )
)

customers_geo_df = customers_clean_df.merge(
    customer_zip_lookup_df,
    how="left",
    on="customer_zip_code_prefix",
    validate="many_to_one"
)

customers_geo_df = customers_geo_df.merge(
    customer_city_lookup_df,
    how="left",
    on=[
        "customer_city",
        "customer_state"
    ],
    validate="many_to_one"
)

customers_geo_df = customers_geo_df.merge(
    customer_state_lookup_df,
    how="left",
    on="customer_state",
    validate="many_to_one"
)

customers_geo_df["customer_latitude"] = (
    customers_geo_df["customer_zip_latitude"]
    .combine_first(
        customers_geo_df["customer_city_latitude"]
    )
)
```

```
)
    .combine_first(
        customers_geo_df["customer_state_latitude"]
    )
)

customers_geo_df["customer_longitude"] = (
    customers_geo_df["customer_zip_longitude"]
    .combine_first(
        customers_geo_df["customer_city_longitude"]
    )
    .combine_first(
        customers_geo_df["customer_state_longitude"]
    )
)

customer_zip_available = (
    customers_geo_df["customer_zip_latitude"].notna()
    & customers_geo_df["customer_zip_longitude"].notna()
)

customer_city_available = (
    customers_geo_df["customer_city_latitude"].notna()
    & customers_geo_df["customer_city_longitude"].notna()
)

customer_state_available = (
    customers_geo_df["customer_state_latitude"].notna()
    & customers_geo_df["customer_state_longitude"].notna()
)

customers_geo_df["customer_coordinate_source"] = np.select(
    condlist=[
        customer_zip_available,
        ~customer_zip_available & customer_city_available,
        (
            ~customer_zip_available
            & ~customer_city_available
            & customer_state_available
        )
    ],
    choicelist=[
        "zip_prefix",
        "city_state_fallback",
        "state_fallback"
    ],
    default="unmatched"
)

assert customers_geo_df["customer_id"].is_unique

display(
    customers_geo_df[
        "customer_coordinate_source"
    ].value_counts(dropna=False).to_frame(
        name="customer_count"
    )
)
```

```
)  
)
```

customer_count	
customer_coordinate_source	
zip_prefix	99162
city_state_fallback	221
state_fallback	58

```
In [31]: # =====  
# SELLER GEOGRAPHICAL ENRICHMENT  
# =====  
  
seller_zip_lookup_df = (  
    geolocation_by_zip_df[  
        [  
            "geolocation_zip_code_prefix",  
            "zip_latitude",  
            "zip_longitude",  
            "zip_observation_count"  
        ]  
    ]  
    .rename(  
        columns={  
            "geolocation_zip_code_prefix":  
                "seller_zip_code_prefix",  
            "zip_latitude":  
                "seller_zip_latitude",  
            "zip_longitude":  
                "seller_zip_longitude",  
            "zip_observation_count":  
                "seller_zip_observation_count"  
        }  
    )  
)  
  
seller_city_lookup_df = (  
    geolocation_city_state_df  
    .rename(  
        columns={  
            "geolocation_city":  
                "seller_city",  
            "geolocation_state":  
                "seller_state",  
            "city_state_latitude":  
                "seller_city_latitude",  
            "city_state_longitude":  
                "seller_city_longitude",  
            "city_state_observation_count":  
                "seller_city_observation_count"  
        }  
    )  
)
```

```
)

seller_state_lookup_df = (
    geolocation_state_df
    .rename(
        columns={
            "geolocation_state":
                "seller_state",
            "state_latitude":
                "seller_state_latitude",
            "state_longitude":
                "seller_state_longitude",
            "state_observation_count":
                "seller_state_observation_count"
        }
    )
)

sellers_geo_df = sellers_clean_df.merge(
    seller_zip_lookup_df,
    how="left",
    on="seller_zip_code_prefix",
    validate="many_to_one"
)

sellers_geo_df = sellers_geo_df.merge(
    seller_city_lookup_df,
    how="left",
    on=[
        "seller_city",
        "seller_state"
    ],
    validate="many_to_one"
)

sellers_geo_df = sellers_geo_df.merge(
    seller_state_lookup_df,
    how="left",
    on="seller_state",
    validate="many_to_one"
)

sellers_geo_df["seller_latitude"] = (
    sellers_geo_df["seller_zip_latitude"]
    .combine_first(
        sellers_geo_df["seller_city_latitude"]
    )
    .combine_first(
        sellers_geo_df["seller_state_latitude"]
    )
)

sellers_geo_df["seller_longitude"] = (
    sellers_geo_df["seller_zip_longitude"]
    .combine_first(
        sellers_geo_df["seller_city_longitude"]
    )
)
```

```

    )
    .combine_first(
        sellers_geo_df["seller_state_longitude"]
    )
)

seller_zip_available = (
    sellers_geo_df["seller_zip_latitude"].notna()
    & sellers_geo_df["seller_zip_longitude"].notna()
)

seller_city_available = (
    sellers_geo_df["seller_city_latitude"].notna()
    & sellers_geo_df["seller_city_longitude"].notna()
)

seller_state_available = (
    sellers_geo_df["seller_state_latitude"].notna()
    & sellers_geo_df["seller_state_longitude"].notna()
)

sellers_geo_df["seller_coordinate_source"] = np.select(
    condlist=[
        seller_zip_available,
        ~seller_zip_available & seller_city_available,
        (
            ~seller_zip_available
            & ~seller_city_available
            & seller_state_available
        )
    ],
    choicelist=[
        "zip_prefix",
        "city_state_fallback",
        "state_fallback"
    ],
    default="unmatched"
)

assert sellers_geo_df["seller_id"].is_unique

display(
    sellers_geo_df[
        "seller_coordinate_source"
    ].value_counts(dropna=False).to_frame(
        name="seller_count"
    )
)
```

seller_count	
seller_coordinate_source	
zip_prefix	3088
city_state_fallback	7

```

In [32]: # =====
# FINAL GEOGRAPHICAL COVERAGE AUDIT
# =====

def create_coordinate_coverage_record(
    dataframe,
    entity_name,
    latitude_column,
    longitude_column,
    source_column
):
    """
    Summarise final geographical coordinate coverage.
    """

    complete_coordinates = (
        dataframe[latitude_column].notna()
        & dataframe[longitude_column].notna()
    )

    return {
        "entity": entity_name,
        "total_records": len(dataframe),
        "complete_coordinate_records": int(
            complete_coordinates.sum()
        ),
        "missing_coordinate_records": int(
            (~complete_coordinates).sum()
        ),
        "coordinate_coverage_percentage": (
            complete_coordinates.mean() * 100
        ),
        "zip_prefix_coordinates": int(
            (
                dataframe[source_column]
                == "zip_prefix"
            ).sum()
        ),
        "city_state_fallbacks": int(
            (
                dataframe[source_column]
                == "city_state_fallback"
            ).sum()
        ),
        "state_fallbacks": int(
            (
                dataframe[source_column]
                == "state_fallback"
            ).sum()
        ),
        "unmatched_records": int(
            (
                dataframe[source_column]
                == "unmatched"
            ).sum()
        )
    }

```

```
    )
}

coordinate_coverage_audit_df = pd.DataFrame(
    [
        create_coordinate_coverage_record(
            dataframe=customers_geo_df,
            entity_name="customers",
            latitude_column="customer_latitude",
            longitude_column="customer_longitude",
            source_column="customer_coordinate_source"
        ),
        create_coordinate_coverage_record(
            dataframe=sellers_geo_df,
            entity_name="sellers",
            latitude_column="seller_latitude",
            longitude_column="seller_longitude",
            source_column="seller_coordinate_source"
        )
    ]
)

coordinate_coverage_audit_df
```

Out [32]:

	entity	total_records	complete_coordinate_records	missing_coordinate_records	coordinate_coverage_percentage	zip_prefix_coordinates	city_state_fallbacks	state_fallbacks	un
0	customers	99441	99441	0	100.000	99162	221	58	
1	sellers	3095	3095	0	100.000	3088	7	0	

5.1 Order-Level Context

Although each analytical observation represents one order item, the wider order may contain multiple products or sellers. Order-level contextual variables are therefore calculated before merging.

The total freight value of an order is deliberately excluded because it contains the target variable and would create direct target leakage.

```
In [33]: # =====
# ORDER-LEVEL CONTEXTUAL FEATURES
# =====

order_context_df = (
    order_items_clean_df
    .groupby(
        "order_id",
        as_index=False
    )
    .agg(
        order_item_count=(
            "order_item_id",
            "count"
        ),
        order_distinct_product_count=(
```

```
        "product_id",
        "nunique"
    ),
    order_distinct_seller_count=(
        "seller_id",
        "nunique"
    ),
    order_total_product_value=(
        "price",
        "sum"
    ),
    order_average_product_price=(
        "price",
        "mean"
    ),
    order_max_product_price=(
        "price",
        "max"
    )
)

)

)

assert order_context_df["order_id"].is_unique

display(order_context_df.head())

print(
    "Order context records:",
    f"{len(order_context_df):,}"
)
```

	order_id	order_item_count	order_distinct_product_count	order_distinct_seller_count	order_total_product_value	order_average_product_price	order_max_p
0	00010242fe8c5a6d1ba2dd792cb16214	1	1	1	58.900	58.900	
1	00018f77f2f0320c557190d7a144bdd3	1	1	1	239.900	239.900	
2	000229ec398224ef6ca0657da4fc703e	1	1	1	199.000	199.000	
3	00024acbcd0a6daa1e931b038114c75	1	1	1	12.990	12.990	
4	00042b26cf59d7ce69dfabb4e55b4fd9	1	1	1	199.900	199.900	

Order context records: 98,666

```
In [34]: # =====
# PREPARE CONTROLLED MERGE TABLES
# =====

orders_for_merge_df = orders_clean_df[
    [
        "order_id",
        "customer_id",
        "order_status",
        "order_purchase_timestamp",
        "order_approved_at",
        "order_delivered_carrier_date",
```

```
        "order_delivered_customer_date",
        "order_estimated_delivery_date"
    ]
].copy()

customers_for_merge_df = customers_geo_df[
    [
        "customer_id",
        "customer_unique_id",
        "customer_zip_code_prefix",
        "customer_city",
        "customer_state",
        "customer_latitude",
        "customer_longitude",
        "customer_coordinate_source"
    ]
].copy()

sellers_for_merge_df = sellers_geo_df[
    [
        "seller_id",
        "seller_zip_code_prefix",
        "seller_city",
        "seller_state",
        "seller_latitude",
        "seller_longitude",
        "seller_coordinate_source"
    ]
].copy()

products_for_merge_df = (
    products_enriched_df[
        [
            "product_id",
            "product_category_name",
            "product_category_name_english",
            "product_name_lenght",
            "product_description_lenght",
            "product_photos_qty",
            "product_weight_g",
            "product_length_cm",
            "product_height_cm",
            "product_width_cm"
        ]
    ]
    .rename(
        columns={
            "product_name_lenght":
                "product_name_length",
            "product_description_lenght":
                "product_description_length"
        }
    )
)

# Confirm that parent-table join keys are unique
```

```

assert orders_for_merge_df["order_id"].is_unique
assert customers_for_merge_df["customer_id"].is_unique
assert sellers_for_merge_df["seller_id"].is_unique
assert products_for_merge_df["product_id"].is_unique

print("All controlled merge tables passed uniqueness checks.")

```

All controlled merge tables passed uniqueness checks.

```

In [35]: # =====
# AUDITED MANY-TO-ONE MERGE FUNCTION
# =====

def audited_many_to_one_merge(
    left_dataframe,
    right_dataframe,
    join_columns,
    merge_name
):
    """
    Perform a validated left many-to-one merge and record
    whether rows were matched without multiplying observations.
    """

    rows_before = len(left_dataframe)
    indicator_column = f"_merge_{merge_name}"

    merged_dataframe = left_dataframe.merge(
        right_dataframe,
        how="left",
        on=join_columns,
        validate="many_to_one",
        indicator=indicator_column
    )

    rows_after = len(merged_dataframe)

    matched_rows = int(
        (
            merged_dataframe[indicator_column]
            == "both"
        ).sum()
    )

    unmatched_rows = int(
        (
            merged_dataframe[indicator_column]
            == "left_only"
        ).sum()
    )

    audit_record = {
        "merge_name": merge_name,
        "join_columns": (
            ", ".join(join_columns)
            if isinstance(join_columns, list)

```

```

        else join_columns
    ),
    "rows_before": rows_before,
    "rows_after": rows_after,
    "row_difference": rows_after - rows_before,
    "matched_rows": matched_rows,
    "unmatched_rows": unmatched_rows,
    "match_percentage": (
        matched_rows / rows_after * 100
        if rows_after > 0
        else np.nan
    )
}

if rows_after != rows_before:
    raise ValueError(
        f"{merge_name} changed the number of rows "
        f"from {rows_before:,} to {rows_after:,}."
    )

merged_dataframe = merged_dataframe.drop(
    columns=indicator_column
)

return merged_dataframe, audit_record

```

```

In [36]: # =====
# BUILD THE ITEM-LEVEL ANALYTICAL DATASET
# =====

analytical_df = order_items_clean_df.copy()

merge_audit_records = []

analytical_df, merge_record = audited_many_to_one_merge(
    left_dataframe=analytical_df,
    right_dataframe=orders_for_merge_df,
    join_columns="order_id",
    merge_name="order_items_to_orders"
)
merge_audit_records.append(merge_record)

analytical_df, merge_record = audited_many_to_one_merge(
    left_dataframe=analytical_df,
    right_dataframe=customers_for_merge_df,
    join_columns="customer_id",
    merge_name="orders_to_customers"
)
merge_audit_records.append(merge_record)

analytical_df, merge_record = audited_many_to_one_merge(
    left_dataframe=analytical_df,
    right_dataframe=sellers_for_merge_df,
    join_columns="seller_id",
    merge_name="order_items_to_sellers"
)

```

```
merge_audit_records.append(merge_record)

analytical_df, merge_record = audited_many_to_one_merge(
    left_dataframe=analytical_df,
    right_dataframe=products_for_merge_df,
    join_columns="product_id",
    merge_name="order_items_to_products"
)
merge_audit_records.append(merge_record)

analytical_df, merge_record = audited_many_to_one_merge(
    left_dataframe=analytical_df,
    right_dataframe=order_context_df,
    join_columns="order_id",
    merge_name="order_items_to_order_context"
)
merge_audit_records.append(merge_record)

merge_audit_df = pd.DataFrame(
    merge_audit_records
)

merge_audit_df
```

Out[36]:

	merge_name	join_columns	rows_before	rows_after	row_difference	matched_rows	unmatched_rows	match_percentage
0	order_items_to_orders	order_id	112650	112650	0	112650	0	100.000
1	orders_to_customers	customer_id	112650	112650	0	112650	0	100.000
2	order_items_to_sellers	seller_id	112650	112650	0	112650	0	100.000
3	order_items_to_products	product_id	112650	112650	0	112650	0	100.000
4	order_items_to_order_context	order_id	112650	112650	0	112650	0	100.000

5.2 Geographical Distance

Freight cost is expected to be influenced by the distance between the seller and customer. Straight-line geographical distance is calculated using the Haversine formula, which estimates the great-circle distance between two latitude-longitude coordinates on the Earth.

The calculated distance is an approximation rather than an exact road distance. Nevertheless, it provides a consistent geographical indicator across all Brazilian orders.

In [37]:

```
# =====
# HAVERSINE DISTANCE CALCULATION
# =====

def calculate_haversine_distance(
    latitude_1,
    longitude_1,
    latitude_2,
    longitude_2
):
    """
    Calculate great-circle distance in kilometres between
```

```
two coordinate pairs using the Haversine formula.
"""

earth_radius_km = 6_371.0088

latitude_1_radians = np.radians(latitude_1)
longitude_1_radians = np.radians(longitude_1)
latitude_2_radians = np.radians(latitude_2)
longitude_2_radians = np.radians(longitude_2)

latitude_difference = (
    latitude_2_radians
    - latitude_1_radians
)

longitude_difference = (
    longitude_2_radians
    - longitude_1_radians
)

haversine_component = (
    np.sin(latitude_difference / 2) ** 2
    + np.cos(latitude_1_radians)
    * np.cos(latitude_2_radians)
    * np.sin(longitude_difference / 2) ** 2
)

haversine_component = np.clip(
    haversine_component,
    0,
    1
)

angular_distance = 2 * np.arcsin(
    np.sqrt(haversine_component)
)

return earth_radius_km * angular_distance

analytical_df["distance_km"] = (
    calculate_haversine_distance(
        latitude_1=analytical_df[
            "seller_latitude"
        ],
        longitude_1=analytical_df[
            "seller_longitude"
        ],
        latitude_2=analytical_df[
            "customer_latitude"
        ],
        longitude_2=analytical_df[
            "customer_longitude"
        ]
    )
)
```

```
print("Distance feature created successfully.")

display(
    analytical_df["distance_km"]
    .describe()
    .to_frame(
        name="distance_km"
    )
)
```

Distance feature created successfully.

	distance_km
count	112,650.000
mean	597.432
std	588.615
min	0.000
25%	184.934
50%	432.996
75%	794.450
max	3,579.267

```
In [38]: # =====
# TEMPORAL AND DESCRIPTIVE VARIABLES
# =====

analytical_df["purchase_year"] = (
    analytical_df[
        "order_purchase_timestamp"
    ].dt.year
)

analytical_df["purchase_month"] = (
    analytical_df[
        "order_purchase_timestamp"
    ].dt.month
)

analytical_df["purchase_quarter"] = (
    analytical_df[
        "order_purchase_timestamp"
    ].dt.quarter
)

analytical_df["purchase_day_of_week"] = (
    analytical_df[
        "order_purchase_timestamp"
    ].dt.dayofweek
)
```

```
analytical_df["purchase_hour"] = (
    analytical_df[
        "order_purchase_timestamp"
    ].dt.hour
)

analytical_df["purchase_is_weekend"] = (
    analytical_df[
        "purchase_day_of_week"
    ]
    .isin([5, 6])
    .astype(int)
)

analytical_df["shipping_limit_days"] = (
    (
        analytical_df["shipping_limit_date"]
        - analytical_df[
            "order_purchase_timestamp"
        ]
    )
    .dt.total_seconds()
    / 86_400
)

# Descriptive variables only – not suitable as model inputs
analytical_df["delivery_days"] = (
    (
        analytical_df[
            "order_delivered_customer_date"
        ]
        - analytical_df[
            "order_purchase_timestamp"
        ]
    )
    .dt.total_seconds()
    / 86_400
)

analytical_df["estimated_delivery_days"] = (
    (
        analytical_df[
            "order_estimated_delivery_date"
        ]
        - analytical_df[
            "order_purchase_timestamp"
        ]
    )
    .dt.total_seconds()
    / 86_400
)

analytical_df["delivery_delay_days"] = (
    (
        analytical_df[
```

```

        "order_delivered_customer_date"
    ]
    - analytical_df[
        "order_estimated_delivery_date"
    ]
)
.dt.total_seconds()
/ 86_400
)

valid_state_pair = (
    analytical_df["customer_state"].notna()
    & analytical_df["seller_state"].notna()
)

analytical_df["same_state"] = np.where(
    valid_state_pair,
    (
        analytical_df["customer_state"]
        == analytical_df["seller_state"]
    ).astype(int),
    np.nan
)

print("Temporal and descriptive variables created successfully.")

```

Temporal and descriptive variables created successfully.

```

In [39]: # =====
# ANALYTICAL DATASET VALIDATION
# =====

composite_key_duplicates = int(
    analytical_df.duplicated(
        subset=[
            "order_id",
            "order_item_id"
        ]
    ).sum()
)

analytical_dataset_audit_df = pd.DataFrame(
    {
        "measurement": [
            "Total analytical rows",
            "Total analytical columns",
            "Unique orders",
            "Unique products",
            "Unique sellers",
            "Duplicate order-item composite keys",
            "Missing freight target values",
            "Negative freight target values",
            "Missing product price values",
            "Negative product price values",
            "Missing distance values",
            "Missing customer coordinates",

```

```
        "Missing seller coordinates"
    ],
    "value": [
        len(analytical_df),
        analytical_df.shape[1],
        analytical_df["order_id"].nunique(),
        analytical_df["product_id"].nunique(),
        analytical_df["seller_id"].nunique(),
        composite_key_duplicates,
        analytical_df[
            "freight_value"
        ].isna().sum(),
        (
            analytical_df["freight_value"] < 0
        ).sum(),
        analytical_df["price"].isna().sum(),
        (
            analytical_df["price"] < 0
        ).sum(),
        analytical_df["distance_km"].isna().sum(),
        analytical_df[
            "customer_latitude"
        ].isna().sum(),
        analytical_df[
            "seller_latitude"
        ].isna().sum()
    ]
}
)

analytical_dataset_audit_df
```

Out[39]:

	measurement	value
0	Total analytical rows	112650
1	Total analytical columns	54
2	Unique orders	98666
3	Unique products	32951
4	Unique sellers	3095
5	Duplicate order-item composite keys	0
6	Missing freight target values	0
7	Negative freight target values	0
8	Missing product price values	0
9	Negative product price values	0
10	Missing distance values	0
11	Missing customer coordinates	0
12	Missing seller coordinates	0

5.3 Modelling Population

Rows without a valid target cannot be used for supervised regression. Therefore, the modelling population retains observations where freight value and product price are present and non-negative.

Missing predictor values are not removed at this stage. They will later be imputed within a Scikit-learn pipeline fitted exclusively on the training data, preventing data leakage.

```
In [40]: # =====
# CREATE THE VALID MODELLING POPULATION
# =====

valid_target_mask = (
    analytical_df["freight_value"].notna()
    & (analytical_df["freight_value"] >= 0)
)

valid_price_mask = (
    analytical_df["price"].notna()
    & (analytical_df["price"] >= 0)
)

model_population_df = (
    analytical_df.loc[
        valid_target_mask
        & valid_price_mask
    ]
    .copy()
    .reset_index(drop=True)
)

population_exclusion_audit_df = pd.DataFrame(
    {
        "measurement": [
            "Rows in analytical dataset",
            "Rows with invalid freight target",
            "Rows with invalid price",
            "Rows retained for modelling",
            "Percentage retained"
        ],
        "value": [
            len(analytical_df),
            int((~valid_target_mask).sum()),
            int((~valid_price_mask).sum()),
            len(model_population_df),
            (
                len(model_population_df)
                / len(analytical_df)
                * 100
            )
        ]
    }
)

population_exclusion_audit_df
```

Out [40]:

	measurement	value
0	Rows in analytical dataset	112,650.000
1	Rows with invalid freight target	0.000
2	Rows with invalid price	0.000
3	Rows retained for modelling	112,650.000
4	Percentage retained	100.000

5.4 Reproducible Sampling

The raw modelling population contains substantially more than 10,000 observations. The coursework recommends a smaller dataset to support efficient loading and processing.

A reproducible simple random sample of 9,000 order items is therefore created using a fixed random state. Sampling is performed after the relational joins and essential cleaning so that every sampled row contains the complete analytical structure.

A comparison between the full modelling population and the sample is then produced to evaluate whether the sample preserves the central characteristics of the source data.

In [41]:

```
# =====
# CREATE A REPRODUCIBLE MODELLING SAMPLE
# =====

if len(model_population_df) > SAMPLE_SIZE:
    modelling_df = (
        model_population_df
        .sample(
            n=SAMPLE_SIZE,
            random_state=RANDOM_STATE,
            replace=False
        )
        .reset_index(drop=True)
    )
else:
    modelling_df = (
        model_population_df
        .copy()
        .reset_index(drop=True)
    )

print(
    "Full modelling population:",
    f"{len(model_population_df):,}"
)

print(
    "Final modelling sample:",
    f"{len(modelling_df):,}"
)

print(
    "Sampling percentage:",
```

```
f"{len(modelling_df) / len(model_population_df) * 100:.2f}%"
)
```

Full modelling population: 112,650

Final modelling sample: 9,000

Sampling percentage: 7.99%

```
In [42]: # =====
# SAMPLE REPRESENTATIVENESS AUDIT
# =====

representativeness_columns = [
    "freight_value",
    "price",
    "product_weight_g",
    "product_length_cm",
    "product_height_cm",
    "product_width_cm",
    "distance_km",
    "order_item_count"
]

representativeness_records = []

for column_name in representativeness_columns:

    population_mean = (
        model_population_df[column_name].mean()
    )

    sample_mean = (
        modelling_df[column_name].mean()
    )

    population_median = (
        model_population_df[column_name].median()
    )

    sample_median = (
        modelling_df[column_name].median()
    )

    population_standard_deviation = (
        model_population_df[column_name].std()
    )

    sample_standard_deviation = (
        modelling_df[column_name].std()
    )

    relative_mean_difference = (
        (
            sample_mean - population_mean
        )
        / population_mean
        * 100
```

```
        if pd.notna(population_mean)
        and population_mean != 0
        else np.nan
    )

    representativeness_records.append(
        {
            "variable": column_name,
            "population_mean": population_mean,
            "sample_mean": sample_mean,
            "relative_mean_difference_percentage":
                relative_mean_difference,
            "population_median": population_median,
            "sample_median": sample_median,
            "population_standard_deviation":
                population_standard_deviation,
            "sample_standard_deviation":
                sample_standard_deviation
        }
    )

sample_representativeness_df = pd.DataFrame(
    representativeness_records
)

sample_representativeness_df
```

Out [42]:

	variable	population_mean	sample_mean	relative_mean_difference_percentage	population_median	sample_median	population_standard_deviation	sample_standard_deviation
0	freight_value	19.990	19.700	-1.452	16.260	16.250	15.806	14.7
1	price	120.654	120.249	-0.336	74.990	74.000	183.634	188.6
2	product_weight_g	2,093.821	2,043.763	-2.391	700.000	700.000	3,751.689	3,638.
3	product_length_cm	30.154	30.058	-0.316	25.000	25.000	16.153	16.
4	product_height_cm	16.594	16.514	-0.481	13.000	13.000	13.443	13.5
5	product_width_cm	22.997	22.895	-0.442	20.000	20.000	11.707	11.5
6	distance_km	597.432	603.365	0.993	432.996	431.693	588.615	602.7
7	order_item_count	1.396	1.382	-0.987	1.000	1.000	1.120	1.0

```
In [43]: # =====
# CATEGORICAL SAMPLE REPRESENTATIVENESS
# =====

population_state_distribution = (
    model_population_df[
        "customer_state"
    ]
    .value_counts(
        normalize=True,
        dropna=False
    )
)
```

```
)

sample_state_distribution = (
    modelling_df[
        "customer_state"
    ]
    .value_counts(
        normalize=True,
        dropna=False
    )
)

state_distribution_comparison_df = (
    pd.concat(
        [
            population_state_distribution.rename(
                "population_proportion"
            ),
            sample_state_distribution.rename(
                "sample_proportion"
            )
        ],
        axis=1
    )
    .fillna(0)
)

state_distribution_comparison_df[
    "absolute_difference_percentage_points"
] = (
    (
        state_distribution_comparison_df[
            "sample_proportion"
        ]
        - state_distribution_comparison_df[
            "population_proportion"
        ]
    )
    .abs()
    * 100
)

state_distribution_comparison_df = (
    state_distribution_comparison_df
    .sort_values(
        "population_proportion",
        ascending=False
    )
)

display(state_distribution_comparison_df.head(15))

print(
    "Largest absolute state-distribution difference:",
    f"{state_distribution_comparison_df['absolute_difference_percentage_points'].max():.3f}",
    "percentage points"
```

)

	population_proportion	sample_proportion	absolute_difference_percentage_points
customer_state			
SP	0.421	0.428	0.690
RJ	0.129	0.127	0.197
MG	0.117	0.114	0.299
RS	0.055	0.050	0.502
PR	0.051	0.052	0.149
SC	0.037	0.035	0.174
BA	0.034	0.036	0.250
DF	0.021	0.020	0.136
GO	0.021	0.021	0.015
ES	0.020	0.021	0.064
PE	0.016	0.019	0.275
CE	0.013	0.015	0.155
PA	0.010	0.010	0.041
MT	0.009	0.008	0.137
MA	0.007	0.007	0.031

Largest absolute state-distribution difference: 0.690 percentage points

```
In [44]: # =====
# SAVE ANALYTICAL DATASETS AND AUDITS
# =====

coordinate_coverage_audit_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "coordinate_coverage_audit.csv",
    index=False
)

merge_audit_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "merge_audit.csv",
    index=False
)

analytical_dataset_audit_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "analytical_dataset_audit.csv",
    index=False
)

population_exclusion_audit_df.to_csv(
```

```
    PROCESSED_DATA_DIRECTORY
    / "population_exclusion_audit.csv",
    index=False
)

sample_representativeness_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "sample_representativeness.csv",
    index=False
)

state_distribution_comparison_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "state_distribution_comparison.csv"
)

analytical_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "full_analytical_dataset.csv",
    index=False
)

modelling_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "modelling_dataset_9000.csv",
    index=False
)

print("Analytical datasets and audit reports saved successfully.")
```

Analytical datasets and audit reports saved successfully.

Analytical Dataset Interpretation

The analytical dataset was constructed at the order-item level because freight value is recorded separately for each product item. Controlled many-to-one merges attached order, customer, seller, product and order-context information without increasing the original number of order-item rows.

Customer and seller coordinates were first obtained from ZIP-code prefixes. Where ZIP-level coordinates were unavailable, city-state and state-level median coordinates were used as fallback values. This reduced avoidable geographical data loss while documenting the source of each coordinate.

Seller-to-customer distance was calculated using the Haversine formula. This variable represents straight-line geographical distance and is therefore an approximation of transport distance rather than an exact road route.

Payment and review variables were excluded from the primary modelling table. Payments could create target leakage because payment values may contain freight charges, while both payments and reviews contain repeated order identifiers that could produce many-to-many row multiplication.

The final analytical structure satisfies First Normal Form because each row represents one uniquely identified order item, each field contains an atomic value and repeated entities have been resolved through validated relational joins.

A fixed random sample of 9,000 valid order items was produced to follow the coursework recommendation while maintaining reproducibility. Numerical summaries and customer-state proportions were compared with the complete modelling population to assess the representativeness of the sample.

6. Advanced Preprocessing Diagnostics and Statistical Analysis

The cleaned 9,000-row sample is now examined in greater detail before machine-learning development. This stage evaluates data leakage, grouped observations, predictor completeness, distribution shape, extreme values, categorical cardinality and appropriate preprocessing decisions.

No scaling, imputation, outlier capping or category grouping is applied to the full dataset at this stage. These operations must later be learned exclusively from the training data through a Scikit-learn pipeline. This prevents information from the test set influencing the fitted model.

The statistical analysis includes measures of central tendency, measures of spread, skewness, excess kurtosis and robust dispersion. Conventional and robust outlier diagnostics are compared because e-commerce variables such as price, weight, distance and freight cost commonly contain strongly asymmetric distributions.

```
In [45]: # =====
# DATA-LEAKAGE AND FEATURE-AVAILABILITY AUDIT
# =====

feature_leakage_audit_df = pd.DataFrame(
    [
        {
            "variable_or_group": "freight_value",
            "role": "Target",
            "available_at_prediction_time": False,
            "use_in_primary_model": False,
            "reason": "This is the value being predicted."
        },
        {
            "variable_or_group": "order_id",
            "role": "Grouping identifier",
            "available_at_prediction_time": True,
            "use_in_primary_model": False,
            "reason": (
                "Used to prevent items from the same order appearing "
                "in both training and testing datasets."
            )
        },
        {
            "variable_or_group": (
                "customer_id, customer_unique_id, "
                "product_id and seller_id"
            ),
            "role": "Identifiers",
            "available_at_prediction_time": True,
            "use_in_primary_model": False,
            "reason": (
                "High-cardinality identifiers may cause memorisation "
                "rather than generalisable prediction."
            )
        },
        {
            "variable_or_group": "price",
            "role": "Predictor",
            "available_at_prediction_time": True,
            "use_in_primary_model": True,
            "reason": (
                "Product price is known when the order is placed and "
```

```

        "may be associated with product type and logistics."
    )
},
{
    "variable_or_group": "Product weight and dimensions",
    "role": "Predictors",
    "available_at_prediction_time": True,
    "use_in_primary_model": True,
    "reason": (
        "Physical measurements influence parcel handling "
        "and transport requirements."
    )
},
{
    "variable_or_group": "distance_km",
    "role": "Predictor",
    "available_at_prediction_time": True,
    "use_in_primary_model": True,
    "reason": (
        "Customer and seller locations are known before "
        "delivery and are relevant to freight cost."
    )
},
{
    "variable_or_group": "Product category",
    "role": "Predictor",
    "available_at_prediction_time": True,
    "use_in_primary_model": True,
    "reason": (
        "Product category may represent handling, packaging "
        "and logistics differences."
    )
},
{
    "variable_or_group": "Customer and seller state",
    "role": "Predictors",
    "available_at_prediction_time": True,
    "use_in_primary_model": True,
    "reason": (
        "Regional location is known before shipment and may "
        "capture logistical differences not explained by distance alone."
    )
},
{
    "variable_or_group": "Order item-count variables",
    "role": "Predictors",
    "available_at_prediction_time": True,
    "use_in_primary_model": True,
    "reason": (
        "The complete basket is known when freight is calculated."
    )
},
{
    "variable_or_group": "Purchase time variables",
    "role": "Predictors",
    "available_at_prediction_time": True,

```

```

    "use_in_primary_model": True,
    "reason": (
        "Purchase month, weekday and hour are available "
        "when the transaction occurs."
    )
},
{
    "variable_or_group": "payment_value",
    "role": "Excluded leakage variable",
    "available_at_prediction_time": False,
    "use_in_primary_model": False,
    "reason": (
        "Payment value may already contain the freight charge "
        "and could directly reveal the target."
    )
},
{
    "variable_or_group": (
        "delivery_days, delivery_delay_days and "
        "delivered timestamps"
    ),
    "role": "Post-outcome variables",
    "available_at_prediction_time": False,
    "use_in_primary_model": False,
    "reason": (
        "These values are only known after shipment and would "
        "introduce future information."
    )
},
{
    "variable_or_group": "review variables",
    "role": "Post-outcome variables",
    "available_at_prediction_time": False,
    "use_in_primary_model": False,
    "reason": (
        "Reviews are created after the order has been delivered."
    )
},
{
    "variable_or_group": "order_status",
    "role": "Potential post-outcome variable",
    "available_at_prediction_time": False,
    "use_in_primary_model": False,
    "reason": (
        "Final order status may reflect events occurring "
        "after freight cost was calculated."
    )
},
{
    "variable_or_group": "shipping_limit_days",
    "role": "Operational variable",
    "available_at_prediction_time": "Uncertain",
    "use_in_primary_model": False,
    "reason": (
        "Its exact availability at freight-quotation time is "
        "uncertain, so it is excluded from the primary model."
    )
}

```

```
    )
    }
]
)

feature_leakage_audit_df
```

Out[45]:

	variable_or_group	role	available_at_prediction_time	use_in_primary_model	reason
0	freight_value	Target	False	False	This is the value being predicted.
1	order_id	Grouping identifier	True	False	Used to prevent items from the same order appe...
2	customer_id, customer_unique_id, product_id an...	Identifiers	True	False	High-cardinality identifiers may cause memoris...
3	price	Predictor	True	True	Product price is known when the order is place...
4	Product weight and dimensions	Predictors	True	True	Physical measurements influence parcel handlin...
5	distance_km	Predictor	True	True	Customer and seller locations are known before...
6	Product category	Predictor	True	True	Product category may represent handling, packa...
7	Customer and seller state	Predictors	True	True	Regional location is known before shipment and...
8	Order item-count variables	Predictors	True	True	The complete basket is known when freight is c...
9	Purchase time variables	Predictors	True	True	Purchase month, weekday and hour are available...
10	payment_value	Excluded leakage variable	False	False	Payment value may already contain the freight ...
11	delivery_days, delivery_delay_days and deliver...	Post-outcome variables	False	False	These values are only known after shipment and...
12	review variables	Post-outcome variables	False	False	Reviews are created after the order has been d...
13	order_status	Potential post-outcome variable	False	False	Final order status may reflect events occurrin...
14	shipping_limit_days	Operational variable	Uncertain	False	Its exact availability at freight-quotation ti...

6.1 Grouped-Observation and Split-Leakage Audit

Order items belonging to the same order share customer, location, purchase-time and order-level attributes. Treating them as completely independent observations could result in information leakage if items from the same order appear in both the training and testing datasets.

The later train-test split and cross-validation procedure will therefore use `order_id` as a grouping variable. This provides a more demanding and realistic estimate of generalisation performance.

```
In [46]: # =====
# GROUPED-OBSERVATION AUDIT
# =====

sample_order_frequency = (
    modelling_df
    .groupby("order_id")
    .size()
)
```

```
orders_with_multiple_sampled_items = (
    sample_order_frequency > 1
)

rows_from_multi_item_orders = int(
    sample_order_frequency.loc[
        orders_with_multiple_sampled_items
    ].sum()
)

group_structure_audit_df = pd.DataFrame(
    {
        "measurement": [
            "Rows in modelling sample",
            "Unique orders in modelling sample",
            "Orders represented by multiple sampled rows",
            "Rows belonging to multi-row orders",
            "Maximum sampled rows from one order",
            "Mean sampled rows per represented order",
            "Duplicate order-item composite keys"
        ],
        "value": [
            len(modelling_df),
            modelling_df["order_id"].nunique(),
            int(orders_with_multiple_sampled_items.sum()),
            rows_from_multi_item_orders,
            int(sample_order_frequency.max()),
            sample_order_frequency.mean(),
            int(
                modelling_df.duplicated(
                    subset=[
                        "order_id",
                        "order_item_id"
                    ]
                ).sum()
            )
        ]
    }
)
```

group_structure_audit_df

Out[46]:

	measurement	value
0	Rows in modelling sample	9,000.000
1	Unique orders in modelling sample	8,874.000
2	Orders represented by multiple sampled rows	120.000
3	Rows belonging to multi-row orders	246.000
4	Maximum sampled rows from one order	4.000
5	Mean sampled rows per represented order	1.014
6	Duplicate order-item composite keys	0.000

```
In [47]: assert modelling_df.duplicated(
        subset=["order_id", "order_item_id"]
    ).sum() == 0

print(
    "The order-item key remains unique. "
    "Group-aware splitting will be required."
)
```

The order-item key remains unique. Group-aware splitting will be required.

```
In [48]: # =====
# CANDIDATE PREDICTOR DEFINITIONS
# =====

baseline_numerical_features = [
    "price",
    "product_weight_g",
    "product_length_cm",
    "product_height_cm",
    "product_width_cm",
    "product_name_length",
    "product_description_length",
    "product_photos_qty",
    "distance_km",
    "order_item_count",
    "order_distinct_product_count",
    "order_distinct_seller_count",
    "order_total_product_value",
    "order_average_product_price",
    "purchase_month",
    "purchase_day_of_week",
    "purchase_hour",
    "purchase_is_weekend",
    "same_state"
]

baseline_categorical_features = [
    "product_category_name_english",
    "customer_state",
    "seller_state",
    "customer_coordinate_source",
    "seller_coordinate_source"
]

target_column = "freight_value"
group_column = "order_id"

all_candidate_features = (
    baseline_numerical_features
    + baseline_categorical_features
)

missing_defined_columns = [
    column_name
    for column_name in (
```

```

        all_candidate_features
        + [target_column, group_column]
    )
    if column_name not in modelling_df.columns
]

if missing_defined_columns:
    raise KeyError(
        "The following required columns are absent: "
        f"{missing_defined_columns}"
    )

print("Candidate predictor definitions validated successfully.")
print(
    f"Numerical predictors: "
    f"{len(baseline_numerical_features)}"
)
print(
    f"Categorical predictors: "
    f"{len(baseline_categorical_features)}"
)

```

Candidate predictor definitions validated successfully.
 Numerical predictors: 19
 Categorical predictors: 5

6.2 Predictor Completeness

Predictor missingness is analysed separately from target completeness. Missing numerical product attributes will later be imputed using training-set medians. Missing categorical values will be assigned an explicit category within the modelling pipeline.

The missingness percentage is retained under the existing notebook naming convention.

```

In [49]: # =====
# ADVANCED PREDICTOR MISSINGNESS PROFILE
# =====

predictor_missingness_records = []

for column_name in all_candidate_features:

    column_series = modelling_df[column_name]

    missing_count = int(
        column_series.isna().sum()
    )

    predictor_missingness_records.append(
        {
            "column": column_name,
            "data_type": str(column_series.dtype),
            "missing_values": missing_count,
            "missing_percentage": (
                missing_count
                / len(modelling_df)
            )
        }
    )

```

```
        * 100
    ),
    "non_missing_values": int(
        column_series.notna().sum()
    ),
    "unique_non_missing_values": int(
        column_series.nunique(
            dropna=True
        )
    )
}
)

predictor_missingness_df = (
    pd.DataFrame(
        predictor_missingness_records
    )
    .sort_values(
        by=[
            "missing_percentage",
            "unique_non_missing_values"
        ],
        ascending=[
            False,
            False
        ]
    )
    .reset_index(drop=True)
)

predictor_missingness_df
```

Out[49]:

	column	data_type	missing_values	missing_percentage	non_missing_values	unique_non_missing_values
0	product_description_length	float64	125	1.389	8875	1809
1	product_name_length	float64	125	1.389	8875	56
2	product_photos_qty	float64	125	1.389	8875	14
3	product_weight_g	float64	1	0.011	8999	977
4	product_length_cm	float64	1	0.011	8999	92
5	product_height_cm	float64	1	0.011	8999	87
6	product_width_cm	float64	1	0.011	8999	79
7	distance_km	float64	0	0.000	9000	8811
8	order_total_product_value	float64	0	0.000	9000	2438
9	order_average_product_price	float64	0	0.000	9000	2126
10	price	float64	0	0.000	9000	1838
11	product_category_name_english	string	0	0.000	9000	70
12	customer_state	string	0	0.000	9000	27
13	purchase_hour	int32	0	0.000	9000	24
14	seller_state	string	0	0.000	9000	21
15	order_item_count	int64	0	0.000	9000	15
16	purchase_month	int32	0	0.000	9000	12
17	order_distinct_product_count	int64	0	0.000	9000	7
18	purchase_day_of_week	int32	0	0.000	9000	7
19	order_distinct_seller_count	int64	0	0.000	9000	5
20	customer_coordinate_source	str	0	0.000	9000	3
21	purchase_is_weekend	int64	0	0.000	9000	2
22	same_state	float64	0	0.000	9000	2
23	seller_coordinate_source	str	0	0.000	9000	2

In [50]:

```
print(  
    "Predictors containing missing values:",  
    int(  
        (  
            predictor_missingness_df[  
                "missing_values"  
            ] > 0  
        ).sum()  
    )  
)  
  
print(  
    "Predictors containing missing values:",  
    int(  
        (  
            predictor_missingness_df[  
                "missing_values"  
            ] > 0  
        ).sum()  
    )  
)
```

```
    "Total missing predictor cells:",
    f"{predictor_missingness_df['missing_values'].sum():,}"
)
```

Predictors containing missing values: 7
Total missing predictor cells: 379

6.3 Measures of Central Tendency, Spread and Distribution Shape

The mean and median are reported together because strongly skewed e-commerce variables may be poorly represented by the arithmetic mean alone. Standard deviation, range and interquartile range measure conventional spread, while the median absolute deviation provides a robust measure less affected by extreme observations.

Skewness describes distribution asymmetry. Positive skewness indicates a long right tail, while negative skewness indicates a long left tail. Excess kurtosis describes tail weight relative to a normal distribution, for which excess kurtosis equals zero.

Additional Work: Advanced Regression Analysis

```
In [51]: # =====
# ADVANCED STATISTICAL SUMMARY
# =====

statistical_analysis_columns = [
    "freight_value",
    "price",
    "product_weight_g",
    "product_length_cm",
    "product_height_cm",
    "product_width_cm",
    "product_name_length",
    "product_description_length",
    "product_photos_qty",
    "distance_km",
    "order_item_count",
    "order_distinct_product_count",
    "order_distinct_seller_count",
    "order_total_product_value",
    "order_average_product_price"
]

def classify_skewness(skewness_value):
    """
    Classify distribution asymmetry using absolute skewness.
    """

    if pd.isna(skewness_value):
        return "Undefined"

    if abs(skewness_value) < 0.50:
        return "Approximately symmetric"

    if skewness_value >= 1.00:
        return "Strongly right-skewed"
```

```
if skewness_value >= 0.50:
    return "Moderately right-skewed"

if skewness_value <= -1.00:
    return "Strongly left-skewed"

return "Moderately left-skewed"

def classify_kurtosis(excess_kurtosis_value):
    """
    Describe tail weight using Fisher excess kurtosis.
    """

    if pd.isna(excess_kurtosis_value):
        return "Undefined"

    if excess_kurtosis_value > 1:
        return "Heavy-tailed / leptokurtic"

    if excess_kurtosis_value < -1:
        return "Light-tailed / platykurtic"

    return "Near mesokurtic range"

def create_advanced_statistical_summary(
    dataframe,
    numerical_columns
):
    """
    Calculate conventional and robust descriptive statistics.
    """

    summary_records = []

    for column_name in numerical_columns:

        values = pd.to_numeric(
            dataframe[column_name],
            errors="coerce"
        ).dropna()

        first_quartile = values.quantile(0.25)
        third_quartile = values.quantile(0.75)
        interquartile_range = (
            third_quartile - first_quartile
        )

        mean_value = values.mean()
        standard_deviation = values.std()

        coefficient_of_variation = (
            standard_deviation / mean_value
            if mean_value != 0
        )
```

```
        else np.nan
    )

    skewness_value = values.skew()
    kurtosis_value = values.kurt()

    summary_records.append(
        {
            "variable": column_name,
            "count": len(values),
            "missing_values": int(
                dataframe[column_name]
                .isna()
                .sum()
            ),
            "mean": mean_value,
            "trimmed_mean_5_percent": (
                stats.trim_mean(
                    values,
                    proportiontocut=0.05
                )
            ),
            "median": values.median(),
            "mode": (
                values.mode().iloc[0]
                if not values.mode().empty
                else np.nan
            ),
            "standard_deviation":
                standard_deviation,
            "robust_mad": (
                stats.median_abs_deviation(
                    values,
                    scale="normal",
                    nan_policy="omit"
                )
            ),
            "minimum": values.min(),
            "first_quartile": first_quartile,
            "third_quartile": third_quartile,
            "maximum": values.max(),
            "range": (
                values.max()
                - values.min()
            ),
            "interquartile_range":
                interquartile_range,
            "coefficient_of_variation":
                coefficient_of_variation,
            "skewness": skewness_value,
            "excess_kurtosis": kurtosis_value,
            "skewness_interpretation":
                classify_skewness(
                    skewness_value
                ),
            "kurtosis_interpretation":
```

```
        classify_kurtosis(  
            kurtosis_value  
        )  
    }  
)  
  
return pd.DataFrame(summary_records)  
  
advanced_statistical_summary_df = (  
    create_advanced_statistical_summary(  
        dataframe=modelling_df,  
        numerical_columns=statistical_analysis_columns  
    )  
)  
  
advanced_statistical_summary_df
```

Out [51]:

	variable	count	missing_values	mean	trimmed_mean_5_percent	median	mode	standard_deviation	robust_mad	minimum	first_quartile	third_quartile
0	freight_value	9000	0	19.700	17.778	16.250	15.100	14.787	5.337	0.000	13.008	21.023
1	price	9000	0	120.249	93.612	74.000	59.900	188.680	62.418	1.200	39.900	135.000
2	product_weight_g	8999	1	2,043.763	1,440.838	700.000	200.000	3,638.391	741.301	50.000	300.000	1,754.000
3	product_length_cm	8999	1	30.058	28.241	25.000	16.000	16.017	11.861	7.000	18.000	38.000
4	product_height_cm	8999	1	16.514	14.946	13.000	10.000	13.362	8.896	2.000	8.000	20.000
5	product_width_cm	8999	1	22.895	21.765	20.000	20.000	11.560	8.896	6.000	15.000	30.000
6	product_name_length	8875	125	48.718	49.419	51.000	60.000	10.073	10.378	5.000	43.000	57.000
7	product_description_length	8875	125	784.419	709.604	601.000	348.000	640.863	431.437	31.000	348.000	982.500
8	product_photos_qty	8875	125	2.197	1.989	1.000	1.000	1.713	0.000	1.000	1.000	3.000
9	distance_km	9000	0	603.365	538.236	431.693	0.000	602.756	445.461	0.000	173.851	795.480
10	order_item_count	9000	0	1.382	1.204	1.000	1.000	1.076	0.000	1.000	1.000	1.000
11	order_distinct_product_count	9000	0	1.085	1.017	1.000	1.000	0.366	0.000	1.000	1.000	1.000
12	order_distinct_seller_count	9000	0	1.030	1.000	1.000	1.000	0.188	0.000	1.000	1.000	1.000
13	order_total_product_value	9000	0	150.913	116.143	89.990	59.900	269.907	74.264	3.000	49.000	160.000
14	order_average_product_price	9000	0	120.155	93.736	74.900	59.900	186.624	62.269	1.514	39.900	134.990

In [52]:

```
# =====  
# DISTRIBUTION AND NORMALITY DIAGNOSTICS  
# =====  
  
NORMALITY_TEST_MAXIMUM_SAMPLE = 5_000  
NORMALITY_SIGNIFICANCE_LEVEL = 0.05  
  
normality_records = []  
  
for column_name in statistical_analysis_columns:  
    valid_values = pd.to_numeric(  
        modelling_df[column_name],  
        errors="coerce"
```

```
    ).dropna()

    if len(valid_values) > NORMALITY_TEST_MAXIMUM_SAMPLE:
        tested_values = valid_values.sample(
            n=NORMALITY_TEST_MAXIMUM_SAMPLE,
            random_state=RANDOM_STATE
        )
    else:
        tested_values = valid_values

    if len(tested_values) >= 8:
        test_statistic, p_value = stats.normaltest(
            tested_values
        )
    else:
        test_statistic = np.nan
        p_value = np.nan

    normality_records.append(
        {
            "variable": column_name,
            "available_observations":
                len(valid_values),
            "observations_tested":
                len(tested_values),
            "dagostino_k2_statistic":
                test_statistic,
            "p_value": p_value,
            "normality_result": (
                "Evidence against normality"
                if pd.notna(p_value)
                and p_value
                < NORMALITY_SIGNIFICANCE_LEVEL
                else
                "Insufficient evidence against normality"
            )
        }
    )

normality_diagnostics_df = pd.DataFrame(
    normality_records
)

normality_diagnostics_df
```

Out [52]:

	variable	available_observations	observations_tested	dagostino_k2_statistic	p_value	normality_result
0	freight_value	9000	5000	5,538.173	0.000	Evidence against normality
1	price	9000	5000	7,439.748	0.000	Evidence against normality
2	product_weight_g	8999	5000	3,802.191	0.000	Evidence against normality
3	product_length_cm	8999	5000	1,733.672	0.000	Evidence against normality
4	product_height_cm	8999	5000	2,492.929	0.000	Evidence against normality
5	product_width_cm	8999	5000	1,688.049	0.000	Evidence against normality
6	product_name_length	8875	5000	529.318	0.000	Evidence against normality
7	product_description_length	8875	5000	1,973.124	0.000	Evidence against normality
8	product_photos_qty	8875	5000	2,016.562	0.000	Evidence against normality
9	distance_km	9000	5000	1,410.403	0.000	Evidence against normality
10	order_item_count	9000	5000	5,714.931	0.000	Evidence against normality
11	order_distinct_product_count	9000	5000	5,773.876	0.000	Evidence against normality
12	order_distinct_seller_count	9000	5000	6,112.479	0.000	Evidence against normality
13	order_total_product_value	9000	5000	11,347.799	0.000	Evidence against normality
14	order_average_product_price	9000	5000	7,466.000	0.000	Evidence against normality

The normality test is interpreted cautiously because large samples can produce statistically significant results for small and practically unimportant departures from normality. Consequently, distribution shape will be evaluated using the combined evidence from histograms, Q–Q plots, skewness, kurtosis, the mean–median relationship and residual diagnostics rather than relying on the p-value alone.

6.4 Outlier Diagnostics

Potential outliers are detected using two complementary techniques. The interquartile-range method identifies observations outside 1.5 times the IQR, while the modified z-score uses the median and median absolute deviation.

Outlier detection does not automatically imply removal. High freight costs, large parcels and long transport distances may be valid commercial observations. The baseline model will initially retain these values. Their influence will later be assessed using residual analysis, transformations and a sensitivity model.

In [53]:

```
# =====  
# IQR AND ROBUST MODIFIED Z-SCORE OUTLIER AUDIT  
# =====  
  
continuous_outlier_columns = [  
    "freight_value",  
    "price",  
    "product_weight_g",  
    "product_length_cm",  
    "product_height_cm",  
    "product_width_cm",  
    "product_name_length",
```

```
    "product_description_length",
    "product_photos_qty",
    "distance_km",
    "order_total_product_value",
    "order_average_product_price"
]

IQR_MULTIPLIER = 1.5
EXTREME_IQR_MULTIPLIER = 3.0
MODIFIED_Z_THRESHOLD = 3.5

outlier_records = []

for column_name in continuous_outlier_columns:

    values = pd.to_numeric(
        modelling_df[column_name],
        errors="coerce"
    )

    non_missing_values = values.dropna()

    first_quartile = (
        non_missing_values.quantile(0.25)
    )

    third_quartile = (
        non_missing_values.quantile(0.75)
    )

    interquartile_range = (
        third_quartile - first_quartile
    )

    lower_iqr_bound = (
        first_quartile
        - IQR_MULTIPLIER
        * interquartile_range
    )

    upper_iqr_bound = (
        third_quartile
        + IQR_MULTIPLIER
        * interquartile_range
    )

    lower_extreme_bound = (
        first_quartile
        - EXTREME_IQR_MULTIPLIER
        * interquartile_range
    )

    upper_extreme_bound = (
        third_quartile
        + EXTREME_IQR_MULTIPLIER
        * interquartile_range
    )
```

```
)

iqr_outlier_mask = (
    (values < lower_iqr_bound)
    | (values > upper_iqr_bound)
)

extreme_iqr_mask = (
    (values < lower_extreme_bound)
    | (values > upper_extreme_bound)
)

median_value = non_missing_values.median()

raw_mad = stats.median_abs_deviation(
    non_missing_values,
    scale=1,
    nan_policy="omit"
)

if raw_mad > 0:
    modified_z_scores = (
        0.6745
        * (
            non_missing_values
            - median_value
        )
        / raw_mad
    )

    modified_z_outlier_count = int(
        (
            modified_z_scores.abs()
            > MODIFIED_Z_THRESHOLD
        ).sum()
    )
else:
    modified_z_outlier_count = 0

outlier_records.append(
    {
        "variable": column_name,
        "non_missing_values":
            len(non_missing_values),
        "lower_iqr_bound":
            lower_iqr_bound,
        "upper_iqr_bound":
            upper_iqr_bound,
        "iqr_outlier_count": int(
            iqr_outlier_mask.fillna(False).sum()
        ),
        "iqr_outlier_percentage": (
            iqr_outlier_mask
            .fillna(False)
            .mean()
            * 100
    )
}
```

```
        ),
        "extreme_iqr_outlier_count": int(
            extreme_iqr_mask
            .fillna(False)
            .sum()
        ),
        "modified_z_outlier_count":
            modified_z_outlier_count,
        "maximum_observed_value":
            non_missing_values.max()
    }
)

outlier_diagnostics_df = pd.DataFrame(
    outlier_records
)

outlier_diagnostics_df
```

Out[53]:

	variable	non_missing_values	lower_iqr_bound	upper_iqr_bound	iqr_outlier_count	iqr_outlier_percentage	extreme_iqr_outlier_count	modified_z_outlier_count	n
0	freight_value	9000	0.985	33.045	949	10.544	416	810	
1	price	9000	-102.750	277.650	671	7.456	319	608	
2	product_weight_g	8999	-1,881.000	3,935.000	1251	13.900	875	1407	
3	product_length_cm	8999	-12.000	68.000	272	3.022	78	326	
4	product_height_cm	8999	-10.000	38.000	602	6.689	220	443	
5	product_width_cm	8999	-7.500	52.500	193	2.144	28	205	
6	product_name_length	8875	22.000	78.000	98	1.089	0	13	
7	product_description_length	8875	-603.750	1,934.250	546	6.067	177	448	
8	product_photos_qty	8875	-2.000	6.000	238	2.644	45	0	
9	distance_km	9000	-758.593	1,727.924	733	8.144	53	569	
10	order_total_product_value	9000	-117.500	326.500	780	8.667	425	673	
11	order_average_product_price	9000	-102.735	277.625	668	7.422	322	606	

```
In [54]: # =====
# FREIGHT TARGET QUALITY AND EXTREME-VALUE AUDIT
# =====

freight_series = modelling_df[
    "freight_value"
].dropna()

freight_target_audit_df = pd.DataFrame(
    {
        "measurement": [
            "Number of observations",
            "Missing freight values",
```

```

        "Negative freight values",
        "Zero freight values",
        "Zero freight percentage",
        "Mean freight value",
        "Median freight value",
        "95th percentile",
        "99th percentile",
        "Maximum freight value",
        "Skewness",
        "Excess kurtosis"
    ],
    "value": [
        len(modelling_df),
        modelling_df[
            "freight_value"
        ].isna().sum(),
        (
            modelling_df[
                "freight_value"
            ] < 0
        ).sum(),
        (
            modelling_df[
                "freight_value"
            ] == 0
        ).sum(),
        (
            modelling_df[
                "freight_value"
            ] == 0
        ).mean() * 100,
        freight_series.mean(),
        freight_series.median(),
        freight_series.quantile(0.95),
        freight_series.quantile(0.99),
        freight_series.max(),
        freight_series.skew(),
        freight_series.kurt()
    ]
}
)

```

freight_target_audit_df

Out [54]:

	measurement	value
0	Number of observations	9,000.000
1	Missing freight values	0.000
2	Negative freight values	0.000
3	Zero freight values	33.000
4	Zero freight percentage	0.367
5	Mean freight value	19.700
6	Median freight value	16.250
7	95th percentile	44.020
8	99th percentile	82.583
9	Maximum freight value	306.060
10	Skewness	4.958
11	Excess kurtosis	44.644

6.5 Categorical Cardinality

Categorical predictors with many rare levels can produce sparse one-hot encoded variables and unstable regression coefficients. Their frequency distribution is therefore examined before modelling.

Rare-level grouping will later be performed within the training pipeline rather than on the full dataset. This prevents category-frequency information from the test set influencing preprocessing decisions.

In [55]:

```
# =====  
# CATEGORICAL CARDINALITY AND RARE-LEVEL AUDIT  
# =====  
  
RARE_CATEGORY_THRESHOLD = 0.01  
  
categorical_cardinality_records = []  
  
for column_name in baseline_categorical_features:  
    category_proportions = (  
        modelling_df[column_name]  
        .fillna("missing")  
        .value_counts(  
            normalize=True,  
            dropna=False  
        )  
    )  
  
    category_counts = (  
        modelling_df[column_name]  
        .fillna("missing")
```

```
        .value_counts(
            dropna=False
        )
    )

    rare_categories = category_proportions[
        category_proportions
        < RARE_CATEGORY_THRESHOLD
    ].index

    rare_row_count = int(
        modelling_df[column_name]
        .fillna("missing")
        .isin(rare_categories)
        .sum()
    )

    categorical_cardinality_records.append(
        {
            "variable": column_name,
            "unique_categories": int(
                category_counts.size
            ),
            "most_common_category":
                category_counts.index[0],
            "most_common_category_count": int(
                category_counts.iloc[0]
            ),
            "most_common_percentage": (
                category_proportions.iloc[0]
                * 100
            ),
            "categories_below_1_percent": int(
                len(rare_categories)
            ),
            "rows_in_categories_below_1_percent":
                rare_row_count,
            "rare_row_percentage": (
                rare_row_count
                / len(modelling_df)
                * 100
            )
        }
    )

categorical_cardinality_df = pd.DataFrame(
    categorical_cardinality_records
)

categorical_cardinality_df
```

Out [55]:

	variable	unique_categories	most_common_category	most_common_category_count	most_common_percentage	categories_below_1_percent	rows_in_category
0	product_category_name_english	70	bed_bath_table	918	10.200	48	
1	customer_state	27	SP	3853	42.811	14	
2	seller_state	21	SP	6495	72.167	15	
3	customer_coordinate_source	3	zip_prefix	8980	99.778	2	
4	seller_coordinate_source	2	zip_prefix	8978	99.756	1	

In [56]:

```
product_category_frequency_df = (  
    modelling_df[  
        "product_category_name_english"  
    ]  
    .fillna("missing")  
    .value_counts()  
    .rename_axis("product_category")  
    .reset_index(  
        name="observation_count"  
    )  
)  
  
product_category_frequency_df[  
    "percentage"  
] = (  
    product_category_frequency_df[  
        "observation_count"  
    ]  
    / len(modelling_df)  
    * 100  
)  
  
product_category_frequency_df.head(20)
```

Out [56]:

	product_category	observation_count	percentage
0	bed_bath_table	918	10.200
1	health_beauty	777	8.633
2	sports_leisure	679	7.544
3	furniture_decor	658	7.311
4	computers_accessories	614	6.822
5	housewares	588	6.533
6	watches_gifts	474	5.267
7	telephony	369	4.100
8	auto	360	4.000
9	garden_tools	343	3.811
10	toys	309	3.433
11	cool_stuff	279	3.100
12	perfumery	274	3.044
13	baby	271	3.011
14	electronics	239	2.656
15	stationery	185	2.056
16	fashion_bags_accessories	151	1.678
17	office_furniture	138	1.533
18	pet_shop	133	1.478
19	unknown	125	1.389

In [57]:

```
# =====  
# PREPROCESSING DECISION REGISTER  
# =====  
  
preprocessing_decision_register_df = pd.DataFrame(  
    [  
        {  
            "issue": "Missing numerical predictors",  
            "decision": (  
                "Median imputation inside the "  
                "training pipeline"  
            ),  
            "justification": (  
                "The median is robust to skewness and extreme values. "  
                "Fitting inside the pipeline prevents data leakage."  
            )  
        },  
    ],  
    {
```

```
"issue": "Missing categorical predictors",
"decision": (
    "Explicit missing category or most-frequent "
    "imputation inside the pipeline"
),
"justification": (
    "Preserves observations and allows missingness "
    "to remain identifiable."
)
},
{
    "issue": "Different numerical scales",
    "decision": (
        "StandardScaler fitted on training data only"
    ),
    "justification": (
        "Weight, distance, price and dimensions use very "
        "different units. Scaling is also required for "
        "regularised regression comparisons."
    )
},
{
    "issue": "Rare categorical levels",
    "decision": (
        "Group infrequent categories during one-hot encoding"
    ),
    "justification": (
        "Reduces sparse unstable coefficients while preserving "
        "common categories."
    )
},
{
    "issue": "Unseen test-set categories",
    "decision": (
        "OneHotEncoder with unknown-category handling"
    ),
    "justification": (
        "Prevents prediction failures when a category was not "
        "observed during model fitting."
    )
},
{
    "issue": "Strong positive skewness",
    "decision": (
        "Retain raw values for the baseline model and "
        "test log transformations in feature engineering"
    ),
    "justification": (
        "Maintains a transparent baseline and allows the "
        "benefit of transformation to be measured."
    )
},
{
    "issue": "Potential outliers",
    "decision": (
        "Retain initially; assess through residuals and "
```

```
        "sensitivity analysis"
    ),
    "justification": (
        "Extreme commercial observations may be valid rather "
        "than data errors."
    )
},
{
    "issue": "Multiple items within one order",
    "decision": (
        "Group-aware train-test split and cross-validation"
    ),
    "justification": (
        "Prevents shared order information from appearing "
        "in both training and validation data."
    )
},
{
    "issue": "Identifiers",
    "decision": (
        "Exclude from predictors"
    ),
    "justification": (
        "Identifiers encourage memorisation and have no "
        "meaningful continuous regression interpretation."
    )
},
{
    "issue": "Post-outcome variables",
    "decision": (
        "Exclude delivery, review and final-status variables"
    ),
    "justification": (
        "These are not available at prediction time and "
        "would cause target leakage."
    )
}
]
)

preprocessing_decision_register_df
```

Out [57]:

	issue	decision	justification
0	Missing numerical predictors	Median imputation inside the training pipeline	The median is robust to skewness and extreme v...
1	Missing categorical predictors	Explicit missing category or most-frequent imp...	Preserves observations and allows missingness ...
2	Different numerical scales	StandardScaler fitted on training data only	Weight, distance, price and dimensions use ver...
3	Rare categorical levels	Group infrequent categories during one-hot enc...	Reduces sparse unstable coefficients while pre...
4	Unseen test-set categories	OneHotEncoder with unknown-category handling	Prevents prediction failures when a category w...
5	Strong positive skewness	Retain raw values for the baseline model and t...	Maintains a transparent baseline and allows th...
6	Potential outliers	Retain initially; assess through residuals and...	Extreme commercial observations may be valid r...
7	Multiple items within one order	Group-aware train-test split and cross-validation	Prevents shared order information from appeari...
8	Identifiers	Exclude from predictors	Identifiers encourage memorisation and have no...
9	Post-outcome variables	Exclude delivery, review and final-status vari...	These are not available at prediction time and...

In [58]:

```
# =====
# SAVE ADVANCED PREPROCESSING AND STATISTICAL AUDITS
# =====

feature_leakage_audit_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "feature_leakage_audit.csv",
    index=False
)

group_structure_audit_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "group_structure_audit.csv",
    index=False
)

predictor_missingness_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "predictor_missingness.csv",
    index=False
)

advanced_statistical_summary_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "advanced_statistical_summary.csv",
    index=False
)

normality_diagnostics_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "normality_diagnostics.csv",
    index=False
)

outlier_diagnostics_df.to_csv(
    PROCESSED_DATA_DIRECTORY
```

```
    / "outlier_diagnostics.csv",
    index=False
)

freight_target_audit_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "freight_target_audit.csv",
    index=False
)

categorical_cardinality_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "categorical_cardinality.csv",
    index=False
)

product_category_frequency_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "product_category_frequency.csv",
    index=False
)

preprocessing_decision_register_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "preprocessing_decision_register.csv",
    index=False
)

print(
    "Advanced preprocessing and statistical "
    "audit files saved successfully."
)
```

Advanced preprocessing and statistical audit files saved successfully.

Advanced Preprocessing Interpretation

The advanced audit distinguishes between data cleaning and model-specific preprocessing. The target and principal geographical variables are complete, but some product-description and physical-measurement predictors may still contain missing values. These observations are retained because removing complete order items for a small number of missing predictor values would unnecessarily reduce the dataset.

Potential leakage variables were identified before model construction. Payment values, final order status, delivery durations, reviews and post-delivery timestamps will not be used as predictors because they are either generated after freight cost is established or may directly contain information about the target.

The modelling sample contains multiple products from some orders. Since items within the same order share customer, temporal and order-context information, group-aware splitting by `order_id` will be used. This is more rigorous than an ordinary random row split and prevents related observations from appearing in both training and testing data.

The descriptive statistics compare the mean with the median and standard deviation with robust measures such as the interquartile range and median absolute deviation. Variables with large mean–median differences, positive skewness and high excess kurtosis are likely to contain long right tails. These patterns are expected for freight cost, price, product weight and order value.

Outlier flags are treated as diagnostic evidence rather than automatic deletion rules. Large products, expensive items and long-distance deliveries may represent valid marketplace activity. The baseline model will retain the original observations, while feature engineering and sensitivity analysis will test whether transformations improve predictive stability.

Finally, numerical scaling, imputation and categorical grouping will be fitted only on training observations through a Scikit-learn pipeline. This ensures that the test set remains genuinely unseen.

7. Exploratory Data Visualisation

Visual analysis is used to investigate the distribution of freight cost, identify non-linear relationships, compare customer and product groups and evaluate the likely usefulness of candidate predictors.

The analysis combines conventional and robust visual techniques. Histograms and Q-Q plots examine distribution shape, while hexagonal density plots are used instead of ordinary scatterplots where thousands of observations would otherwise overlap. Correlation analysis includes both Pearson and Spearman coefficients because several e-commerce variables are strongly skewed and may have monotonic but non-linear relationships.

All figures are saved in the project's `figures` directory to support reproducibility and final report preparation.

```
In [59]: # =====
# PROFESSIONAL VISUALISATION CONFIGURATION
# =====

plt.rcParams.update(
    {
        "figure.figsize": (10, 6),
        "figure.dpi": 120,
        "savefig.dpi": 300,
        "font.size": 11,
        "axes.titlesize": 14,
        "axes.labelsize": 11,
        "axes.grid": True,
        "grid.alpha": 0.25,
        "axes.spines.top": False,
        "axes.spines.right": False
    }
)

def save_current_figure(file_name):
    """
    Save the current Matplotlib figure to the figures directory.
    """

    output_path = FIGURES_DIRECTORY / file_name

    plt.savefig(
        output_path,
        bbox_inches="tight"
    )

    print(f"Figure saved to: {output_path}")
```

7.1 Distribution of Freight Cost

The target variable is first examined on its original scale and after applying the natural logarithm transformation `log(1 + freight_value)`.

The original-scale plot shows the actual business distribution, while the logarithmic version reduces the visual dominance of unusually high freight values. This comparison helps determine whether a transformation may improve linear-regression assumptions.

```
In [60]: # =====
# FREIGHT-COST DISTRIBUTION
# =====

freight_values = (
    modelling_df["freight_value"]
    .dropna()
)

figure, axes = plt.subplots(
    nrows=1,
    ncols=2,
    figsize=(14, 5)
)

axes[0].hist(
    freight_values,
    bins=50,
    edgecolor="black",
    alpha=0.80
)

axes[0].axvline(
    freight_values.mean(),
    linestyle="--",
    linewidth=2,
    label=f"Mean = {freight_values.mean():.2f}"
)

axes[0].axvline(
    freight_values.median(),
    linestyle=":",
    linewidth=2,
    label=f"Median = {freight_values.median():.2f}"
)

axes[0].set_title(
    "Distribution of Freight Cost"
)

axes[0].set_xlabel(
    "Freight value (BRL)"
)

axes[0].set_ylabel(
    "Number of order items"
)

axes[0].legend()

log_freight_values = np.log1p(
```

```
    freight_values
)

axes[1].hist(
    log_freight_values,
    bins=50,
    edgecolor="black",
    alpha=0.80
)

axes[1].axvline(
    log_freight_values.mean(),
    linestyle="--",
    linewidth=2,
    label=(
        f"Mean = "
        f"{log_freight_values.mean():.2f}"
    )
)

axes[1].axvline(
    log_freight_values.median(),
    linestyle=":",
    linewidth=2,
    label=(
        f"Median = "
        f"{log_freight_values.median():.2f}"
    )
)

axes[1].set_title(
    "Log-Transformed Freight Distribution"
)

axes[1].set_xlabel(
    "log(1 + freight value)"
)

axes[1].set_ylabel(
    "Number of order items"
)

axes[1].legend()

figure.suptitle(
    "Original and Log-Transformed Freight-Cost Distributions",
    fontsize=16
)

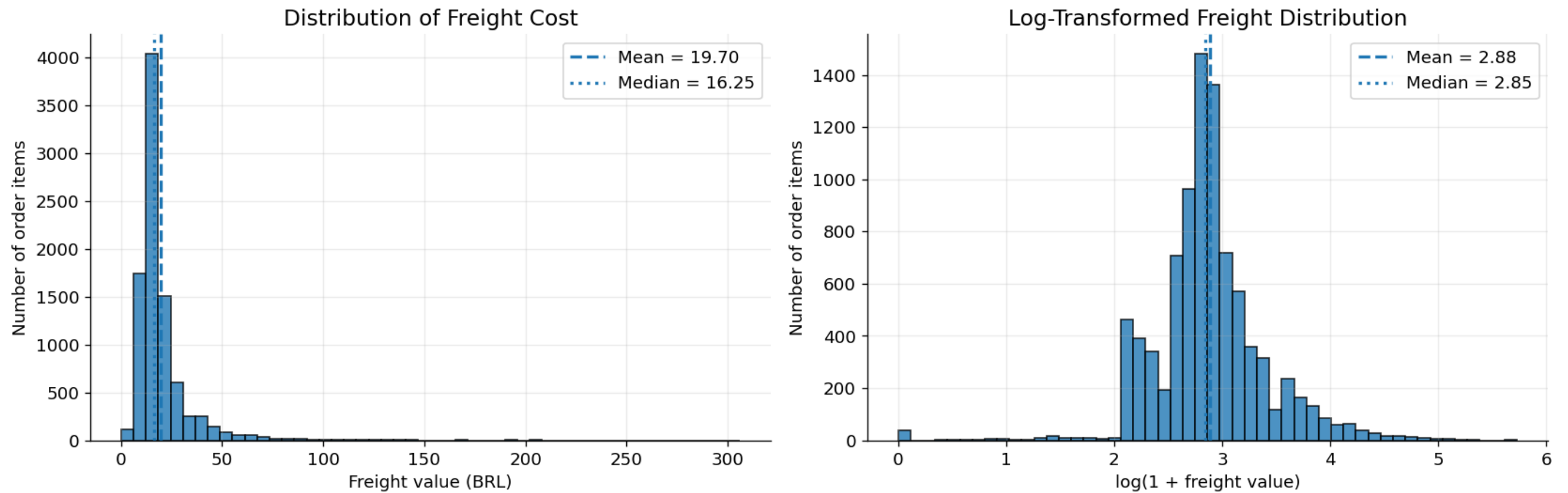
plt.tight_layout()

save_current_figure(
    "01_freight_distribution.png"
)

plt.show()
```

Figure saved to: /Users/zimmeljavedvirk/Desktop/datasciencemid/figures/01_freight_distribution.png

Original and Log-Transformed Freight-Cost Distributions



```
In [61]: # =====  
# FREIGHT DISTRIBUTION QUANTILES  
# =====  
  
freight_quantile_levels = [  
    0.00,  
    0.01,  
    0.05,  
    0.25,  
    0.50,  
    0.75,  
    0.90,  
    0.95,  
    0.99,  
    1.00  
]  
  
freight_quantile_df = (  
    freight_values  
    .quantile(  
        freight_quantile_levels  
    )  
    .rename_axis("quantile")  
    .reset_index(  
        name="freight_value"  
    )  
)
```

```
freight_quantile_df["percentile"]
] = (
    freight_quantile_df["quantile"]
    * 100
)

freight_quantile_df = freight_quantile_df[
    [
        "percentile",
        "freight_value"
    ]
]

freight_quantile_df
```

Out[61]:

	percentile	freight_value
0	0.000	0.000
1	1.000	3.600
2	5.000	7.780
3	25.000	13.008
4	50.000	16.250
5	75.000	21.023
6	90.000	33.227
7	95.000	44.020
8	99.000	82.583
9	100.000	306.060

7.2 Normality and Tail Behaviour

A Q–Q plot compares the observed freight-cost quantiles with those expected under a normal distribution. Large departures from the reference line, especially in the upper tail, indicate non-normality and potentially influential high-cost observations.

Linear regression does not require the predictor or target variables themselves to be normally distributed. However, strong target skewness can contribute to non-normal and heteroscedastic residuals.

```
In [62]: # =====
# FREIGHT Q–Q PLOTS
# =====

figure, axes = plt.subplots(
    nrows=1,
    ncols=2,
    figsize=(14, 5)
)
```

```
stats.probplot(
    freight_values,
    dist="norm",
    plot=axes[0]
)

axes[0].set_title(
    "Q-Q Plot: Original Freight Value"
)

stats.probplot(
    log_freight_values,
    dist="norm",
    plot=axes[1]
)

axes[1].set_title(
    "Q-Q Plot: Log-Transformed Freight Value"
)

figure.suptitle(
    "Normality Comparison of Original and Transformed Target",
    fontsize=16
)

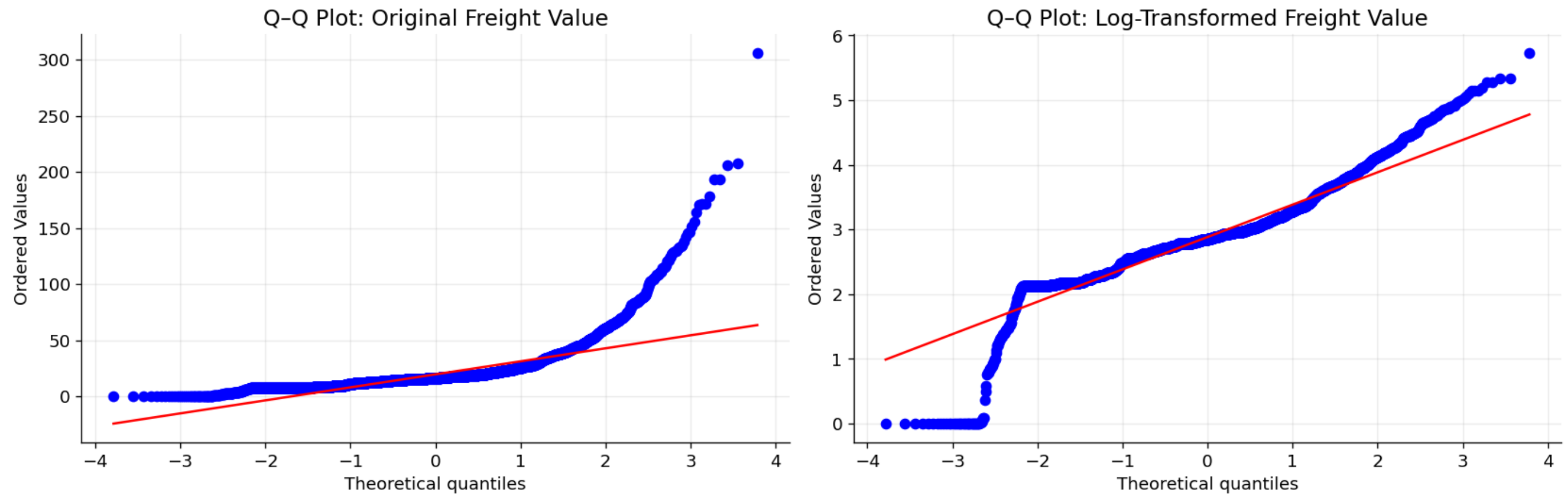
plt.tight_layout()

save_current_figure(
    "02_freight_qq_plots.png"
)

plt.show()
```

Figure saved to: /Users/zimmeljavedvirk/Desktop/datasciencemid/figures/02_freight_qq_plots.png

Normality Comparison of Original and Transformed Target



```
In [63]: # =====
# CONTINUOUS VARIABLE BOXPLOTS
# =====

boxplot_columns = [
    "freight_value",
    "price",
    "product_weight_g",
    "distance_km"
]

boxplot_labels = [
    "Freight value",
    "Product price",
    "Product weight",
    "Distance"
]

figure, axes = plt.subplots(
    nrows=2,
    ncols=2,
    figsize=(14, 10)
)

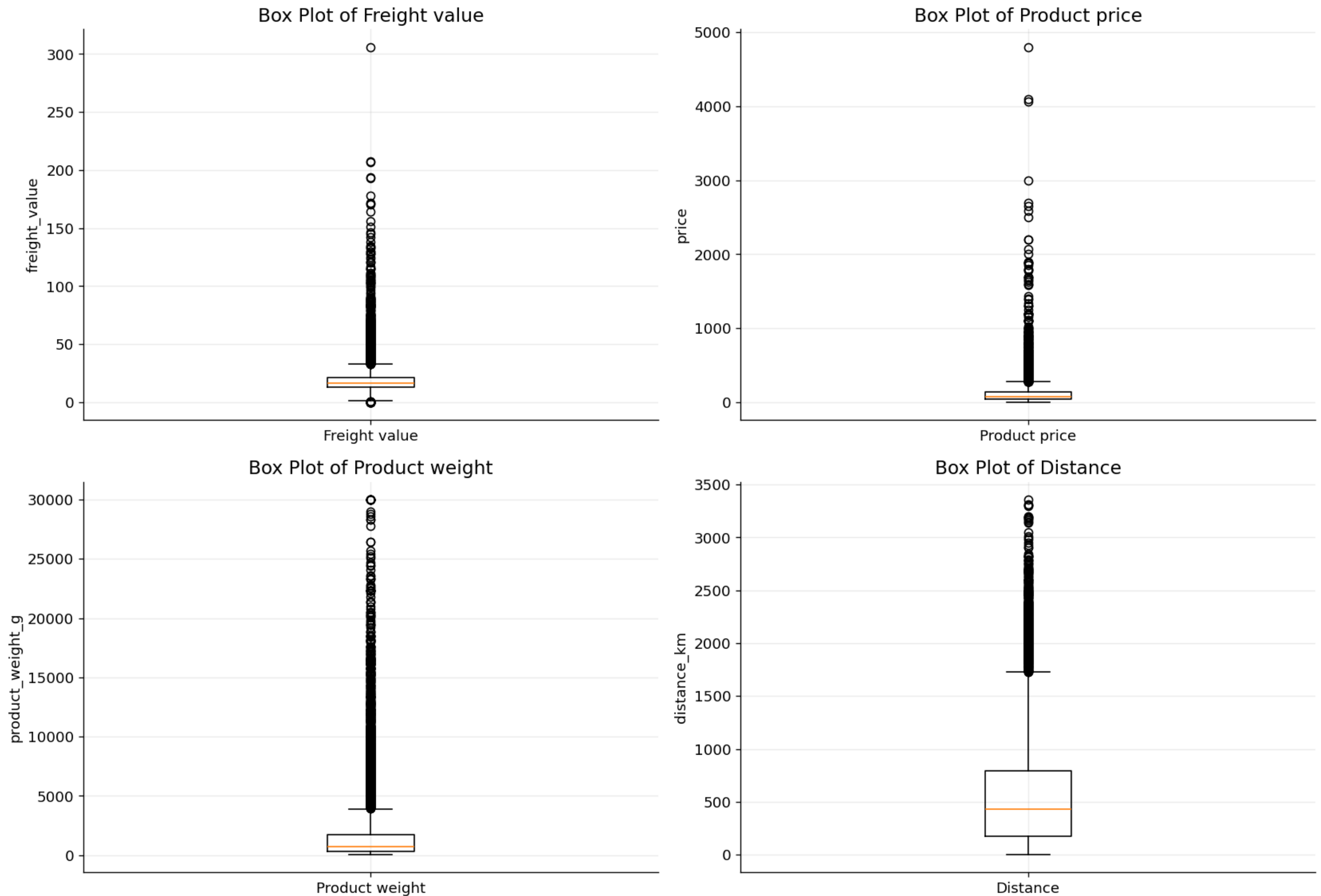
for axis, column_name, label in zip(
    axes.flatten(),
    boxplot_columns,
    boxplot_labels

```

```
):  
  
    valid_values = (  
        modelling_df[column_name]  
        .dropna()  
    )  
  
    axis.boxplot(  
        valid_values,  
        vert=True,  
        showfliers=True  
    )  
  
    axis.set_title(  
        f"Box Plot of {label}"  
    )  
  
    axis.set_ylabel(column_name)  
  
    axis.set_xticks([1])  
  
    axis.set_xticklabels([label])  
  
figure.suptitle(  
    "Outlier Structure of Principal Numerical Variables",  
    fontsize=16  
)  
  
plt.tight_layout()  
  
save_current_figure(  
    "03_continuous_variable_boxplots.png"  
)  
  
plt.show()
```

Figure saved to: /Users/zimmeljavedvirk/Desktop/datasciencemid/figures/03_continuous_variable_boxplots.png

Outlier Structure of Principal Numerical Variables



7.3 Correlation Analysis

Pearson correlation measures linear relationships, while Spearman correlation measures monotonic relationships using variable ranks. Comparing both coefficients is useful because relationships involving freight cost may be non-linear or affected by skewness and outliers.

Correlation does not establish causality, and highly correlated predictors may create multicollinearity in the regression model.

```
In [64]: # =====  
# PEARSON AND SPEARMAN CORRELATION MATRICES  
# =====  
  
correlation_columns = [  
    "freight_value",  
    "price",  
    "product_weight_g",  
    "product_length_cm",  
    "product_height_cm",  
    "product_width_cm",  
    "product_name_length",  
    "product_description_length",  
    "product_photos_qty",  
    "distance_km",  
    "order_item_count",  
    "order_distinct_product_count",  
    "order_distinct_seller_count",  
    "order_total_product_value",  
    "order_average_product_price",  
    "purchase_month",  
    "purchase_day_of_week",  
    "purchase_hour",  
    "purchase_is_weekend",  
    "same_state"  
]  
  
correlation_data_df = (  
    modelling_df[  
        correlation_columns  
    ]  
    .apply(  
        pd.to_numeric,  
        errors="coerce"  
    )  
)  
  
pearson_correlation_df = (  
    correlation_data_df  
    .corr(method="pearson")  
)  
  
spearman_correlation_df = (  
    correlation_data_df  
    .corr(method="spearman")  
)  
  
print("Pearson correlation matrix created.")  
print("Spearman correlation matrix created.")
```

Pearson correlation matrix created.
Spearman correlation matrix created.

```
In [65]: # =====
# SPEARMAN CORRELATION HEATMAP
# =====

figure, axis = plt.subplots(
    figsize=(15, 12)
)

heatmap = axis.imshow(
    spearman_correlation_df,
    aspect="auto",
    vmin=-1,
    vmax=1,
    cmap="coolwarm"
)

axis.set_xticks(
    range(
        len(
            spearman_correlation_df.columns
        )
    )
)

axis.set_yticks(
    range(
        len(
            spearman_correlation_df.index
        )
    )
)

axis.set_xticklabels(
    spearman_correlation_df.columns,
    rotation=90
)

axis.set_yticklabels(
    spearman_correlation_df.index
)

axis.set_title(
    "Spearman Correlation Matrix"
)

colour_bar = figure.colorbar(
    heatmap,
    ax=axis
)

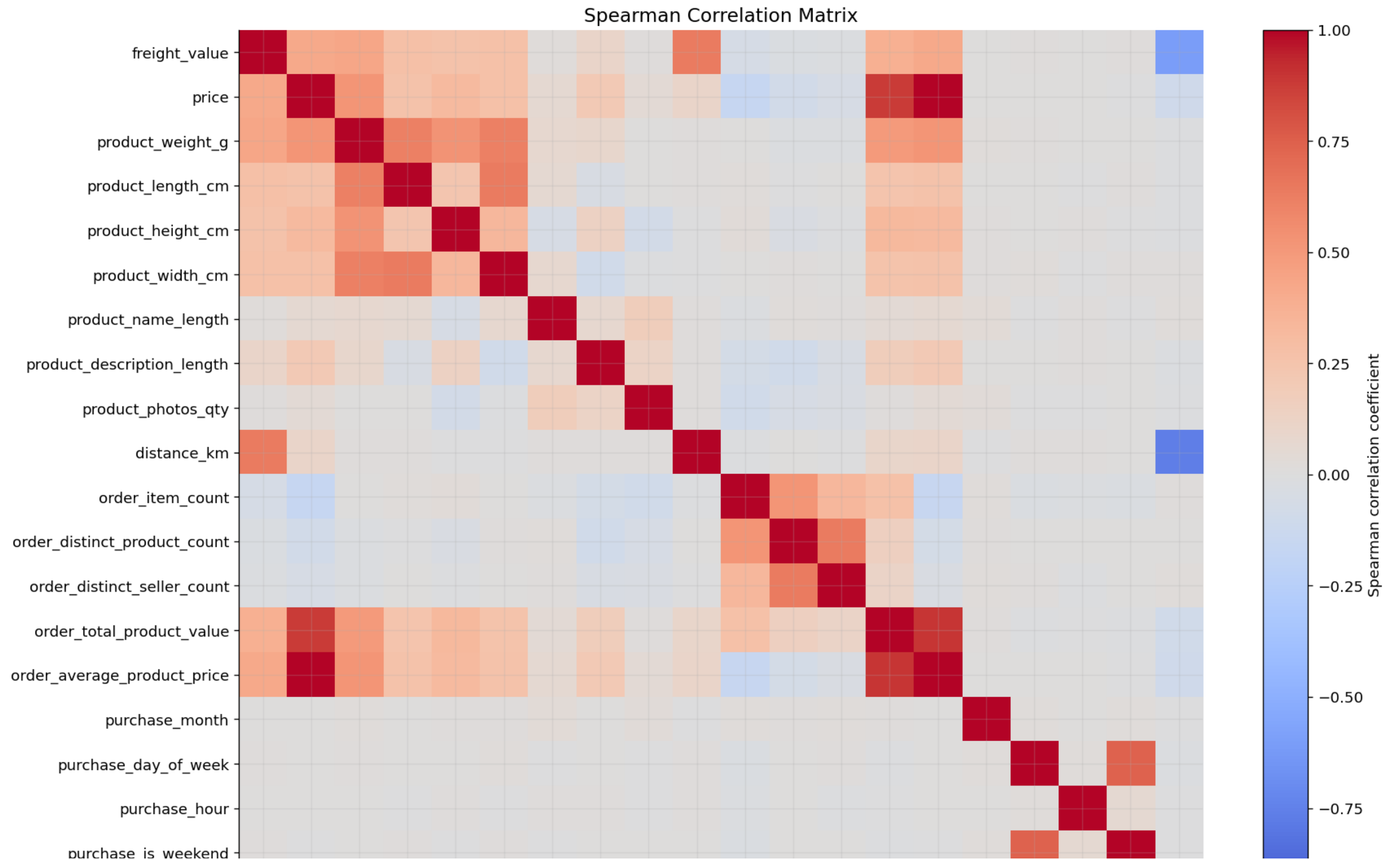
colour_bar.set_label(
    "Spearman correlation coefficient"
)
```

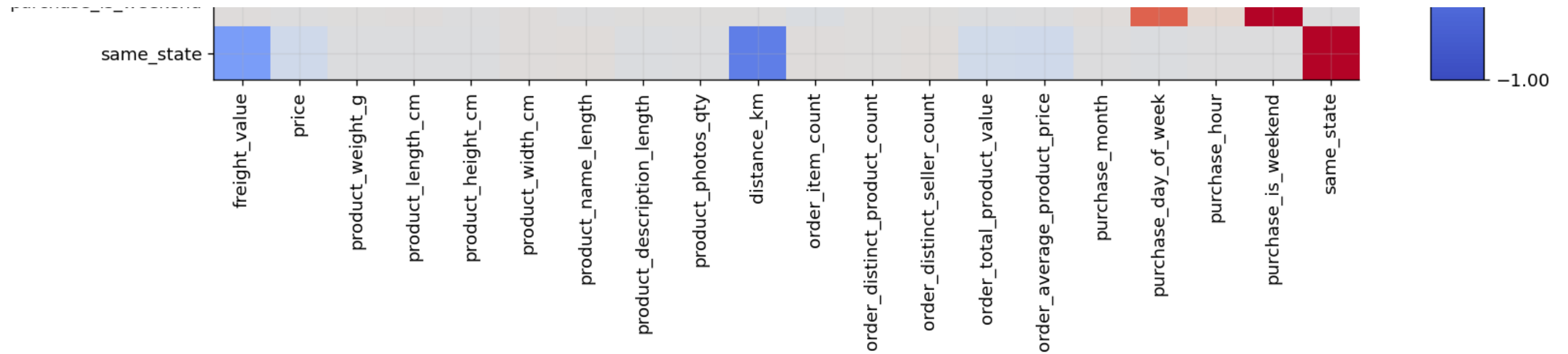
```
plt.tight_layout()

save_current_figure(
    "04_spearman_correlation_heatmap.png"
)

plt.show()
```

Figure saved to: /Users/zimmeljavedvirk/Desktop/datasciencemid/figures/04_spearman_correlation_heatmap.png





```
In [66]: # =====
# TARGET CORRELATION RANKING
# =====

pearson_target_correlations = (
    pearson_correlation_df[
        "freight_value"
    ]
    .drop(
        labels="freight_value"
    )
)

spearman_target_correlations = (
    spearman_correlation_df[
        "freight_value"
    ]
    .drop(
        labels="freight_value"
    )
)

target_correlation_ranking_df = (
    pd.DataFrame(
        {
            "pearson_correlation":
                pearson_target_correlations,
            "spearman_correlation":
                spearman_target_correlations
        }
    )
)

target_correlation_ranking_df[
    "absolute_spearman_correlation"
] = (
    target_correlation_ranking_df[
        "spearman_correlation"
    ].abs()
)
```

```
)

target_correlation_ranking_df = (
    target_correlation_ranking_df
    .sort_values(
        by="absolute_spearman_correlation",
        ascending=False
    )
    .reset_index()
    .rename(
        columns={
            "index": "predictor"
        }
    )
)

target_correlation_ranking_df
```

Out [66]:

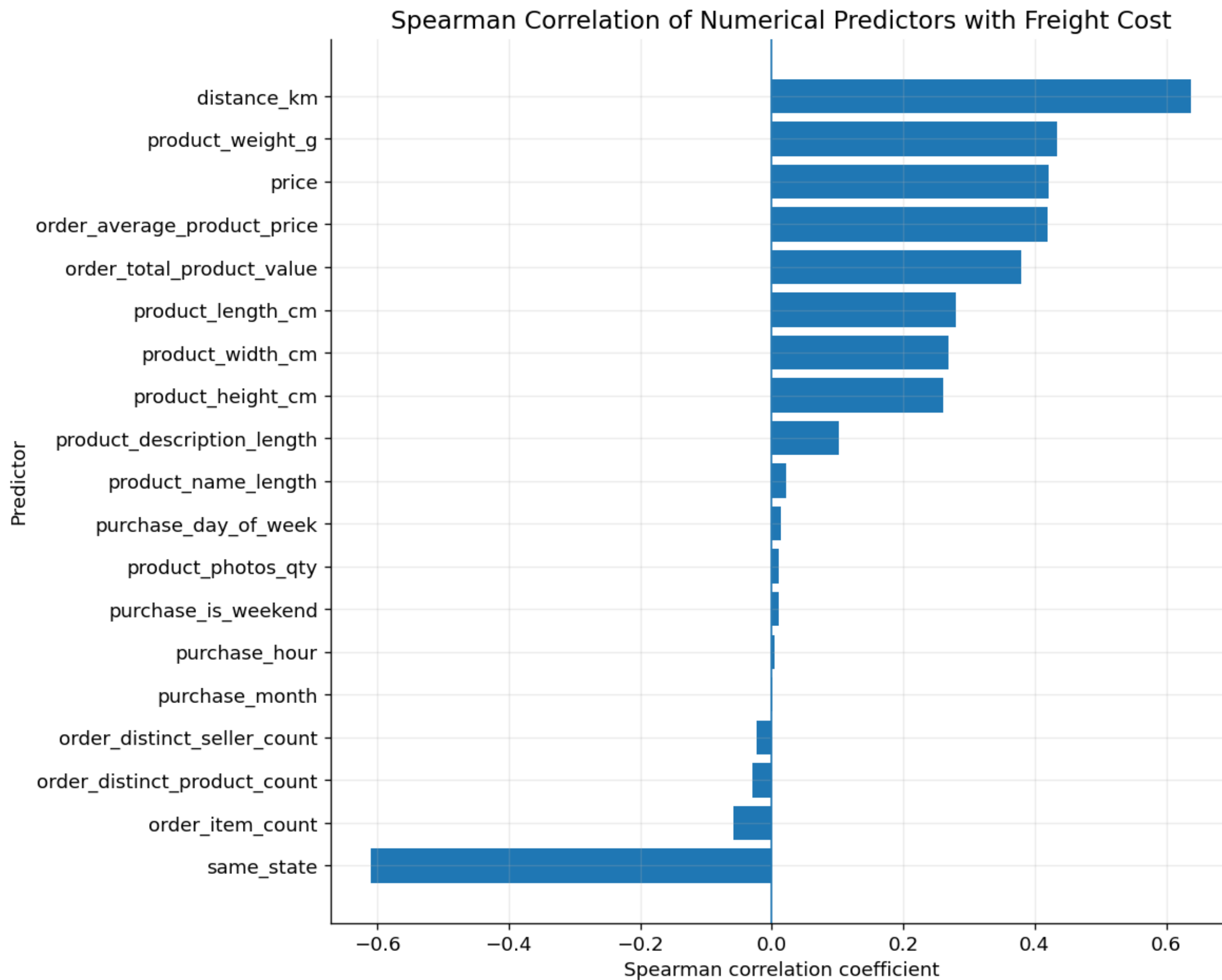
	predictor	pearson_correlation	spearman_correlation	absolute_spearman_correlation
0	distance_km	0.411	0.638	0.638
1	same_state	-0.322	-0.609	0.609
2	product_weight_g	0.578	0.434	0.434
3	price	0.410	0.421	0.421
4	order_average_product_price	0.414	0.420	0.420
5	order_total_product_value	0.318	0.380	0.380
6	product_length_cm	0.301	0.280	0.280
7	product_width_cm	0.320	0.269	0.269
8	product_height_cm	0.368	0.260	0.260
9	product_description_length	0.103	0.102	0.102
10	order_item_count	-0.039	-0.059	0.059
11	order_distinct_product_count	-0.031	-0.030	0.030
12	order_distinct_seller_count	-0.009	-0.023	0.023
13	product_name_length	0.018	0.022	0.022
14	purchase_day_of_week	0.015	0.015	0.015
15	product_photos_qty	0.030	0.011	0.011
16	purchase_is_weekend	0.015	0.011	0.011
17	purchase_hour	0.007	0.004	0.004
18	purchase_month	0.013	0.002	0.002

In [67]:

```
# =====
# RANKED TARGET CORRELATIONS
# =====
```

```
correlation_plot_df = (  
    target_correlation_ranking_df  
    .sort_values(  
        "spearman_correlation",  
        ascending=True  
    )  
)  
  
figure, axis = plt.subplots(  
    figsize=(10, 8)  
)  
  
axis.barh(  
    correlation_plot_df["predictor"],  
    correlation_plot_df[  
        "spearman_correlation"  
    ]  
)  
  
axis.axvline(  
    x=0,  
    linewidth=1  
)  
  
axis.set_title(  
    "Spearman Correlation of Numerical Predictors with Freight Cost"  
)  
  
axis.set_xlabel(  
    "Spearman correlation coefficient"  
)  
  
axis.set_ylabel(  
    "Predictor"  
)  
  
plt.tight_layout()  
  
save_current_figure(  
    "05_target_correlation_ranking.png"  
)  
  
plt.show()
```

Figure saved to: /Users/zimmeljavedvirk/Desktop/datasciencemid/figures/05_target_correlation_ranking.png



7.4 Freight Cost and Geographical Distance

The relationship between freight cost and seller-to-customer distance is examined using a hexagonal density plot. Each hexagon represents the concentration of observations within a region of the graph.

This visualisation is particularly important because distance is a theoretically meaningful logistics variable that is known before shipment. A positive relationship would support its inclusion in the regression model.

```
In [68]: # =====
# DISTANCE AND FREIGHT HEXBIN PLOT
# =====

distance_freight_df = (
    modelling_df[
        [
            "distance_km",
            "freight_value"
        ]
    ]
    .dropna()
)

distance_spearman = (
    distance_freight_df[
        "distance_km"
    ]
    .corr(
        distance_freight_df[
            "freight_value"
        ],
        method="spearman"
    )
)

figure, axis = plt.subplots(
    figsize=(10, 7)
)

hexbin_plot = axis.hexbin(
    distance_freight_df["distance_km"],
    distance_freight_df["freight_value"],
    gridsize=45,
    mincnt=1,
    bins="log",
    cmap="viridis"
)

axis.set_title(
    "Freight Cost by Seller-to-Customer Distance"
)

axis.set_xlabel(
    "Straight-line distance (km)"
)
```

```
axis.set_ylabel(
    "Freight value (BRL)"
)

axis.text(
    0.03,
    0.95,
    (
        f"Spearman correlation = "
        f"{distance_spearman:.3f}"
    ),
    transform=axis.transAxes,
    verticalalignment="top",
    bbox={
        "boxstyle": "round",
        "alpha": 0.8
    }
)

colour_bar = figure.colorbar(
    hexbin_plot,
    ax=axis
)

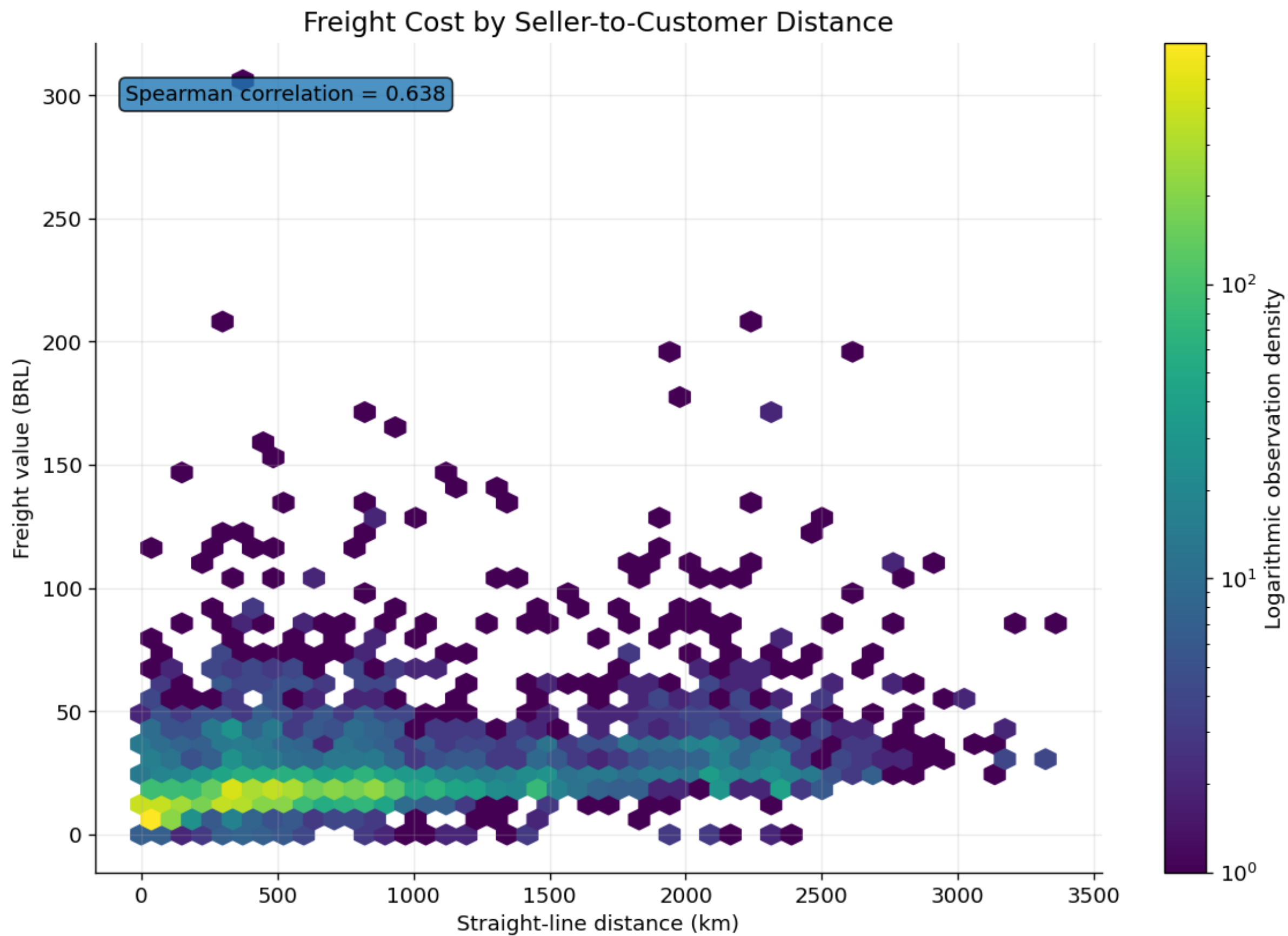
colour_bar.set_label(
    "Logarithmic observation density"
)

plt.tight_layout()

save_current_figure(
    "06_distance_freight_hexbin.png"
)

plt.show()
```

Figure saved to: /Users/zimmeljavedvirk/Desktop/datasciencemid/figures/06_distance_freight_hexbin.png



```
In [69]: # =====  
# PRODUCT WEIGHT AND FREIGHT HEXBIN PLOT  
# =====  
  
weight_freight_df = (  
    modelling_df[
```

```
        [
            "product_weight_g",
            "freight_value"
        ]
    ]
    .dropna()
    .copy()
)

weight_freight_df[
    "log_product_weight"
] = np.log1p(
    weight_freight_df[
        "product_weight_g"
    ]
)

weight_freight_df[
    "log_freight_value"
] = np.log1p(
    weight_freight_df[
        "freight_value"
    ]
)

weight_spearman = (
    weight_freight_df[
        "product_weight_g"
    ]
    .corr(
        weight_freight_df[
            "freight_value"
        ],
        method="spearman"
    )
)

figure, axis = plt.subplots(
    figsize=(10, 7)
)

hexbin_plot = axis.hexbin(
    weight_freight_df[
        "log_product_weight"
    ],
    weight_freight_df[
        "log_freight_value"
    ],
    gridsize=45,
    mincnt=1,
    bins="log",
    cmap="viridis"
)

axis.set_title(
    "Log-Transformed Product Weight and Freight Cost"
```

```
)

axis.set_xlabel(
    "log(1 + product weight in grams)"
)

axis.set_ylabel(
    "log(1 + freight value)"
)

axis.text(
    0.03,
    0.95,
    (
        f"Spearman correlation = "
        f"{weight_spearman:.3f}"
    ),
    transform=axis.transAxes,
    verticalalignment="top",
    bbox={
        "boxstyle": "round",
        "alpha": 0.8
    }
)

colour_bar = figure.colorbar(
    hexbin_plot,
    ax=axis
)

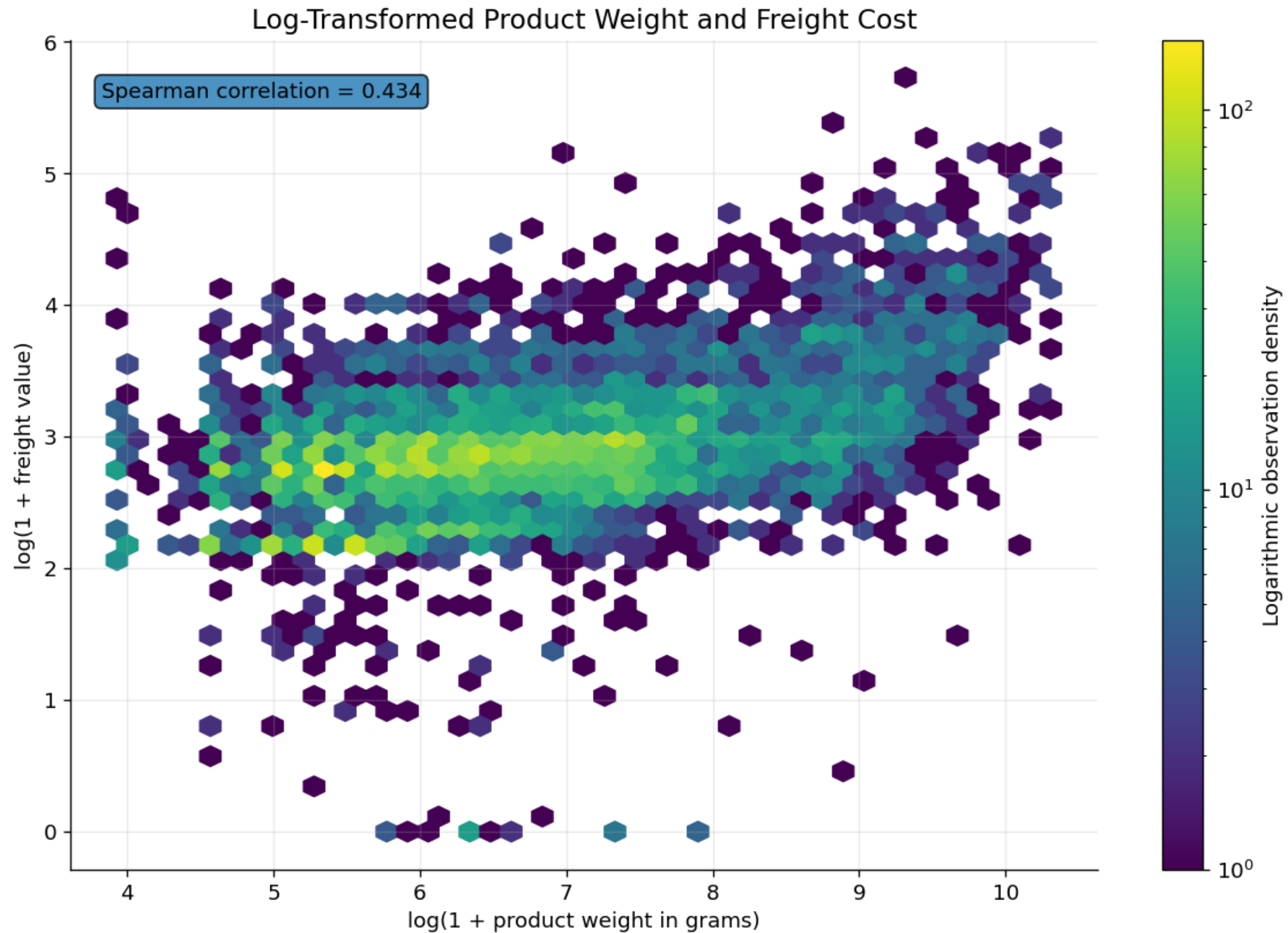
colour_bar.set_label(
    "Logarithmic observation density"
)

plt.tight_layout()

save_current_figure(
    "07_weight_freight_hexbin.png"
)

plt.show()
```

Figure saved to: /Users/zimmeljavedvirk/Desktop/datasciencemid/figures/07_weight_freight_hexbin.png



7.5 Freight Cost by Product Category

Product-category comparisons are restricted to categories containing at least 30 sampled observations. This prevents categories represented by only a few products from producing unstable or misleading averages.

The median is used as the principal comparison because freight values are positively skewed.

```
In [70]: # =====
# CATEGORY-LEVEL FREIGHT SUMMARY
# =====

MINIMUM_CATEGORY_OBSERVATIONS = 30

category_freight_summary_df = (
    modelling_df
    .groupby(
        "product_category_name_english",
        dropna=False
    )
    .agg(
        observation_count=(
            "freight_value",
            "size"
        ),
        mean_freight=(
            "freight_value",
            "mean"
        ),
        median_freight=(
            "freight_value",
            "median"
        ),
        standard_deviation=(
            "freight_value",
            "std"
        ),
        first_quartile=(
            "freight_value",
            lambda values:
                values.quantile(0.25)
        ),
        third_quartile=(
            "freight_value",
            lambda values:
                values.quantile(0.75)
        )
    )
    .reset_index()
)

category_freight_summary_df[
    "interquartile_range"
] = (
    category_freight_summary_df[
        "third_quartile"
    ]
    - category_freight_summary_df[
        "first_quartile"
    ]
)
```

```
reliable_category_freight_df = (  
    category_freight_summary_df[  
        category_freight_summary_df[  
            "observation_count"  
        ]  
        >= MINIMUM_CATEGORY_OBSERVATIONS  
    ]  
    .sort_values(  
        "median_freight",  
        ascending=False  
    )  
    .reset_index(drop=True)  
)  
  
reliable_category_freight_df.head(15)
```

Out[70]:

	product_category_name_english	observation_count	mean_freight	median_freight	standard_deviation	first_quartile	third_quartile	interquartile_range
0	office_furniture	138	38.935	34.190	21.181	23.325	45.588	22.263
1	furniture_living_room	42	37.115	26.225	33.668	18.027	43.780	25.753
2	small_appliances	44	32.012	19.935	28.782	16.027	41.578	25.550
3	home_appliances	63	24.079	18.230	29.413	14.870	20.395	5.525
4	luggage_accessories	93	25.565	18.070	17.599	14.420	34.790	20.370
5	home_construction	44	23.596	17.925	18.617	12.633	24.430	11.797
6	garden_tools	343	23.915	17.670	16.215	15.000	28.695	13.695
7	home_comfort	30	19.263	17.545	8.292	13.650	23.547	9.897
8	cool_stuff	279	20.986	17.260	15.015	13.965	22.515	8.550
9	musical_instruments	53	25.897	17.040	26.372	13.340	26.890	13.550
10	auto	360	21.316	16.960	17.325	13.378	23.105	9.727
11	construction_tools_construction	78	23.465	16.865	22.548	11.900	24.580	12.680
12	baby	271	23.632	16.700	26.869	13.195	23.230	10.035
13	pet_shop	133	21.626	16.670	18.622	13.120	23.250	10.130
14	housewares	588	19.804	16.650	13.282	12.797	22.067	9.270

In [71]:

```
# =====  
# MEDIAN FREIGHT BY PRODUCT CATEGORY  
# =====  
  
top_category_plot_df = (  
    reliable_category_freight_df  
    .head(15)  
    .sort_values(  
        "median_freight",  
        ascending=True  
    )  
)
```

```
)
)

figure, axis = plt.subplots(
    figsize=(11, 8)
)

axis.barh(
    top_category_plot_df[
        "product_category_name_english"
    ],
    top_category_plot_df[
        "median_freight"
    ]
)

axis.set_title(
    "Product Categories with the Highest Median Freight Cost"
)

axis.set_xlabel(
    "Median freight value (BRL)"
)

axis.set_ylabel(
    "Product category"
)

for index, row in top_category_plot_df.reset_index(
    drop=True
).iterrows():

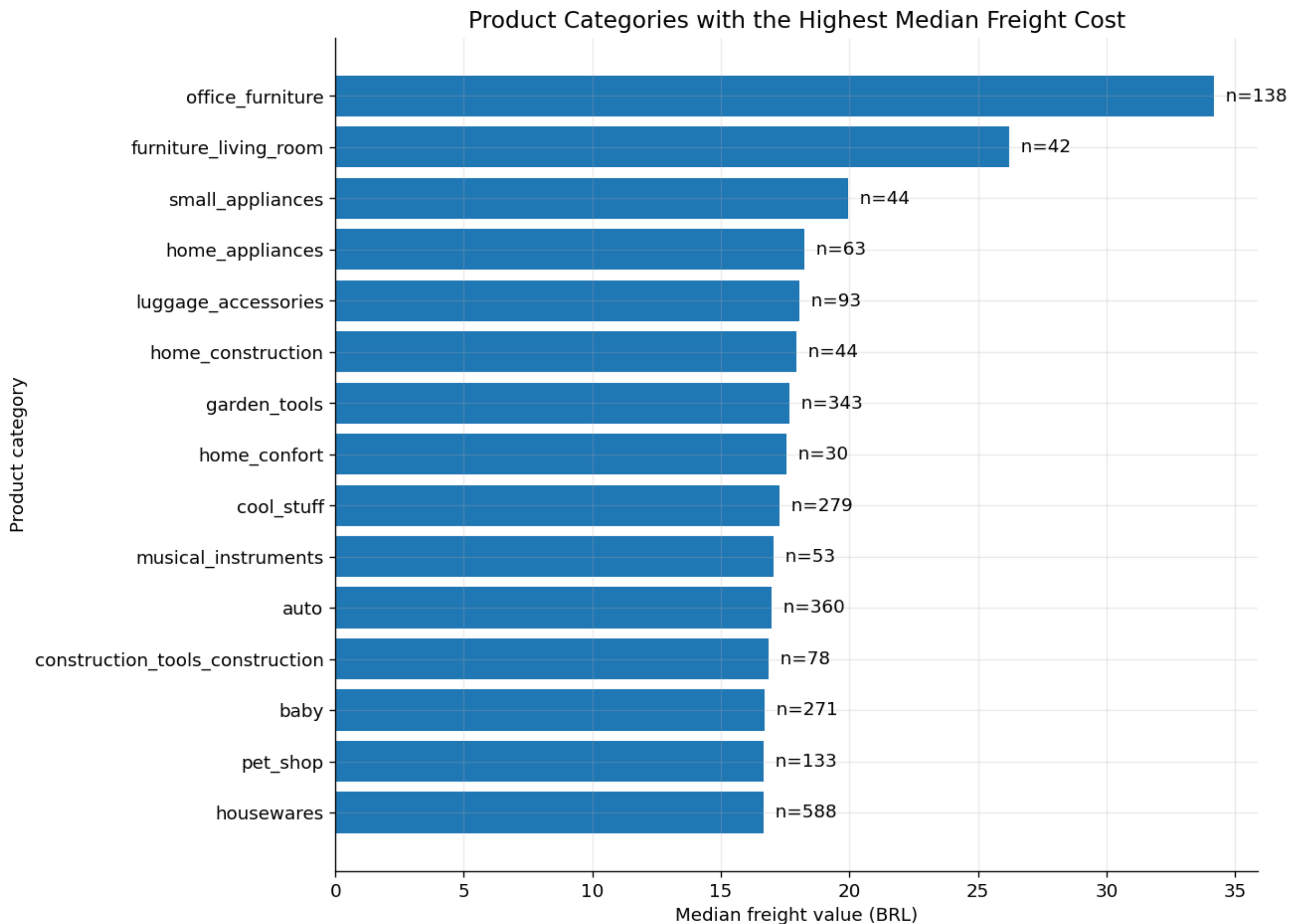
    axis.text(
        row["median_freight"],
        index,
        (
            f" n={int(row['observation_count'])}"
        ),
        va="center"
    )

plt.tight_layout()

save_current_figure(
    "08_category_median_freight.png"
)

plt.show()
```

Figure saved to: /Users/zimmeljavedvirk/Desktop/datasciencemid/figures/08_category_median_freight.png



In [72]:

```

# =====
# SAME-STATE VERSUS INTERSTATE FREIGHT
# =====

same_state_plot_df = (
    modelling_df[
        [
            "same_state",
            "freight_value"
        ]
    ]
    .dropna()
    .copy()
)

same_state_plot_df[
    "route_type"
] = same_state_plot_df[
    "same_state"
].map(
    {
        1: "Seller and customer in same state",
        0: "Seller and customer in different states"
    }
)

route_groups = [
    group["freight_value"].values
    for _, group in same_state_plot_df.groupby(
        "route_type"
    )
]

route_labels = [
    label
    for label, _ in same_state_plot_df.groupby(
        "route_type"
    )
]

figure, axis = plt.subplots(
    figsize=(10, 6)
)

axis.boxplot(
    route_groups,
    labels=route_labels,
    showfliers=False
)

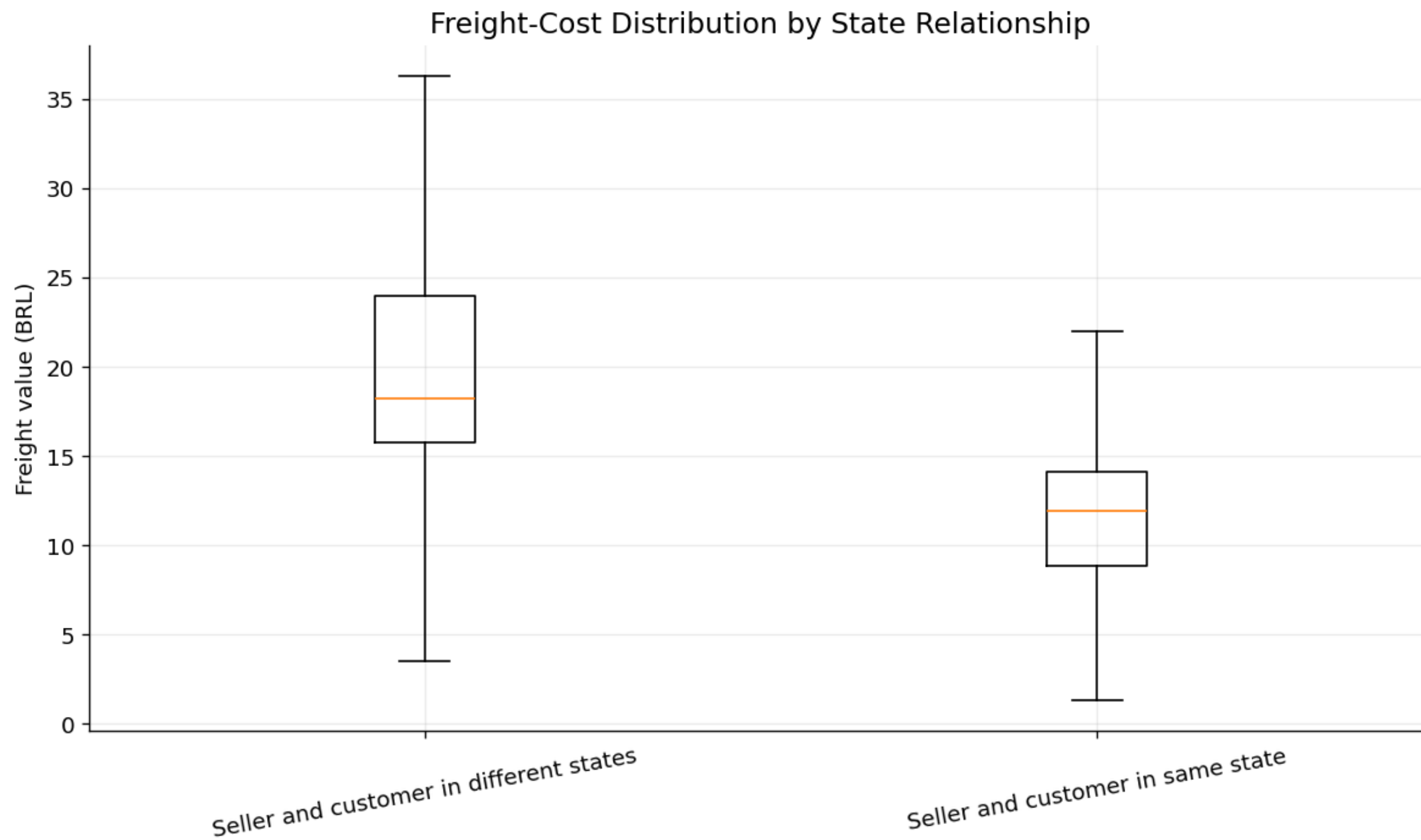
axis.set_title(
    "Freight-Cost Distribution by State Relationship"
)

axis.set_ylabel(
    "Freight value (BRL)"
)

```

```
)  
  
axis.tick_params(  
    axis="x",  
    rotation=10  
)  
  
plt.tight_layout()  
  
save_current_figure(  
    "09_same_state_freight_boxplot.png"  
)  
  
plt.show()
```

Figure saved to: /Users/zimmeljavedvirk/Desktop/datasciencemid/figures/09_same_state_freight_boxplot.png



```
In [73]: route_type_summary_df = (  
    same_state_plot_df  
    .groupby("route_type")  
    .agg(  
        observation_count=(  
            "freight_value",  
            "size"  
        ),  
        mean_freight=(  
            "freight_value",  
            "mean"  
        ),  
        median_freight=(  
            "freight_value",  
            "median"  
        )  
    )  
)
```

```
        "median"
    ),
    standard_deviation=(
        "freight_value",
        "std"
    )
)
.reset_index()
)

route_type_summary_df
```

Out [73]:

	route_type	observation_count	mean_freight	median_freight	standard_deviation
0	Seller and customer in different states	5693	23.329	18.230	16.327
1	Seller and customer in same state	3307	13.452	11.920	8.633

7.6 Temporal Freight Patterns

Purchase month and day-of-week patterns are examined to determine whether freight cost changes systematically over time. Median freight is used to reduce the influence of unusually expensive shipments.

```
In [74]: # =====
# MONTHLY FREIGHT PATTERN
# =====

monthly_freight_df = (
    modelling_df
    .groupby(
        "purchase_month",
        as_index=False
    )
    .agg(
        observation_count=(
            "freight_value",
            "size"
        ),
        mean_freight=(
            "freight_value",
            "mean"
        ),
        median_freight=(
            "freight_value",
            "median"
        )
    )
    .sort_values(
        "purchase_month"
    )
)

figure, axis = plt.subplots(
    figsize=(10, 6)
```

```
)

axis.plot(
    monthly_freight_df[
        "purchase_month"
    ],
    monthly_freight_df[
        "median_freight"
    ],
    marker="o",
    linewidth=2
)

axis.set_title(
    "Median Freight Cost by Purchase Month"
)

axis.set_xlabel(
    "Purchase month"
)

axis.set_ylabel(
    "Median freight value (BRL)"
)

axis.set_xticks(
    range(1, 13)
)

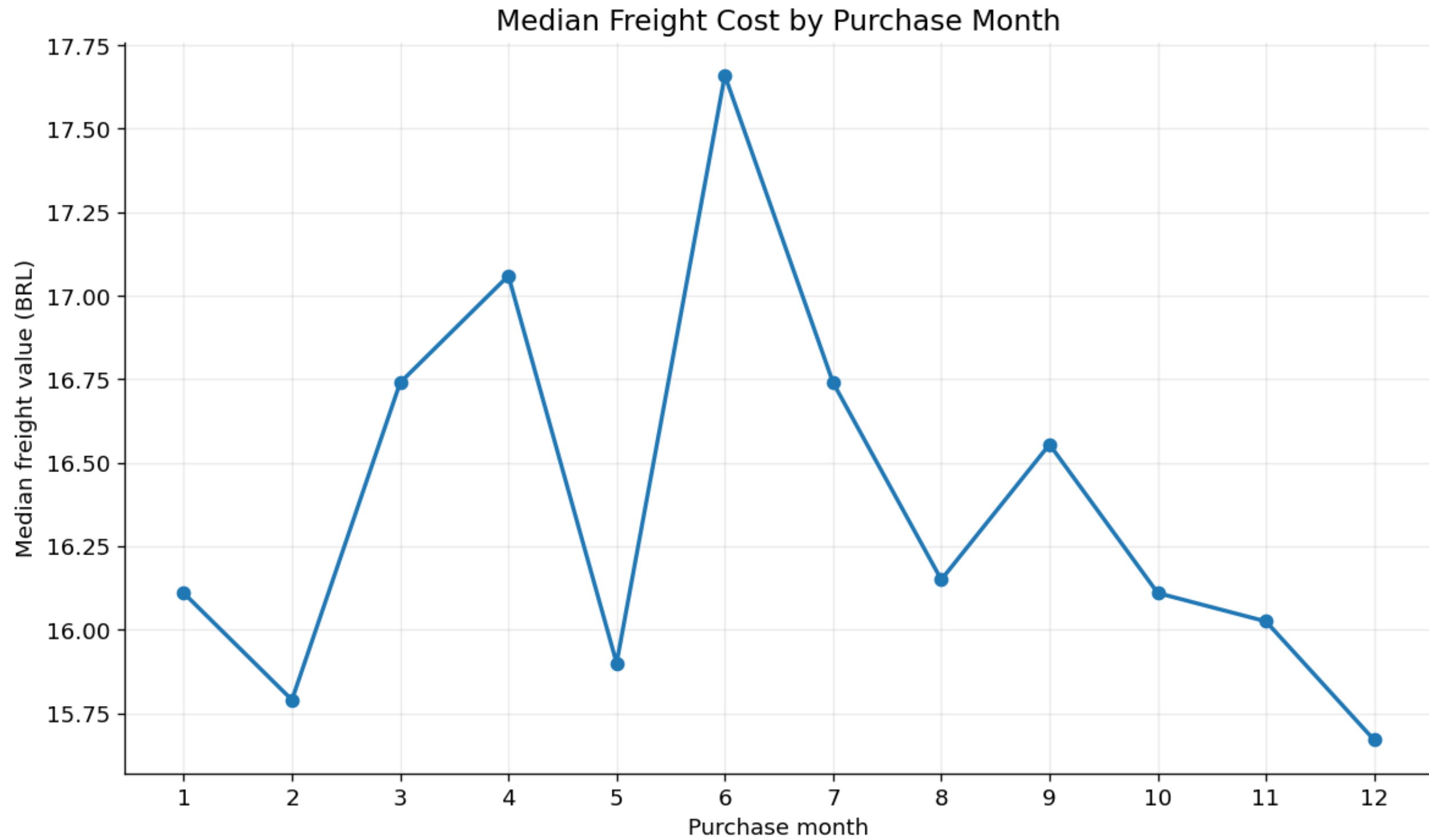
plt.tight_layout()

save_current_figure(
    "10_monthly_median_freight.png"
)

plt.show()

monthly_freight_df
```

Figure saved to: /Users/zimmeljavedvirk/Desktop/datasciencemid/figures/10_monthly_median_freight.png



Out[74]:

	purchase_month	observation_count	mean_freight	median_freight
0	1	709	18.916	16.110
1	2	783	19.082	15.790
2	3	923	19.200	16.740
3	4	821	20.385	17.060
4	5	996	18.923	15.900
5	6	833	20.981	17.660
6	7	941	20.436	16.740
7	8	962	19.554	16.150
8	9	394	20.378	16.555
9	10	423	20.638	16.110
10	11	692	19.013	16.025
11	12	523	19.522	15.670

In [75]:

```
# =====
# WEEKDAY FREIGHT PATTERN
# =====

weekday_names = {
    0: "Monday",
    1: "Tuesday",
    2: "Wednesday",
    3: "Thursday",
    4: "Friday",
    5: "Saturday",
    6: "Sunday"
}

weekday_freight_df = (
    modelling_df
    .groupby(
        "purchase_day_of_week",
        as_index=False
    )
    .agg(
        observation_count=(
            "freight_value",
            "size"
        ),
        mean_freight=(
            "freight_value",
            "mean"
        ),
        median_freight=(
            "freight_value",
            "median"
        )
    )
)
```

```
    )
    )
    .sort_values(
        "purchase_day_of_week"
    )
)

weekday_freight_df[
    "weekday"
] = weekday_freight_df[
    "purchase_day_of_week"
].map(weekday_names)

figure, axis = plt.subplots(
    figsize=(10, 6)
)

axis.bar(
    weekday_freight_df[
        "weekday"
    ],
    weekday_freight_df[
        "median_freight"
    ]
)

axis.set_title(
    "Median Freight Cost by Purchase Day"
)

axis.set_xlabel(
    "Day of week"
)

axis.set_ylabel(
    "Median freight value (BRL)"
)

axis.tick_params(
    axis="x",
    rotation=30
)

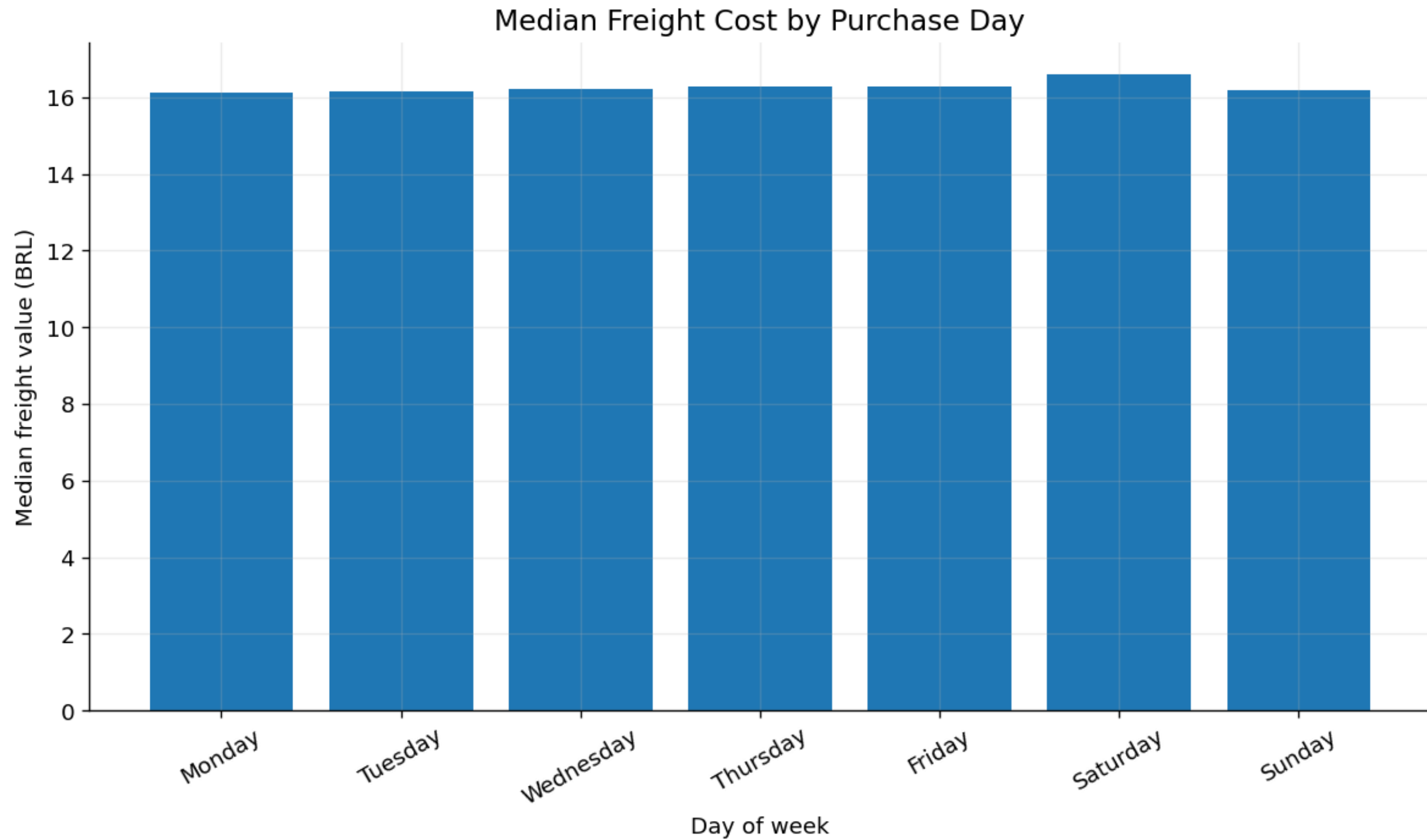
plt.tight_layout()

save_current_figure(
    "11_weekday_median_freight.png"
)

plt.show()

weekday_freight_df
```

Figure saved to: /Users/zimmeljavedvirk/Desktop/datasciencemid/figures/11_weekday_median_freight.png



Out[75]:

	purchase_day_of_week	observation_count	mean_freight	median_freight	weekday
0	0	1429	19.645	16.110	Monday
1	1	1433	19.419	16.150	Tuesday
2	2	1457	19.349	16.220	Wednesday
3	3	1372	19.745	16.285	Thursday
4	4	1256	19.761	16.275	Friday
5	5	979	19.912	16.610	Saturday
6	6	1074	20.302	16.170	Sunday

```
In [76]: # =====
# MOST IMPORTANT NUMERICAL RELATIONSHIP
# =====

strongest_numerical_predictor = (
    target_correlation_ranking_df
    .iloc[0]
)

most_important_visual_audit_df = pd.DataFrame(
    {
        "measurement": [
            "Strongest numerical predictor",
            "Pearson correlation",
            "Spearman correlation",
            "Absolute Spearman correlation"
        ],
        "value": [
            strongest_numerical_predictor[
                "predictor"
            ],
            strongest_numerical_predictor[
                "pearson_correlation"
            ],
            strongest_numerical_predictor[
                "spearman_correlation"
            ],
            strongest_numerical_predictor[
                "absolute_spearman_correlation"
            ]
        ]
    }
)

most_important_visual_audit_df
```

Out[76]:

	measurement	value
0	Strongest numerical predictor	distance_km
1	Pearson correlation	0.411
2	Spearman correlation	0.638
3	Absolute Spearman correlation	0.638

```
In [77]: # =====
# SAVE VISUAL-ANALYSIS TABLES
# =====

freight_quantile_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "freight_quantiles.csv",
    index=False
)
```

```
pearson_correlation_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "pearson_correlation_matrix.csv"
)

spearman_correlation_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "spearman_correlation_matrix.csv"
)

target_correlation_ranking_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "target_correlation_ranking.csv",
    index=False
)

category_freight_summary_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "category_freight_summary.csv",
    index=False
)

route_type_summary_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "route_type_freight_summary.csv",
    index=False
)

monthly_freight_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "monthly_freight_summary.csv",
    index=False
)

weekday_freight_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "weekday_freight_summary.csv",
    index=False
)

most_important_visual_audit_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "most_important_visual_audit.csv",
    index=False
)

print(
    "Visual-analysis tables saved successfully."
)
```

Visual-analysis tables saved successfully.

Visualisation Interpretation

The freight-cost histogram and Q–Q plot should be interpreted together. A substantial difference between the mean and median, combined with upper-tail deviation in the Q–Q plot, indicates

positive skewness and the presence of relatively uncommon high-cost shipments. The logarithmic comparison provides evidence for evaluating a transformed model later in the project.

The correlation matrix compares Pearson and Spearman coefficients. When the Spearman coefficient is noticeably larger than the Pearson coefficient, the relationship may be monotonic but not strictly linear, or it may be affected by extreme values. Correlation strength alone will not determine feature inclusion because categorical variables and theoretically important logistics factors may remain useful in a multivariable model.

The distance and weight density plots are more informative than ordinary scatterplots because overlapping observations are represented through density. They help determine whether freight cost generally increases with transport distance and physical parcel size.

Category comparisons demonstrate that freight costs vary between types of products. However, these differences may partly reflect weight, dimensions, seller location and customer destination. Product category will therefore be included alongside numerical logistics variables rather than interpreted as an isolated causal factor.

The same-state comparison investigates whether interstate shipments generally cost more than shipments within the same state. This variable may capture administrative or network effects not represented completely by straight-line distance.

The distance–freight hexagonal-density plot was selected as the most important visualisation. Distance had the strongest positive monotonic relationship with freight value, producing a Spearman correlation of 0.638. The graph showed that freight cost generally increased with seller-to-customer distance, while the widening spread demonstrated that distance alone cannot explain every freight charge. Product weight, dimensions, category, regional location and order characteristics also influence the outcome. This visualisation is particularly important because it connects a theoretically relevant logistics factor with the prediction target and directly supports the inclusion of distance in the multiple linear regression models.

8. Baseline Multiple Linear Regression Model

A baseline multiple linear regression model is developed to predict item-level freight cost. The model combines product characteristics, geographical information, order context, purchase timing and categorical marketplace attributes.

The target variable is `freight_value`, measured in Brazilian reais for each order item. Identifiers, payment values, reviews, delivery outcomes and other post-purchase information are excluded because they either lack predictive meaning or could introduce target leakage.

Items belonging to the same order share customer, purchase-time and order-level information. A group-aware split based on `order_id` is therefore used instead of an ordinary random row split. This ensures that all items from one order remain entirely within either the training set or the test set.

```
In [78]: # =====  
# ADDITIONAL MODELLING IMPORTS  
# =====  
  
from sklearn.model_selection import (  
    GroupShuffleSplit,  
    GroupKFold,  
    cross_validate  
)  
  
from sklearn.dummy import DummyRegressor  
  
from sklearn.metrics import (  
    mean_squared_error,  
    mean_absolute_error,  
    median_absolute_error,  
    r2_score,  
    make_scorer  
)
```

```
print("Advanced modelling tools imported successfully.")
```

Advanced modelling tools imported successfully.

```
In [79]: # =====
# PREPARE FEATURES, TARGET AND GROUP IDENTIFIERS
# =====

X = modelling_df[
    all_candidate_features
].copy()

y = pd.to_numeric(
    modelling_df[target_column],
    errors="coerce"
).astype(float)

groups = modelling_df[
    group_column
].copy()

# Explicitly standardise numerical data types
for column_name in baseline_numerical_features:
    X[column_name] = pd.to_numeric(
        X[column_name],
        errors="coerce"
    ).astype(float)

# Convert categorical variables to ordinary object columns
# and represent missing values using np.nan
for column_name in baseline_categorical_features:
    X[column_name] = (
        X[column_name]
        .astype("object")
        .where(
            X[column_name].notna(),
            np.nan
        )
    )

# Final target checks
assert y.notna().all()
assert np.isfinite(y).all()
assert len(X) == len(y) == len(groups)

print(f"Feature matrix shape: {X.shape}")
print(f"Target observations: {len(y):,}")
print(f"Unique order groups: {groups.nunique():,}")
```

Feature matrix shape: (9000, 24)

Target observations: 9,000

Unique order groups: 8,874

```
In [80]: # =====
# GROUP-AWARE TRAIN-TEST SPLIT
# =====

group_splitter = GroupShuffleSplit(
    n_splits=1,
    test_size=TEST_SIZE,
    random_state=RANDOM_STATE
)

train_indices, test_indices = next(
    group_splitter.split(
        X=X,
        y=y,
        groups=groups
    )
)

X_train = X.iloc[train_indices].copy()
X_test = X.iloc[test_indices].copy()

y_train = y.iloc[train_indices].copy()
y_test = y.iloc[test_indices].copy()

groups_train = groups.iloc[train_indices].copy()
groups_test = groups.iloc[test_indices].copy()

# Confirm that no order appears in both partitions
overlapping_orders = set(
    groups_train.unique()
).intersection(
    set(groups_test.unique())
)

assert len(overlapping_orders) == 0

print(f"Training rows: {len(X_train):,}")
print(f"Testing rows: {len(X_test):,}")
print(
    f"Training orders: "
    f"{groups_train.nunique():,}"
)
print(
    f"Testing orders: "
    f"{groups_test.nunique():,}"
)
print(
    f"Orders appearing in both sets: "
    f"{len(overlapping_orders)}"
)
```

Training rows: 7,199
Testing rows: 1,801
Training orders: 7,099
Testing orders: 1,775
Orders appearing in both sets: 0

```
In [81]: # =====  
# TRAIN-TEST SPLIT AUDIT  
# =====  
  
split_audit_df = pd.DataFrame(  
    {  
        "measurement": [  
            "Rows",  
            "Unique orders",  
            "Mean freight value",  
            "Median freight value",  
            "Freight standard deviation",  
            "Minimum freight value",  
            "Maximum freight value"  
        ],  
        "training_set": [  
            len(X_train),  
            groups_train.nunique(),  
            y_train.mean(),  
            y_train.median(),  
            y_train.std(),  
            y_train.min(),  
            y_train.max()  
        ],  
        "testing_set": [  
            len(X_test),  
            groups_test.nunique(),  
            y_test.mean(),  
            y_test.median(),  
            y_test.std(),  
            y_test.min(),  
            y_test.max()  
        ]  
    }  
)  
  
split_audit_df[  
    "absolute_difference"  
] = (  
    split_audit_df["training_set"]  
    - split_audit_df["testing_set"]  
) .abs()  
  
split_audit_df
```

Out [81]:

	measurement	training_set	testing_set	absolute_difference
0	Rows	7,199.000	1,801.000	5,398.000
1	Unique orders	7,099.000	1,775.000	5,324.000
2	Mean freight value	19.601	20.096	0.495
3	Median freight value	16.220	16.460	0.240
4	Freight standard deviation	14.068	17.367	3.299
5	Minimum freight value	0.000	0.000	0.000
6	Maximum freight value	206.500	306.060	99.560

The training and test sets are compared to confirm that the group-aware split has not produced a severely unrepresentative test partition. Small differences are expected because complete orders, rather than independent rows, were allocated to each partition.

8.1 Leakage-Safe Preprocessing Pipeline

All model-specific preprocessing is fitted using the training data only.

Numerical variables are imputed using their training-set median and then standardised. Median imputation is appropriate because several predictors contain long right tails and extreme values.

Categorical variables receive an explicit missing label and are one-hot encoded. Infrequent categories are grouped where supported by the installed Scikit-learn version. The first category is dropped to reduce exact dummy-variable multicollinearity, while unseen test-set categories are handled without causing prediction errors.

In [82]:

```
# =====
# CREATE A VERSION-COMPATIBLE ONE-HOT ENCODER
# =====

def create_one_hot_encoder(
    minimum_frequency=20
):
    """
    Create a one-hot encoder compatible with recent and older
    Scikit-learn versions.
    """

    try:
        return OneHotEncoder(
            handle_unknown="ignore",
            min_frequency=minimum_frequency,
            drop="first",
            sparse_output=True
        )

    except TypeError:
        try:
            return OneHotEncoder(
                handle_unknown="ignore",
                min_frequency=minimum_frequency,
                drop="first",
```

```
        sparse=True
    )

except TypeError:
    return OneHotEncoder(
        handle_unknown="ignore",
        drop="first",
        sparse=True
    )

numerical_pipeline = Pipeline(
    steps=[
        (
            "median_imputer",
            SimpleImputer(
                strategy="median",
                add_indicator=True
            )
        ),
        (
            "standard_scaler",
            StandardScaler()
        )
    ]
)

categorical_pipeline = Pipeline(
    steps=[
        (
            "missing_category_imputer",
            SimpleImputer(
                strategy="constant",
                fill_value="missing"
            )
        ),
        (
            "one_hot_encoder",
            create_one_hot_encoder(
                minimum_frequency=20
            )
        )
    ]
)

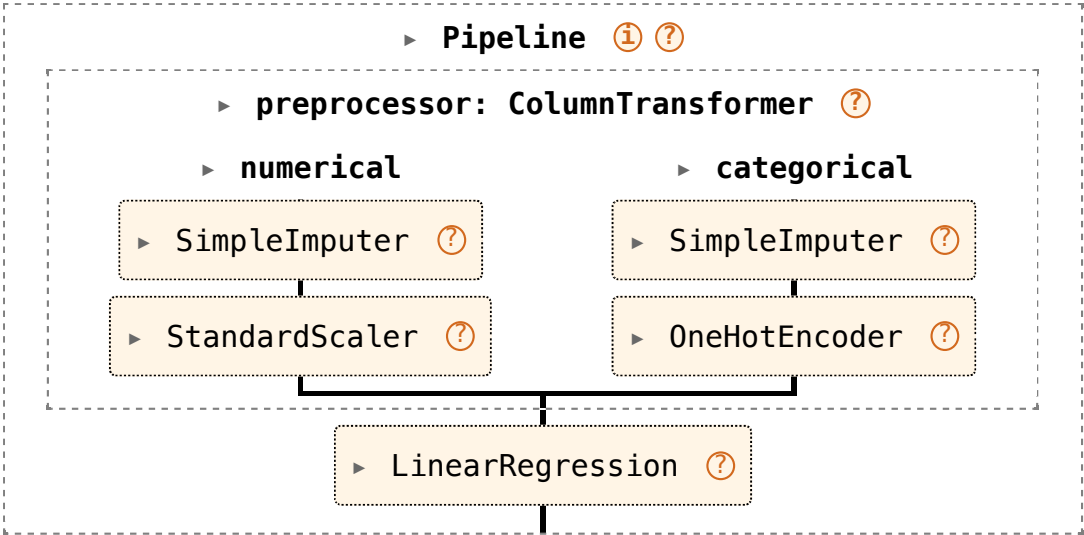
baseline_preprocessor = ColumnTransformer(
    transformers=[
        (
            "numerical",
            numerical_pipeline,
            baseline_numerical_features
        ),
        (
            "categorical",
```

```
        categorical_pipeline,
        baseline_categorical_features
    )
],
remainder="drop",
verbose_feature_names_out=True
)

baseline_regression_pipeline = Pipeline(
    steps=[
        (
            "preprocessor",
            baseline_preprocessor
        ),
        (
            "linear_regression",
            LinearRegression()
        )
    ]
)

baseline_regression_pipeline
```

Out[82]:



In [83]:

```
# =====
# REUSABLE REGRESSION EVALUATION FUNCTIONS
# =====

def calculate_rmse(
    actual_values,
    predicted_values
):
    """
    Calculate root mean squared error.
    """

    return np.sqrt(
        mean_squared_error(
            actual_values,
            predicted_values
        )
    )
```

```
    )
)

def calculate_regression_metrics(
    actual_values,
    predicted_values,
    model_name
):
    """
    Calculate principal regression performance measures.
    """

    rmse_value = calculate_rmse(
        actual_values,
        predicted_values
    )

    mae_value = mean_absolute_error(
        actual_values,
        predicted_values
    )

    median_ae_value = median_absolute_error(
        actual_values,
        predicted_values
    )

    r_squared_value = r2_score(
        actual_values,
        predicted_values
    )

    target_mean = np.mean(actual_values)

    return {
        "model": model_name,
        "RMSE": rmse_value,
        "MAE": mae_value,
        "median_absolute_error":
            median_ae_value,
        "R_squared": r_squared_value,
        "RMSE_as_percentage_of_target_mean": (
            rmse_value
            / target_mean
            * 100
            if target_mean != 0
            else np.nan
        ),
        "predicted_negative_values": int(
            np.sum(
                predicted_values < 0
            )
        )
    }
```

8.2 Naïve Benchmark

A mean-prediction model is used as a minimum benchmark. It predicts the training-set mean freight value for every test observation.

The benchmark has no explanatory features. A useful multiple linear regression model should therefore achieve lower RMSE and MAE and a higher R² than this baseline.

In [84]:

```
# =====
# FIT THE MEAN-PREDICTION BENCHMARK
# =====

dummy_model = DummyRegressor(
    strategy="mean"
)

dummy_training_matrix = np.zeros(
    shape=(len(y_train), 1)
)

dummy_testing_matrix = np.zeros(
    shape=(len(y_test), 1)
)

dummy_model.fit(
    dummy_training_matrix,
    y_train
)

dummy_predictions = dummy_model.predict(
    dummy_testing_matrix
)

dummy_metrics = calculate_regression_metrics(
    actual_values=y_test,
    predicted_values=dummy_predictions,
    model_name="Mean prediction benchmark"
)

pd.DataFrame([dummy_metrics])
```

Out[84]:

	model	RMSE	MAE	median_absolute_error	R_squared	RMSE_as_percentage_of_target_mean	predicted_negative_values
0	Mean prediction benchmark	17.369	8.573	5.331	-0.001	86.431	0

In [85]:

```
# =====
# FIT THE BASELINE MULTIPLE LINEAR REGRESSION MODEL
# =====

baseline_regression_pipeline.fit(
    X_train,
    y_train
)

baseline_predictions = (
    baseline_regression_pipeline.predict(
```

```

        X_test
    )
)

baseline_metrics = calculate_regression_metrics(
    actual_values=y_test,
    predicted_values=baseline_predictions,
    model_name="Baseline multiple linear regression"
)

test_model_comparison_df = pd.DataFrame(
    [
        dummy_metrics,
        baseline_metrics
    ]
)

test_model_comparison_df

```

Out[85]:

	model	RMSE	MAE	median_absolute_error	R_squared	RMSE_as_percentage_of_target_mean	predicted_negative_values
0	Mean prediction benchmark	17.369	8.573	5.331	-0.001	86.431	0
1	Baseline multiple linear regression	11.492	5.166	2.719	0.562	57.185	0

In [86]:

```

# =====
# BASELINE IMPROVEMENT OVER NAÏVE BENCHMARK
# =====

benchmark_rmse = dummy_metrics["RMSE"]
baseline_rmse = baseline_metrics["RMSE"]

benchmark_mae = dummy_metrics["MAE"]
baseline_mae = baseline_metrics["MAE"]

baseline_improvement_df = pd.DataFrame(
    {
        "measurement": [
            "RMSE reduction",
            "RMSE percentage improvement",
            "MAE reduction",
            "MAE percentage improvement",
            "Baseline test R²"
        ],
        "value": [
            benchmark_rmse - baseline_rmse,
            (
                benchmark_rmse
                - baseline_rmse
            )
            / benchmark_rmse
            * 100,
            benchmark_mae - baseline_mae,
            (
                benchmark_mae
                - baseline_mae
            )

```

```
        )
        / benchmark_mae
        * 100,
        baseline_metrics["R_squared"]
    ]
}
)

baseline_improvement_df
```

Out [86]:

	measurement	value
0	RMSE reduction	5.877
1	RMSE percentage improvement	33.838
2	MAE reduction	3.406
3	MAE percentage improvement	39.737
4	Baseline test R²	0.562

```
In [87]: # =====
# TEST-SET PREDICTIONS AND RESIDUALS
# =====

baseline_prediction_results_df = (
    modelling_df
    .iloc[test_indices][
        [
            "order_id",
            "order_item_id",
            "freight_value",
            "price",
            "product_weight_g",
            "distance_km",
            "product_category_name_english",
            "customer_state",
            "seller_state"
        ]
    ]
    .copy()
    .reset_index(drop=True)
)

baseline_prediction_results_df[
    "predicted_freight_value"
] = baseline_predictions

baseline_prediction_results_df[
    "residual"
] = (
    baseline_prediction_results_df[
        "freight_value"
    ]
    - baseline_prediction_results_df[
```

```
        "predicted_freight_value"
    ]
)

baseline_prediction_results_df[
    "absolute_error"
] = (
    baseline_prediction_results_df[
        "residual"
    ].abs()
)

baseline_prediction_results_df[
    "squared_error"
] = (
    baseline_prediction_results_df[
        "residual"
    ] ** 2
)

baseline_prediction_results_df.head(10)
```

Out[87]:

	order_id	order_item_id	freight_value	price	product_weight_g	distance_km	product_category_name_english	customer_state	seller_state	predicted
0	b124967afcc82ef17ec41020fe2a9136	1	8.880	49.990	7,875.000	27.022	furniture_living_room	SP	SP	
1	fd220bb11ef8a046fef08e7c51f11651	1	17.810	79.990	650.000	495.117	computers_accessories	RJ	SP	
2	2b7d922e73fbd058099b6ff4b33e99f7	1	17.760	169.990	433.000	1,293.638	watches_gifts	MT	SP	
3	709df04791b8bd393b34a58de3cf142d	1	19.440	223.000	300.000	822.746	watches_gifts	ES	SP	
4	166b01d0fce75a595b92ad9e84e0843b	1	29.090	98.000	925.000	2,094.074	health_beauty	CE	SP	
5	e1e0cfc8398a7bcd9699fe42c1651566	1	7.780	29.990	350.000	7.658	telephony	SP	SP	
6	ffa9a3142ddd51af248e648db27c647f	1	17.920	29.900	900.000	654.868	furniture_decor	RJ	PR	
7	96b7f5a470ca6a58f261be1af286e4d9	1	29.730	209.990	4,650.000	2,119.849	cool_stuff	PE	SP	
8	bbb39449a277499952671cc64cbd7232	1	34.190	92.500	4,400.000	629.048	construction_tools_construction	MG	SP	
9	587e32dd528769d669d539531d32aeb5	1	22.090	78.000	2,200.000	346.015	furniture_decor	RJ	SP	

9. Group-Aware Cross-Validation

A single train-test split may produce results influenced by one particular partition. Five-fold group cross-validation is therefore performed using the training data.

`GroupKFold` ensures that all order items sharing the same `order_id` remain within the same fold. In each iteration, the complete preprocessing pipeline is refitted, including imputation, scaling, category grouping and one-hot encoding. This prevents validation-fold information from influencing preprocessing.

```
In [88]: # =====
# GROUP-AWARE FIVE-FOLD CROSS-VALIDATION
# =====

def negative_rmse_scorer_function(
```

```

    actual_values,
    predicted_values
):
    """
    Return RMSE for use in a Scikit-learn scorer.
    """

    return calculate_rmse(
        actual_values,
        predicted_values
    )

rmse_scorer = make_scorer(
    negative_rmse_scorer_function,
    greater_is_better=False
)

group_cross_validator = GroupKFold(
    n_splits=NUMBER_OF_FOLDS
)

cross_validation_scoring = {
    "RMSE": rmse_scorer,
    "MAE": "neg_mean_absolute_error",
    "R_squared": "r2"
}

baseline_cv_results = cross_validate(
    estimator=baseline_regression_pipeline,
    X=X_train,
    y=y_train,
    groups=groups_train,
    cv=group_cross_validator,
    scoring=cross_validation_scoring,
    return_train_score=False,
    n_jobs=-1
)

print("Group-aware cross-validation completed successfully.")

```

Group-aware cross-validation completed successfully.

```

In [89]: # =====
# CROSS-VALIDATION FOLD RESULTS
# =====

baseline_cv_fold_results_df = pd.DataFrame(
    {
        "fold": np.arange(
            1,
            NUMBER_OF_FOLDS + 1
        ),
        "RMSE": -baseline_cv_results[
            "test_RMSE"
        ],
    },

```

```
        "MAE": -baseline_cv_results[
            "test_MAE"
        ],
        "R_squared": baseline_cv_results[
            "test_R_squared"
        ],
        "fit_time_seconds": baseline_cv_results[
            "fit_time"
        ],
        "score_time_seconds": baseline_cv_results[
            "score_time"
        ]
    }
)
```

baseline_cv_fold_results_df

Out[89]:

	fold	RMSE	MAE	R_squared	fit_time_seconds	score_time_seconds
0	1	9.513	4.880	0.583	0.046	0.004
1	2	8.379	4.870	0.536	0.046	0.004
2	3	9.779	4.893	0.539	0.040	0.005
3	4	9.272	4.908	0.584	0.038	0.005
4	5	9.460	5.292	0.567	0.044	0.004

In [90]:

```
# =====
# CROSS-VALIDATION SUMMARY
# =====

baseline_cv_summary_df = pd.DataFrame(
    {
        "metric": [
            "RMSE",
            "MAE",
            "R_squared"
        ],
        "mean": [
            baseline_cv_fold_results_df[
                "RMSE"
            ].mean(),
            baseline_cv_fold_results_df[
                "MAE"
            ].mean(),
            baseline_cv_fold_results_df[
                "R_squared"
            ].mean()
        ],
        "standard_deviation": [
            baseline_cv_fold_results_df[
                "RMSE"
            ].std(ddof=1),
            baseline_cv_fold_results_df[
```

```
        "MAE"
    ].std(ddof=1),
    baseline_cv_fold_results_df[
        "R_squared"
    ].std(ddof=1)
],
"minimum": [
    baseline_cv_fold_results_df[
        "RMSE"
    ].min(),
    baseline_cv_fold_results_df[
        "MAE"
    ].min(),
    baseline_cv_fold_results_df[
        "R_squared"
    ].min()
],
"maximum": [
    baseline_cv_fold_results_df[
        "RMSE"
    ].max(),
    baseline_cv_fold_results_df[
        "MAE"
    ].max(),
    baseline_cv_fold_results_df[
        "R_squared"
    ].max()
]
}
)

baseline_cv_summary_df
```

Out[90]:

	metric	mean	standard_deviation	minimum	maximum
0	RMSE	9.280	0.536	8.379	9.779
1	MAE	4.969	0.181	4.870	5.292
2	R_squared	0.562	0.023	0.536	0.584

```
In [91]: # =====
# EXTRACT TRANSFORMED FEATURE NAMES AND COEFFICIENTS
# =====

fitted_preprocessor = (
    baseline_regression_pipeline
    .named_steps["preprocessor"]
)

fitted_linear_model = (
    baseline_regression_pipeline
    .named_steps["linear_regression"]
)

transformed_feature_names = (
```

```
fitted_preprocessor
.get_feature_names_out()
)

baseline_coefficient_df = pd.DataFrame(
    {
        "feature": transformed_feature_names,
        "coefficient": fitted_linear_model.coef_
    }
)

baseline_coefficient_df[
    "absolute_coefficient"
] = (
    baseline_coefficient_df[
        "coefficient"
    ].abs()
)

baseline_coefficient_df = (
    baseline_coefficient_df
    .sort_values(
        "absolute_coefficient",
        ascending=False
    )
    .reset_index(drop=True)
)

print(
    "Number of transformed model features:",
    len(transformed_feature_names)
)

print(
    "Regression intercept:",
    f"{fitted_linear_model.intercept_:.4f}"
)

baseline_coefficient_df.head(20)
```

Number of transformed model features: 99
Regression intercept: 32.9186

Out [91]:

	feature	coefficient	absolute_coefficient
0	categorical__customer_state_BA	-14.608	14.608
1	categorical__customer_state_RS	-12.978	12.978
2	categorical__customer_state_CE	-12.357	12.357
3	categorical__customer_state_SP	-12.326	12.326
4	categorical__customer_state_MG	-12.100	12.100
5	categorical__customer_state_PE	-12.064	12.064
6	categorical__customer_state_SC	-12.057	12.057
7	categorical__customer_state_DF	-11.886	11.886
8	categorical__customer_state_RJ	-11.723	11.723
9	categorical__customer_state_MS	-11.717	11.717
10	categorical__customer_state_PR	-11.477	11.477
11	categorical__customer_state_ES	-11.211	11.211
12	categorical__customer_state_GO	-11.143	11.143
13	categorical__customer_state_infrequent_sklearn	-11.055	11.055
14	categorical__customer_state_RN	-10.177	10.177
15	categorical__customer_state_MT	-9.566	9.566
16	categorical__customer_state_RO	-8.879	8.879
17	categorical__customer_state_PA	-8.368	8.368
18	categorical__customer_state_PB	-7.352	7.352
19	numerical__product_weight_g	6.314	6.314

The coefficient ranking should be interpreted cautiously. Numerical predictors were standardised, but categorical coefficients represent differences from omitted reference categories. Furthermore, correlated predictors can redistribute coefficient magnitude between related variables.

Consequently, coefficient size is used as supporting evidence rather than as a definitive causal importance measure.

In [92]:

```
# =====
# SAVE BASELINE MODEL RESULTS
# =====

split_audit_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "group_train_test_split_audit.csv",
    index=False
)

test_model_comparison_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "baseline_test_model_comparison.csv",
```

```
        index=False
    )

    baseline_improvement_df.to_csv(
        PROCESSED_DATA_DIRECTORY
        / "baseline_improvement_over_benchmark.csv",
        index=False
    )

    baseline_prediction_results_df.to_csv(
        PROCESSED_DATA_DIRECTORY
        / "baseline_test_predictions.csv",
        index=False
    )

    baseline_cv_fold_results_df.to_csv(
        PROCESSED_DATA_DIRECTORY
        / "baseline_group_cv_fold_results.csv",
        index=False
    )

    baseline_cv_summary_df.to_csv(
        PROCESSED_DATA_DIRECTORY
        / "baseline_group_cv_summary.csv",
        index=False
    )

    baseline_coefficient_df.to_csv(
        PROCESSED_DATA_DIRECTORY
        / "baseline_regression_coefficients.csv",
        index=False
    )

    print("Baseline modelling outputs saved successfully.")
```

Baseline modelling outputs saved successfully.

Baseline Model Interpretation

The baseline multiple linear regression achieved a test RMSE of 11.492 BRL, an MAE of 5.166 BRL and an R^2 of 0.562. These results show that the selected product, seller, customer and geographical variables contain meaningful information about freight cost and provide substantially greater predictive value than simply assigning the same average freight value to every order item.

The MAE indicates that the baseline prediction differed from the actual freight charge by approximately 5.17 BRL on average. From a business perspective, this level of error may be acceptable for preliminary budgeting or identifying broadly expensive deliveries, but it may still be too large for producing an exact customer quotation. The higher RMSE of 11.49 BRL shows that some observations produced much larger errors than the typical prediction. This is important because occasional large underestimates could cause an e-commerce business to charge customers too little for delivery, while large overestimates could make an order appear unnecessarily expensive.

The R^2 value of 0.562 means that the baseline model explained approximately 56.2% of the variation in freight costs for unseen orders. Therefore, more than 40% of the variation remained unexplained. This suggests that a simple additive relationship is not sufficient to represent all freight-pricing behaviour, particularly for unusually heavy, bulky or long-distance deliveries.

The group-aware test split makes these results more realistic because products belonging to the same order were not divided between training and testing data. The baseline therefore measures performance on genuinely unseen groups of orders rather than allowing related order information to appear in both datasets.

Overall, the baseline is useful as an interpretable reference model, but its remaining error indicates a need for non-linear transformations, parcel-volume measures and interactions between physical parcel characteristics and delivery distance.

10. Advanced Regression Diagnostics

Predictive metrics alone do not establish whether a multiple linear regression model behaves appropriately. The residuals must also be examined for systematic patterns, unequal variance, non-normality and influential observations.

This section evaluates:

- prediction calibration;
- residual location and distribution;
- heteroscedasticity;
- multicollinearity;
- leverage and Cook's distance;
- error variation across predicted freight-cost levels.

The diagnostic results will guide the feature-engineering and regularisation stages. Potentially influential observations are flagged for investigation rather than automatically removed because unusually large products, long-distance deliveries and high freight charges may represent valid commercial transactions.

In [93]: `%pip install -U statsmodels`

```
Requirement already satisfied: statsmodels in /opt/anaconda3/envs/dlwp_tf/lib/python3.11/site-packages (0.14.6)
Requirement already satisfied: numpy<3,>=1.22.3 in /opt/anaconda3/envs/dlwp_tf/lib/python3.11/site-packages (from statsmodels) (1.26.4)
Requirement already satisfied: scipy!=1.9.2,>=1.8 in /opt/anaconda3/envs/dlwp_tf/lib/python3.11/site-packages (from statsmodels) (1.17.1)
Requirement already satisfied: pandas!=2.1.0,>=1.4 in /opt/anaconda3/envs/dlwp_tf/lib/python3.11/site-packages (from statsmodels) (3.0.1)
Requirement already satisfied: patsy>=0.5.6 in /opt/anaconda3/envs/dlwp_tf/lib/python3.11/site-packages (from statsmodels) (1.0.2)
Requirement already satisfied: packaging>=21.3 in /opt/anaconda3/envs/dlwp_tf/lib/python3.11/site-packages (from statsmodels) (25.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /opt/anaconda3/envs/dlwp_tf/lib/python3.11/site-packages (from pandas!=2.1.0,>=1.4->statsmodels) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /opt/anaconda3/envs/dlwp_tf/lib/python3.11/site-packages (from python-dateutil>=2.8.2->pandas!=2.1.0,>=1.4->statsmodels) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
```

In [94]:

```
# =====
# ADVANCED REGRESSION DIAGNOSTIC IMPORTS
# =====

try:
    import statsmodels.api as sm

    from statsmodels.nonparametric.smoothers_lowess import lowess

    from statsmodels.stats.diagnostic import (
        het_breuschpagan
    )

    from statsmodels.stats.outliers_influence import (
        variance_inflation_factor
    )
```

```

except ImportError as error:
    raise ImportError(
        "The statsmodels package is required for the "
        "advanced diagnostic section."
    ) from error

print("Advanced regression diagnostic tools imported successfully.")

```

Advanced regression diagnostic tools imported successfully.

```

In [95]: # =====
# TRAINING AND TEST RESIDUAL CALCULATION
# =====

baseline_train_predictions = (
    baseline_regression_pipeline.predict(
        X_train
    )
)

baseline_test_predictions = (
    baseline_predictions.copy()
)

training_residuals = (
    y_train.to_numpy()
    - baseline_train_predictions
)

testing_residuals = (
    y_test.to_numpy()
    - baseline_test_predictions
)

training_residual_standard_deviation = np.std(
    training_residuals,
    ddof=1
)

standardised_testing_residuals = (
    testing_residuals
    / training_residual_standard_deviation
)

print(
    "Training residual observations:",
    f"{len(training_residuals):,}"
)

print(
    "Testing residual observations:",
    f"{len(testing_residuals):,}"
)

print(

```

```

    "Training residual standard deviation:",
    f"{training_residual_standard_deviation:.4f}"
)

```

Training residual observations: 7,199

Testing residual observations: 1,801

Training residual standard deviation: 8.9761

```

In [96]: # =====
# RESIDUAL STATISTICAL SUMMARY
# =====

def create_residual_summary(
    residual_values,
    dataset_name
):
    """
    Calculate diagnostic statistics for a set of residuals.
    """

    residual_series = pd.Series(
        residual_values
    ).dropna()

    return {
        "dataset": dataset_name,
        "count": len(residual_series),
        "mean_residual": residual_series.mean(),
        "median_residual": residual_series.median(),
        "standard_deviation": residual_series.std(),
        "minimum": residual_series.min(),
        "first_quartile": residual_series.quantile(0.25),
        "third_quartile": residual_series.quantile(0.75),
        "maximum": residual_series.max(),
        "skewness": residual_series.skew(),
        "excess_kurtosis": residual_series.kurt(),
        "mean_absolute_residual": (
            residual_series.abs().mean()
        ),
        "percentage_within_5_brl": (
            residual_series.abs()
            .le(5)
            .mean()
            * 100
        ),
        "percentage_within_10_brl": (
            residual_series.abs()
            .le(10)
            .mean()
            * 100
        )
    }

residual_summary_df = pd.DataFrame(
    [

```

```
        create_residual_summary(
            training_residuals,
            "Training set"
        ),
        create_residual_summary(
            testing_residuals,
            "Testing set"
        )
    ]
)

residual_summary_df
```

Out[96]:

	dataset	count	mean_residual	median_residual	standard_deviation	minimum	first_quartile	third_quartile	maximum	skewness	excess_kurtosis	mean_absolute_residual	percenta
0	Training set	7199	-0.000	-0.154	8.976	-57.802	-3.142	2.204	141.466	3.541	37.500	4.836	
1	Testing set	1801	0.145	-0.183	11.494	-40.698	-3.138	2.189	229.903	8.228	129.884	5.166	

10.1 Prediction Calibration

An actual-versus-predicted plot evaluates whether predictions follow the ideal 45-degree line. Observations above the line represent underprediction, while observations below the line represent overprediction.

Systematic flattening at high freight values would suggest that the linear model struggles to represent expensive shipments.

```
In [97]: # =====
# ACTUAL VERSUS PREDICTED FREIGHT
# =====

minimum_plot_value = min(
    y_test.min(),
    baseline_test_predictions.min()
)

maximum_plot_value = max(
    y_test.max(),
    baseline_test_predictions.max()
)

figure, axis = plt.subplots(
    figsize=(9, 7)
)

axis.scatter(
    baseline_test_predictions,
    y_test,
    alpha=0.45,
    s=22
)

axis.plot(
```

```
[
    minimum_plot_value,
    maximum_plot_value
],
[
    minimum_plot_value,
    maximum_plot_value
],
linestyle="--",
linewidth=2,
label="Ideal prediction line"
)

axis.set_title(
    "Actual Versus Predicted Freight Cost"
)

axis.set_xlabel(
    "Predicted freight value (BRL)"
)

axis.set_ylabel(
    "Actual freight value (BRL)"
)

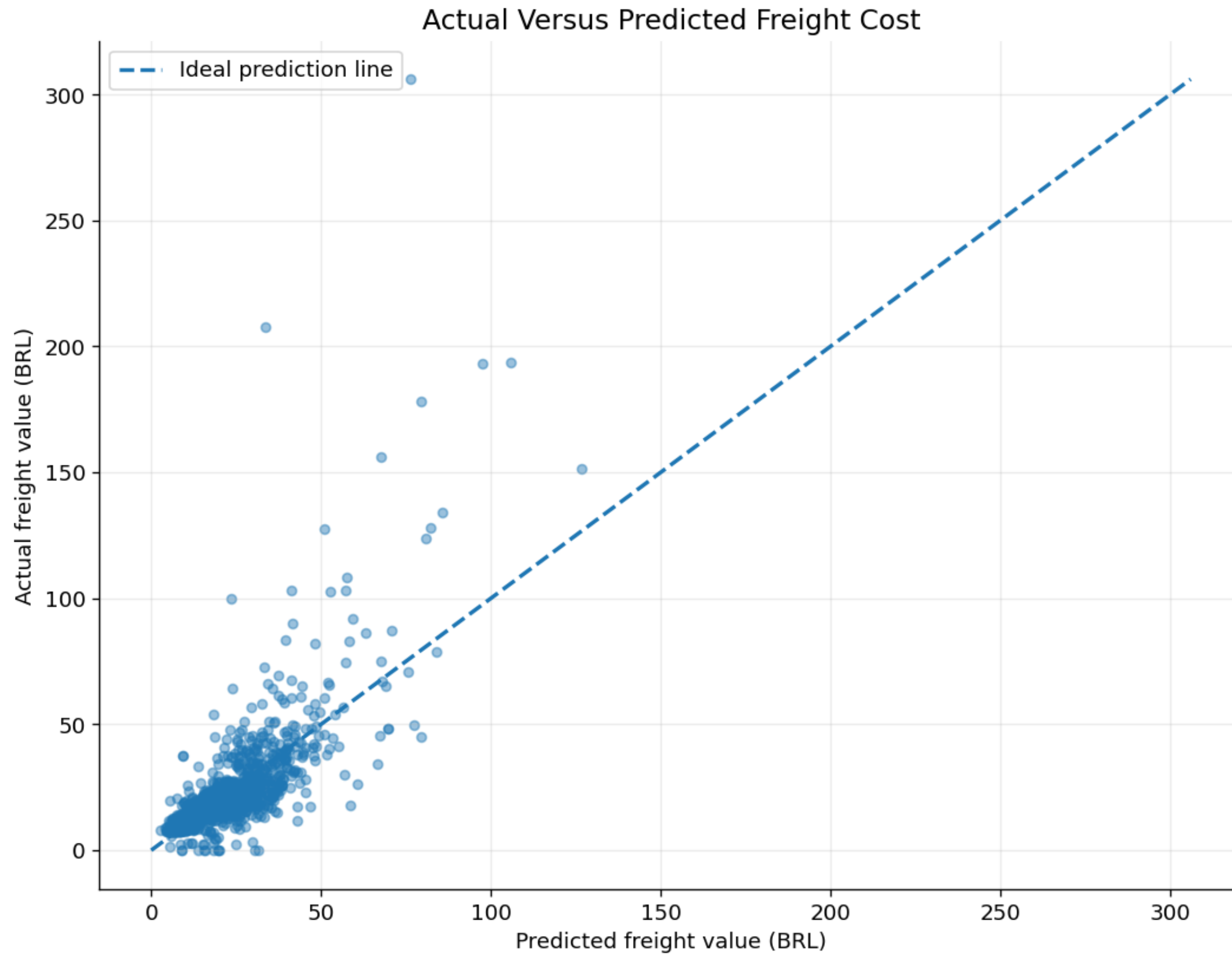
axis.legend()

plt.tight_layout()

save_current_figure(
    "12_actual_vs_predicted_freight.png"
)

plt.show()
```

Figure saved to: /Users/zimmeljavedvirk/Desktop/datasciencemid/figures/12_actual_vs_predicted_freight.png



```
In [98]: # =====  
# RESIDUALS VERSUS FITTED VALUES  
# =====  
  
residual_lowess_line = lowess(  
    endog=testing_residuals,
```

```
    exog=baseline_test_predictions,
    frac=0.30,
    return_sorted=True
)

figure, axis = plt.subplots(
    figsize=(10, 7)
)

axis.scatter(
    baseline_test_predictions,
    testing_residuals,
    alpha=0.45,
    s=22
)

axis.plot(
    residual_lowess_line[:, 0],
    residual_lowess_line[:, 1],
    linewidth=2,
    label="LOWESS residual trend"
)

axis.axhline(
    y=0,
    linestyle="--",
    linewidth=1.5,
    label="Zero residual"
)

axis.set_title(
    "Baseline Model Residuals Versus Fitted Values"
)

axis.set_xlabel(
    "Predicted freight value (BRL)"
)

axis.set_ylabel(
    "Residual: actual - predicted (BRL)"
)

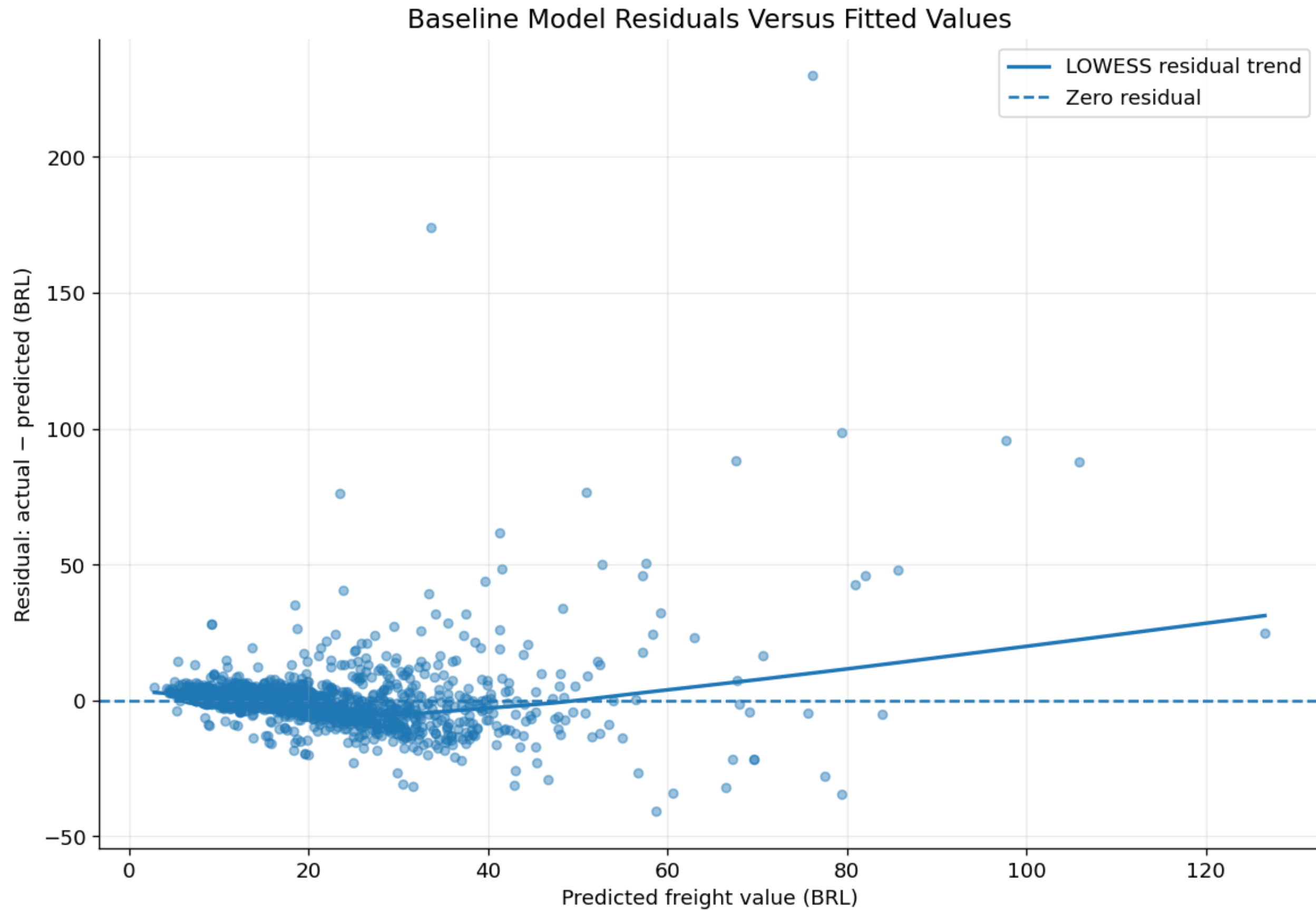
axis.legend()

plt.tight_layout()

save_current_figure(
    "13_residuals_vs_fitted.png"
)

plt.show()
```

Figure saved to: /Users/zimmeljavedvirk/Desktop/datasciencemid/figures/13_residuals_vs_fitted.png



```
In [99]: # =====  
# RESIDUAL DISTRIBUTION AND Q-Q PLOT  
# =====  
  
figure, axes = plt.subplots(  
    nrows=1,
```

```
    ncols=2,
    figsize=(14, 5)
)

axes[0].hist(
    testing_residuals,
    bins=50,
    edgecolor="black",
    alpha=0.80
)

axes[0].axvline(
    x=0,
    linestyle="--",
    linewidth=2
)

axes[0].set_title(
    "Distribution of Test Residuals"
)

axes[0].set_xlabel(
    "Residual (BRL)"
)

axes[0].set_ylabel(
    "Number of observations"
)

stats.probplot(
    testing_residuals,
    dist="norm",
    plot=axes[1]
)

axes[1].set_title(
    "Q-Q Plot of Test Residuals"
)

figure.suptitle(
    "Baseline Regression Residual Distribution",
    fontsize=16
)

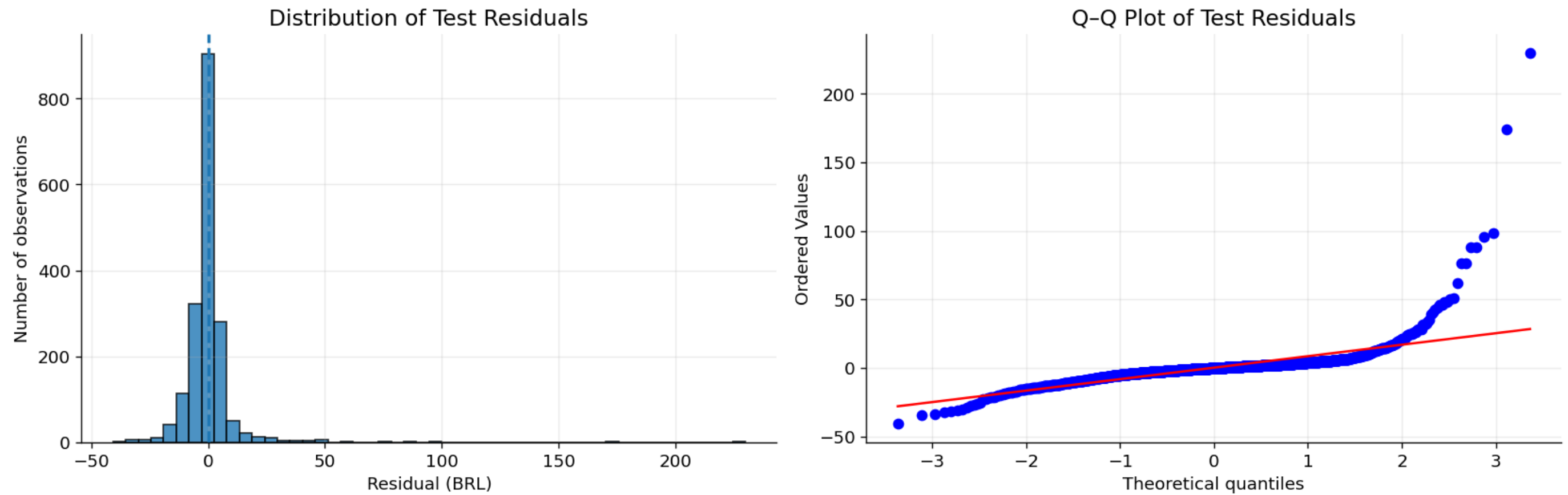
plt.tight_layout()

save_current_figure(
    "14_residual_distribution_qq.png"
)

plt.show()
```

Figure saved to: /Users/zimmeljavedvirk/Desktop/datasciencemid/figures/14_residual_distribution_qq.png

Baseline Regression Residual Distribution



In [100...

```
# =====
# SCALE-LOCATION DIAGNOSTIC
# =====

scale_location_values = np.sqrt(
    np.abs(
        standardised_testing_residuals
    )
)

scale_location_lowess = lowess(
    endog=scale_location_values,
    exog=baseline_test_predictions,
    frac=0.30,
    return_sorted=True
)

figure, axis = plt.subplots(
    figsize=(10, 7)
)

axis.scatter(
    baseline_test_predictions,
    scale_location_values,
    alpha=0.45,
    s=22
)
```

```
axis.plot(
    scale_location_lowess[:, 0],
    scale_location_lowess[:, 1],
    linewidth=2,
    label="LOWESS variance trend"
)

axis.set_title(
    "Scale-Location Plot"
)

axis.set_xlabel(
    "Predicted freight value (BRL)"
)

axis.set_ylabel(
    "Square root of absolute standardised residual"
)

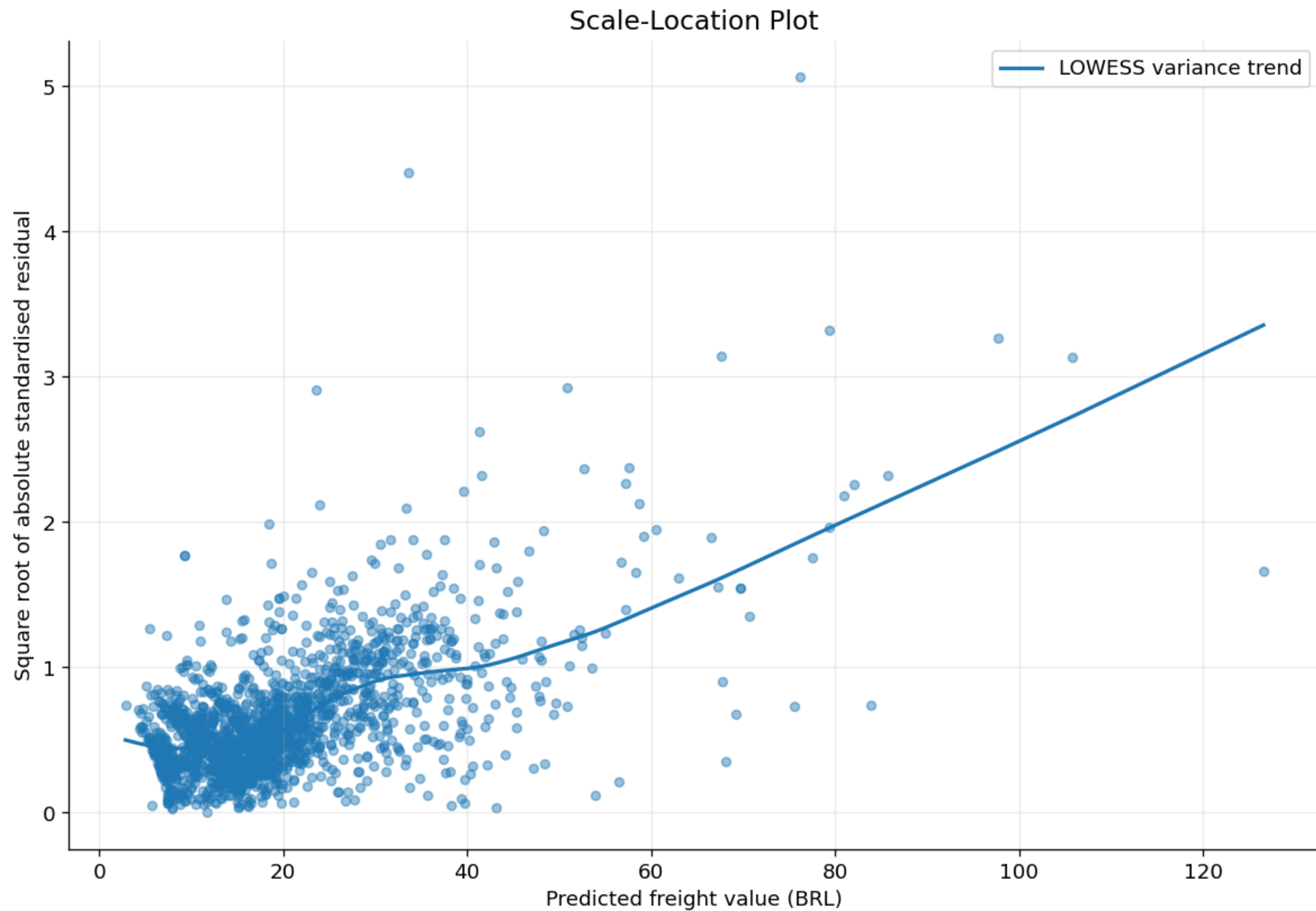
axis.legend()

plt.tight_layout()

save_current_figure(
    "15_scale_location_plot.png"
)

plt.show()
```

Figure saved to: /Users/zimmeljavedvirk/Desktop/datasciencemid/figures/15_scale_location_plot.png



10.2 Heteroscedasticity Testing

The Breusch–Pagan test evaluates whether residual variance is associated with the explanatory variables.

The null hypothesis states that the residual variance is constant. A p-value below 0.05 provides evidence against constant variance. Because the sample is large, the test is considered together

with the residual and scale-location plots.

```
In [101... # =====
# BREUSCH-PAGAN HETEROSCEDASTICITY TEST
# =====

breusch_pagan_imputer = SimpleImputer(
    strategy="median"
)

breusch_pagan_numerical_matrix = (
    breusch_pagan_imputer.fit_transform(
        X_train[
            baseline_numerical_features
        ]
    )
)

breusch_pagan_design_matrix = sm.add_constant(
    breusch_pagan_numerical_matrix,
    has_constant="add"
)

(
    bp_lagrange_multiplier,
    bp_lm_p_value,
    bp_f_statistic,
    bp_f_p_value
) = het_breuschpagan(
    training_residuals,
    breusch_pagan_design_matrix
)

breusch_pagan_results_df = pd.DataFrame(
    {
        "measurement": [
            "Lagrange multiplier statistic",
            "Lagrange multiplier p-value",
            "F statistic",
            "F-test p-value",
            "Significance level",
            "Diagnostic conclusion"
        ],
        "value": [
            bp_lagrange_multiplier,
            bp_lm_p_value,
            bp_f_statistic,
            bp_f_p_value,
            0.05,
            (
                "Evidence of heteroscedasticity"
                if bp_f_p_value < 0.05
                else
                "Insufficient evidence of heteroscedasticity"
            )
        ]
    }
)
```

```
    ]
  }
)

breusch_pagan_results_df
```

Out [101...

	measurement	value
0	Lagrange multiplier statistic	709.288
1	Lagrange multiplier p-value	0.000
2	F statistic	41.296
3	F-test p-value	0.000
4	Significance level	0.050
5	Diagnostic conclusion	Evidence of heteroscedasticity

In [102...

```
# =====
# ERROR ANALYSIS BY PREDICTED FREIGHT DECILE
# =====

prediction_error_analysis_df = pd.DataFrame(
    {
        "actual_freight": y_test.to_numpy(),
        "predicted_freight":
            baseline_test_predictions,
        "residual": testing_residuals
    }
)

prediction_error_analysis_df[
    "absolute_error"
] = (
    prediction_error_analysis_df[
        "residual"
    ].abs()
)

prediction_error_analysis_df[
    "squared_error"
] = (
    prediction_error_analysis_df[
        "residual"
    ] ** 2
)

prediction_error_analysis_df[
    "prediction_decile"
] = pd.qcut(
    prediction_error_analysis_df[
        "predicted_freight"
    ],
    q=10,
    duplicates="drop"
```

```
)

prediction_decile_records = []

for (
    prediction_decile,
    decile_dataframe
) in prediction_error_analysis_df.groupby(
    "prediction_decile",
    observed=True
):

    prediction_decile_records.append(
        {
            "prediction_decile":
                str(prediction_decile),
            "observation_count":
                len(decile_dataframe),
            "mean_predicted_freight":
                decile_dataframe[
                    "predicted_freight"
                ].mean(),
            "mean_actual_freight":
                decile_dataframe[
                    "actual_freight"
                ].mean(),
            "mean_residual":
                decile_dataframe[
                    "residual"
                ].mean(),
            "MAE":
                decile_dataframe[
                    "absolute_error"
                ].mean(),
            "RMSE":
                np.sqrt(
                    decile_dataframe[
                        "squared_error"
                    ].mean()
                ),
            "residual_standard_deviation":
                decile_dataframe[
                    "residual"
                ].std()
        }
    )

prediction_decile_diagnostics_df = pd.DataFrame(
    prediction_decile_records
)

prediction_decile_diagnostics_df
```

Out [102...

	prediction_decile	observation_count	mean_predicted_freight	mean_actual_freight	mean_residual	MAE	RMSE	residual_standard_deviation
0	(2.857, 8.368]	181	6.872	8.988	2.116	2.209	3.026	2.170
1	(8.368, 11.191]	180	9.746	11.612	1.865	2.859	4.607	4.224
2	(11.191, 13.592]	180	12.492	13.579	1.087	2.275	3.014	2.819
3	(13.592, 15.641]	180	14.672	15.418	0.746	2.260	3.579	3.510
4	(15.641, 17.48]	180	16.517	16.260	-0.257	2.205	3.090	3.088
5	(17.48, 19.402]	180	18.489	17.529	-0.960	3.377	5.310	5.237
6	(19.402, 22.056]	180	20.625	18.824	-1.801	3.967	5.541	5.255
7	(22.056, 26.749]	180	24.263	22.639	-1.623	6.800	10.034	9.930
8	(26.749, 33.879]	180	29.915	27.121	-2.794	9.228	16.593	16.402
9	(33.879, 126.572]	180	45.987	49.049	3.062	16.498	28.715	28.630

In [103...

```
# =====
# RMSE BY PREDICTED FREIGHT DECILE
# =====

figure, axis = plt.subplots(
    figsize=(11, 6)
)

axis.plot(
    range(
        1,
        len(
            prediction_decile_diagnostics_df
        ) + 1
    ),
    prediction_decile_diagnostics_df[
        "RMSE"
    ],
    marker="o",
    linewidth=2
)

axis.set_title(
    "Prediction Error Across Freight-Cost Deciles"
)

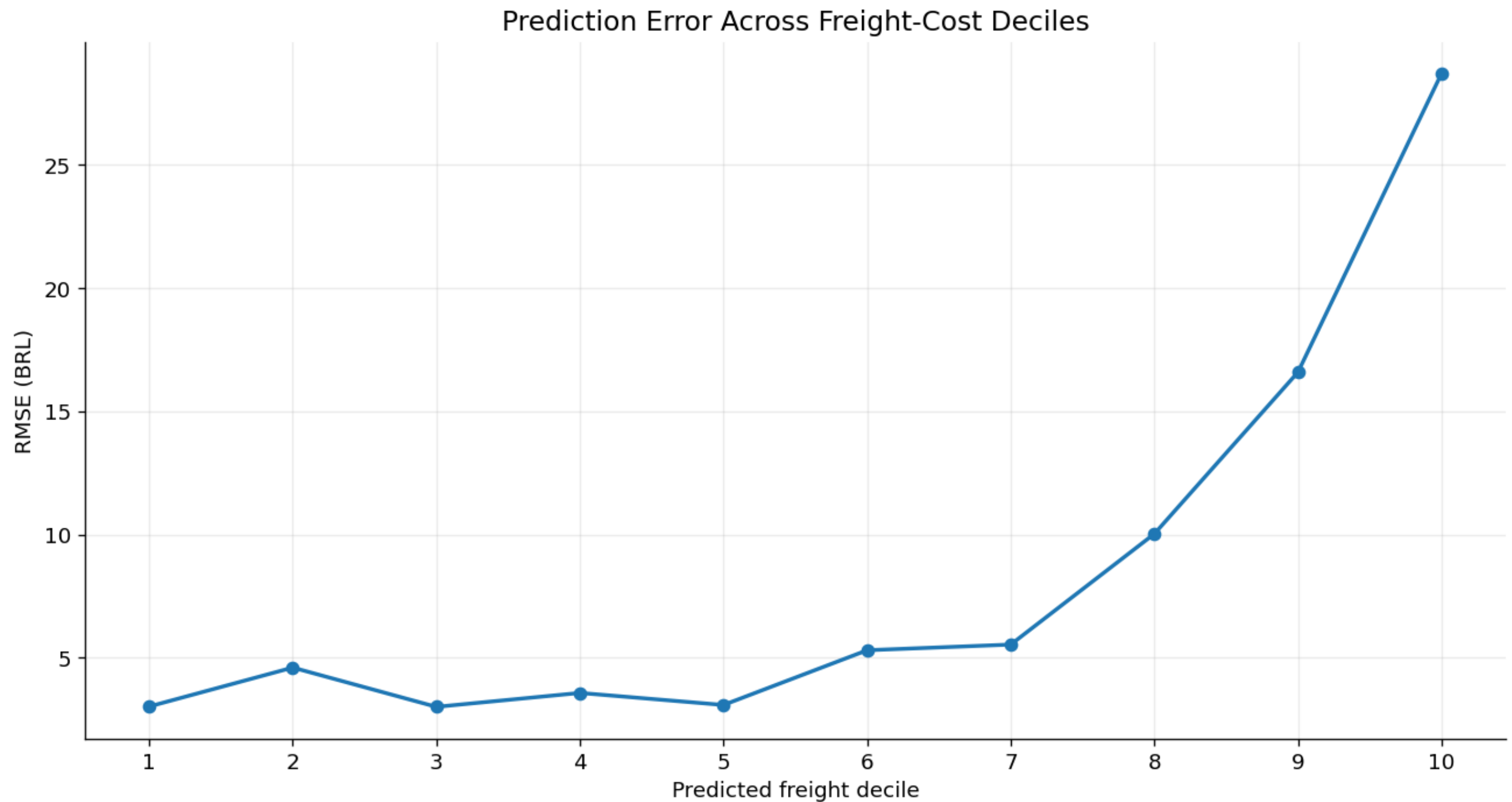
axis.set_xlabel(
    "Predicted freight decile"
)

axis.set_ylabel(
    "RMSE (BRL)"
)

axis.set_xticks(
    range(
```

```
        1,  
        len(  
            prediction_decile_diagnostics_df  
        ) + 1  
    )  
)  
  
plt.tight_layout()  
  
save_current_figure(  
    "16_rmse_by_prediction_decile.png"  
)  
  
plt.show()
```

Figure saved to: /Users/zimmeljavedvirk/Desktop/datasciencemid/figures/16_rmse_by_prediction_decile.png



10.3 Multicollinearity Analysis

Variance inflation factors measure how strongly each numerical predictor can be explained by the other numerical predictors.

Common interpretation thresholds are:

- VIF below 5: limited concern;
- VIF between 5 and 10: potentially problematic;
- VIF above 10: strong multicollinearity.

High VIF does not necessarily reduce predictive accuracy, but it can make ordinary least-squares coefficients unstable and difficult to interpret.

In [104...

```
# =====  
# NUMERICAL VARIANCE INFLATION FACTORS  
# =====  
  
vif_imputer = SimpleImputer(  
    strategy="median"  
)  
  
vif_numerical_matrix = (  
    vif_imputer.fit_transform(  
        X_train[  
            baseline_numerical_features  
        ]  
    )  
)  
  
vif_numerical_df = pd.DataFrame(  
    vif_numerical_matrix,  
    columns=baseline_numerical_features  
)  
  
# Exclude any constant numerical variables  
non_constant_vif_columns = [  
    column_name  
    for column_name in vif_numerical_df.columns  
    if vif_numerical_df[  
        column_name  
    ].nunique() > 1  
]  
  
vif_numerical_df = vif_numerical_df[  
    non_constant_vif_columns  
]  
  
vif_design_matrix = sm.add_constant(  
    vif_numerical_df,  
    has_constant="add"  
)  
  
vif_records = []  
  
for column_position, column_name in enumerate(  
    vif_design_matrix.columns  
):  
  
    if column_name == "const":  
        continue  
  
    vif_value = variance_inflation_factor(  
        vif_design_matrix.to_numpy(),  
        column_position  
    )  
  
    if vif_value >= 10:
```

```
        vif_interpretation = (
            "Strong multicollinearity"
        )

    elif vif_value >= 5:
        vif_interpretation = (
            "Moderate multicollinearity"
        )

    else:
        vif_interpretation = (
            "Limited multicollinearity"
        )

    vif_records.append(
        {
            "predictor": column_name,
            "VIF": vif_value,
            "interpretation":
                vif_interpretation
        }
    )

variance_inflation_factor_df = (
    pd.DataFrame(vif_records)
    .sort_values(
        "VIF",
        ascending=False
    )
    .reset_index(drop=True)
)

variance_inflation_factor_df
```

Out [104...

	predictor	VIF	interpretation
0	price	41.123	Strong multicollinearity
1	order_average_product_price	41.051	Strong multicollinearity
2	order_total_product_value	3.196	Limited multicollinearity
3	purchase_is_weekend	2.486	Limited multicollinearity
4	purchase_day_of_week	2.480	Limited multicollinearity
5	product_weight_g	2.052	Limited multicollinearity
6	order_distinct_product_count	1.880	Limited multicollinearity
7	product_width_cm	1.607	Limited multicollinearity
8	order_distinct_seller_count	1.603	Limited multicollinearity
9	order_item_count	1.582	Limited multicollinearity
10	product_height_cm	1.547	Limited multicollinearity
11	product_length_cm	1.517	Limited multicollinearity
12	distance_km	1.497	Limited multicollinearity
13	same_state	1.489	Limited multicollinearity
14	product_description_length	1.094	Limited multicollinearity
15	product_photos_qty	1.050	Limited multicollinearity
16	product_name_length	1.045	Limited multicollinearity
17	purchase_hour	1.006	Limited multicollinearity
18	purchase_month	1.003	Limited multicollinearity

In [105...

```
# =====  
# STANDARDISED NUMERICAL CONDITION NUMBER  
# =====  
  
condition_scaler = StandardScaler()  
  
standardised_numerical_matrix = (  
    condition_scaler.fit_transform(  
        vif_numerical_df  
    )  
)  
  
numerical_condition_number = np.linalg.cond(  
    standardised_numerical_matrix  
)  
  
condition_number_df = pd.DataFrame(  
    {  
        "measurement": [  
            "Standardised numerical condition number",
```

```
        "Diagnostic interpretation"
    ],
    "value": [
        numerical_condition_number,
        (
            "Potentially severe numerical dependence"
            if numerical_condition_number > 30
            else
            "No severe numerical dependence detected"
        )
    ]
}
)

condition_number_df
```

Out [105...

	measurement	value
0	Standardised numerical condition number	16.045
1	Diagnostic interpretation	No severe numerical dependence detected

10.4 Leverage and Influence

Leverage measures how unusual an observation's predictor values are relative to the training data. Cook's distance combines leverage and residual size to estimate how strongly an observation may influence the fitted regression.

The conventional Cook's-distance screening threshold is $4 / n$. The leverage screening threshold used here is $2p / n$, where p includes the intercept and transformed features. These thresholds identify observations for investigation and are not automatic deletion rules.

In [106...

```
# =====
# STATSMODELS OLS REFIT FOR INFLUENCE DIAGNOSTICS
# =====

transformed_training_matrix = (
    fitted_preprocessor.transform(
        X_train
    )
)

if hasattr(
    transformed_training_matrix,
    "toarray"
):
    transformed_training_dense = (
        transformed_training_matrix.toarray()
    )
else:
    transformed_training_dense = np.asarray(
        transformed_training_matrix
    )

statsmodels_training_design = sm.add_constant(
```

```

    transformed_training_dense,
    has_constant="add"
)

statsmodels_ols_model = sm.OLS(
    y_train.to_numpy(),
    statsmodels_training_design
).fit()

statsmodels_training_predictions = (
    statsmodels_ols_model.fittedvalues
)

maximum_prediction_difference = np.max(
    np.abs(
        statsmodels_training_predictions
        - baseline_train_predictions
    )
)

print(
    "Maximum prediction difference between "
    "Scikit-learn and Statsmodels:",
    f"{maximum_prediction_difference:.10f}"
)

```

Maximum prediction difference between Scikit-learn and Statsmodels: 0.0052146054

In [107...

```

# =====
# COOK'S DISTANCE AND LEVERAGE
# =====

ols_influence = (
    statsmodels_ols_model.get_influence()
)

cooks_distance_values = (
    ols_influence.cooks_distance[0]
)

leverage_values = (
    ols_influence.hat_matrix_diag
)

studentised_residual_values = (
    ols_influence.resid_studentized_internal
)

number_of_training_observations = len(
    y_train
)

number_of_ols_parameters = (
    statsmodels_training_design.shape[1]
)

```

```
cooks_distance_threshold = (  
    4  
    / number_of_training_observations  
)  
  
leverage_threshold = (  
    2  
    * number_of_ols_parameters  
    / number_of_training_observations  
)  
  
training_influence_df = (  
    modelling_df  
    .iloc[train_indices][  
        [  
            "order_id",  
            "order_item_id",  
            "freight_value",  
            "price",  
            "product_weight_g",  
            "distance_km",  
            "product_category_name_english",  
            "customer_state",  
            "seller_state"  
        ]  
    ]  
    .copy()  
    .reset_index(drop=True)  
)  
  
training_influence_df[  
    "predicted_freight_value"  
] = baseline_train_predictions  
  
training_influence_df[  
    "residual"  
] = training_residuals  
  
training_influence_df[  
    "studentised_residual"  
] = studentised_residual_values  
  
training_influence_df[  
    "leverage"  
] = leverage_values  
  
training_influence_df[  
    "cooks_distance"  
] = cooks_distance_values  
  
training_influence_df[  
    "high_cooks_distance"  
] = (  
    training_influence_df[
```

```
        "cooks_distance"
    ]
    > cooks_distance_threshold
)

training_influence_df[
    "high_leverage"
] = (
    training_influence_df[
        "leverage"
    ]
    > leverage_threshold
)

training_influence_df[
    "large_studentised_residual"
] = (
    training_influence_df[
        "studentised_residual"
    ].abs()
    > 3
)

influence_summary_df = pd.DataFrame(
    {
        "measurement": [
            "Training observations",
            "Number of OLS parameters",
            "Cook's distance threshold",
            "High Cook's distance observations",
            "Leverage threshold",
            "High-leverage observations",
            "Absolute studentised residual above 3",
            "Maximum Cook's distance",
            "Maximum leverage"
        ],
        "value": [
            number_of_training_observations,
            number_of_ols_parameters,
            cooks_distance_threshold,
            training_influence_df[
                "high_cooks_distance"
            ].sum(),
            leverage_threshold,
            training_influence_df[
                "high_leverage"
            ].sum(),
            training_influence_df[
                "large_studentised_residual"
            ].sum(),
            training_influence_df[
                "cooks_distance"
            ].max(),
            training_influence_df[
                "leverage"
            ].sum()
        ]
    })
```

```
    ].max()
  }
)

influence_summary_df
```

Out [107...

	measurement	value
0	Training observations	7,199.000
1	Number of OLS parameters	100.000
2	Cook's distance threshold	0.001
3	High Cook's distance observations	411.000
4	Leverage threshold	0.028
5	High-leverage observations	796.000
6	Absolute studentised residual above 3	141.000
7	Maximum Cook's distance	0.644
8	Maximum leverage	1.000

In [108...

```
most_influential_observations_df = (
    training_influence_df
    .sort_values(
        "cooks_distance",
        ascending=False
    )
    .head(20)
)

most_influential_observations_df
```

Out [108...

	order_id	order_item_id	freight_value	price	product_weight_g	distance_km	product_category_name_english	customer_state	seller_state	pre
5364	4412d97cb2093633afa85f11db46316c	1	12.000	2,499.750	9,600.000	1,848.877	computers_accessories	MA	GO	
5447	265b23c0727e5d7a2a3e36f5aac03d67	1	171.320	84.900	1,000.000	2,290.685	books_general_interest	CE	SP	
6162	1429d024a5b24651ee03fd7216d95247	1	206.500	169.900	13,000.000	2,230.349	furniture_decor	PA	MG	
6760	0eeae75286b79b40f203057933fbb70b	2	1.120	34.000	100.000	2,336.984	stationery	RN	SP	
1503	7a53fed7f9006ef9325efa365e6a1992	1	146.300	299.900	9,850.000	1,087.840	pet_shop	MT	SP	
5066	43b496a7732ace398ea4eb3bd520ab22	1	171.500	498.800	21,400.000	2,311.914	home_appliances	MA	SP	
3552	cbaa06517efa6c48a336a917c53e2561	1	170.510	1,890.000	18,000.000	824.866	musical_instruments	MG	PR	
3733	545f259d01f0a271186e324228070de7	1	26.840	99.900	30,000.000	2,359.796	pet_shop	CE	SP	
1047	f3dcf0d2824efde907e341ff4997f693	1	138.230	297.000	16,100.000	1,280.464	cool_stuff	MS	SC	
1046	fd3c2f37b535d300e61e63b029edfd20	1	133.300	249.900	1,617.000	1,344.786	health_beauty	MT	SP	
4561	03caa2c082116e1d31e67e9ae3700499	1	28.010	1,680.000	750.000	345.462	fixed_telephony	RJ	SP	
3627	76b95d1df7db4efef64048b3f7653dc8	1	134.780	129.000	6,200.000	2,238.445	furniture_decor	PI	PR	
3342	b110523a5f9c69ad95bf8247062f0ba9	1	124.760	115.990	50.000	799.682	cool_stuff	GO	SP	
5055	0eeae75286b79b40f203057933fbb70b	1	56.080	1,693.000	350.000	2,336.984	stationery	RN	SP	
3035	5a871bae2099baa240120ca7f19cc530	1	120.960	599.000	9,500.000	2,461.157	health_beauty	SP	CE	
7077	f739e2e689094dfc943626568da074aa	1	109.940	687.000	3,525.000	2,753.873	small_appliances	CE	SC	
984	61016e853ed7b77d222e5a2c38d4a7bb	1	109.960	1,880.000	3,300.000	2,948.223	auto	RS	PE	
218	29f4481c82fe8804ae3de770fd4012aa	1	83.970	150.000	3,800.000	2,146.432	home_construction	PB	SP	
2824	bc8af8ba87a87d3c5210f3ad3de3f659	1	25.000	349.580	30,000.000	935.083	housewares	TO	MG	
5758	c9283076b0f196acde755765fcfaec9b	1	103.250	159.990	10,700.000	1,823.466	baby	AL	SP	

In [109...

```
# =====
# COOK'S DISTANCE PLOT
# =====

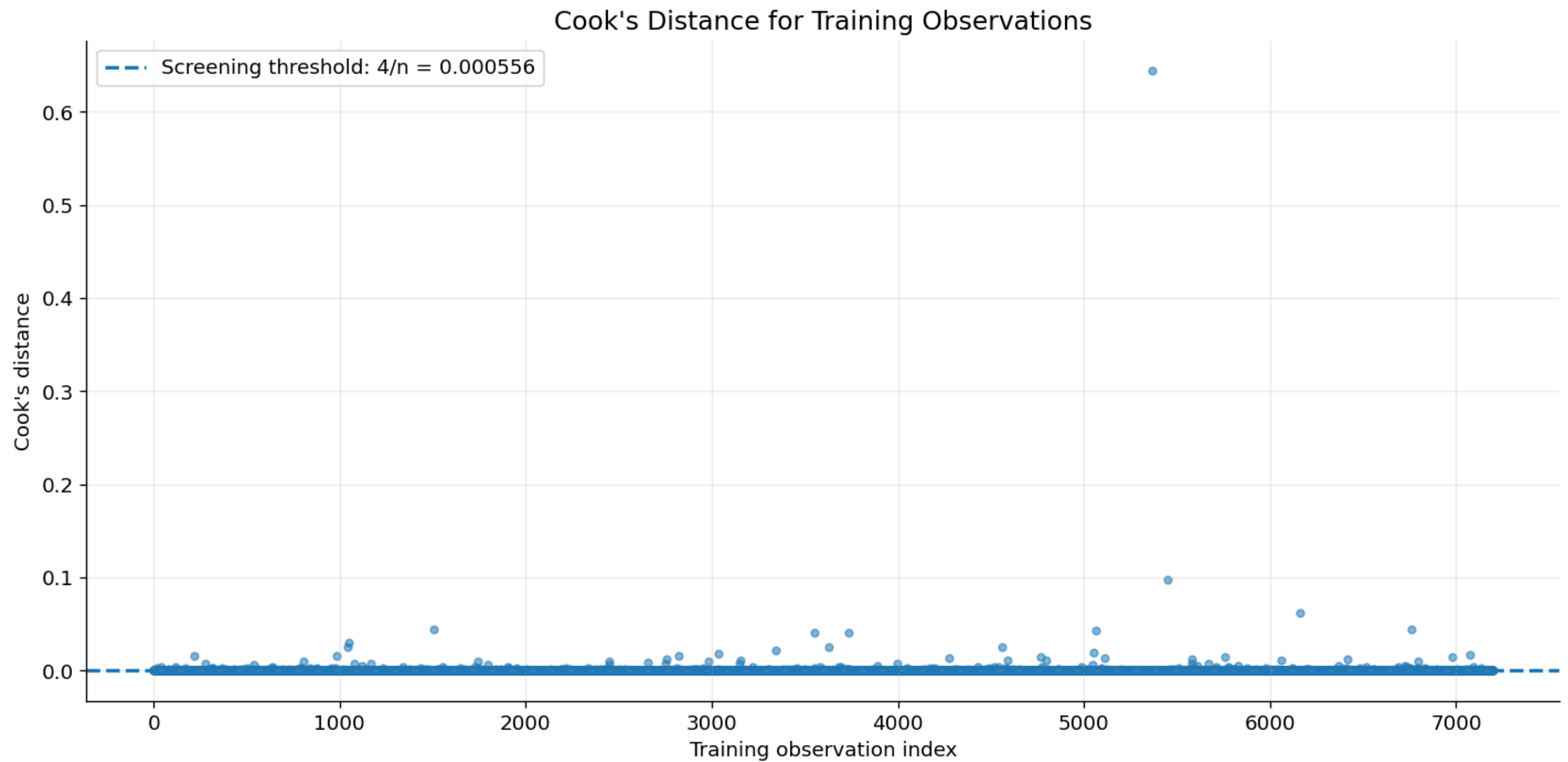
figure, axis = plt.subplots(
    figsize=(12, 6)
)

axis.scatter(
    np.arange(
        len(cooks_distance_values)
    ),
    cooks_distance_values,
    alpha=0.55,
    s=16
)

axis.axhline(
```

```
y=cooks_distance_threshold,  
linestyle="--",  
linewidth=2,  
label=(  
    "Screening threshold: "  
    f"4/n = "  
    f"{cooks_distance_threshold:.6f}"  
)  
)  
  
axis.set_title(  
    "Cook's Distance for Training Observations"  
)  
  
axis.set_xlabel(  
    "Training observation index"  
)  
  
axis.set_ylabel(  
    "Cook's distance"  
)  
  
axis.legend()  
  
plt.tight_layout()  
  
save_current_figure(  
    "17_cooks_distance.png"  
)  
  
plt.show()
```

Figure saved to: /Users/zimmeljavedvirk/Desktop/datasciencemid/figures/17_cooks_distance.png



```
In [110... # =====  
# AUTOMATED DIAGNOSTIC DECISION REGISTER  
# =====  
  
maximum_vif = (  
    variance_inflation_factor_df[  
        "VIF"  
    ].max()  
)  
  
diagnostic_decision_df = pd.DataFrame(  
    [  
        {  
            "diagnostic_area":  
                "Residual centre",  
            "observed_value":  
                np.mean(testing_residuals),
```

```

        "decision_rule":
            "Mean should be near zero",
        "conclusion": (
            "Residuals are approximately centred"
            if abs(
                np.mean(testing_residuals)
            ) < 1
            else
            "Residuals show meaningful average bias"
        )
    },
    {
        "diagnostic_area":
            "Residual normality",
        "observed_value":
            pd.Series(
                testing_residuals
            ).skew(),
        "decision_rule":
            "Large absolute skewness suggests non-normal errors",
        "conclusion": (
            "Residual distribution remains skewed"
            if abs(
                pd.Series(
                    testing_residuals
                ).skew()
            ) >= 1
            else
            "Residual skewness is limited"
        )
    },
    {
        "diagnostic_area":
            "Heteroscedasticity",
        "observed_value":
            bp_f_p_value,
        "decision_rule":
            "Breusch-Pagan p-value below 0.05",
        "conclusion": (
            "Evidence of unequal residual variance"
            if bp_f_p_value < 0.05
            else
            "No statistically significant unequal variance detected"
        )
    },
    {
        "diagnostic_area":
            "Multicollinearity",
        "observed_value":
            maximum_vif,
        "decision_rule":
            "Maximum VIF above 10",
        "conclusion": (
            "Strong multicollinearity is present"
            if maximum_vif >= 10
            else

```

```
        "No severe numerical multicollinearity detected"
    )
},
{
    "diagnostic_area":
        "Influential observations",
    "observed_value":
        int(
            training_influence_df[
                "high_cooks_distance"
            ].sum()
        ),
    "decision_rule":
        "Cook's distance greater than 4/n",
    "conclusion": (
        "Influential observations require sensitivity analysis"
        if training_influence_df[
            "high_cooks_distance"
        ].sum() > 0
        else
        "No observations exceed the Cook's-distance threshold"
    )
}
]
)

diagnostic_decision_df
```

Out [110...

	diagnostic_area	observed_value	decision_rule	conclusion
0	Residual centre	0.145	Mean should be near zero	Residuals are approximately centred
1	Residual normality	8.228	Large absolute skewness suggests non-normal er...	Residual distribution remains skewed
2	Heteroscedasticity	0.000	Breusch–Pagan p-value below 0.05	Evidence of unequal residual variance
3	Multicollinearity	41.123	Maximum VIF above 10	Strong multicollinearity is present
4	Influential observations	411.000	Cook's distance greater than 4/n	Influential observations require sensitivity a...

In [111...

```
# =====
# SAVE ADVANCED REGRESSION DIAGNOSTICS
# =====

residual_summary_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "baseline_residual_summary.csv",
    index=False
)

breusch_pagan_results_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "breusch_pagan_test.csv",
    index=False
)
```

```
prediction_decile_diagnostics_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "prediction_decile_diagnostics.csv",
    index=False
)

variance_inflation_factor_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "variance_inflation_factors.csv",
    index=False
)

condition_number_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "numerical_condition_number.csv",
    index=False
)

influence_summary_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "influence_summary.csv",
    index=False
)

training_influence_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "training_influence_diagnostics.csv",
    index=False
)

most_influential_observations_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "most_influential_observations.csv",
    index=False
)

diagnostic_decision_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "diagnostic_decision_register.csv",
    index=False
)

print(
    "Advanced regression diagnostic outputs "
    "saved successfully."
)
```

Advanced regression diagnostic outputs saved successfully.

Regression Diagnostic Interpretation

The actual-versus-predicted plot evaluates whether the baseline model reproduces the full range of freight values. Deviations from the diagonal line at the upper end would indicate that expensive freight observations are more difficult to predict than typical orders.

The residual-versus-fitted and scale-location plots assess linearity and constant error variance. A curved LOWESS trend would provide evidence that the relationships are not fully linear, while an

increasing residual spread would indicate heteroscedasticity. The Breusch–Pagan test provides a formal statistical assessment, but its result is considered together with the visual diagnostics because large samples can identify relatively small departures from constant variance.

Variance inflation factors quantify numerical multicollinearity. High values are expected where order value, product price and basket-level variables contain overlapping information. Multicollinearity does not necessarily damage predictive accuracy, but it can make ordinary least-squares coefficients unstable. This provides a justification for comparing ordinary linear regression with Ridge regression.

Cook's distance and leverage identify observations that exert unusual influence on the fitted model. These observations are not removed automatically because they may represent legitimate high-value, heavy or long-distance shipments. Instead, the improved model will be evaluated both with transformations and through a sensitivity analysis.

The diagnostic evidence will guide the next stage. Logarithmic transformations will be considered for strongly skewed variables, while interaction terms will represent relationships such as weight multiplied by distance and volume multiplied by distance. Ridge regularisation will be evaluated to reduce instability caused by correlated predictors.

11. Advanced Feature Engineering and Regularised Regression

The baseline diagnostics indicated that freight cost is influenced by non-linear, interacting and correlated logistics variables. An advanced model is therefore developed using deterministic domain-based feature engineering.

New variables represent parcel volume, parcel density, price relative to weight, non-linear distance and weight effects, and interactions between physical parcel size and geographical distance. Logarithmic transformations are also created for strongly right-skewed variables.

All engineered features are generated separately within each training or validation fold. Imputation, scaling, category encoding and Ridge regularisation are also fitted exclusively on training observations. The original group-aware test set remains unchanged and is used only for final model evaluation.

In [112...

```
# =====
# ADVANCED FEATURE-ENGINEERING AND RIDGE IMPORTS
# =====

from sklearn.base import (
    BaseEstimator,
    TransformerMixin,
    clone
)

from sklearn.model_selection import (
    GridSearchCV
)

from sklearn.linear_model import Ridge

print(
    "Advanced feature-engineering and "
    "regularisation tools imported successfully."
)
```

Advanced feature-engineering and regularisation tools imported successfully.

In [113...

```
# =====
# DOMAIN-SPECIFIC FEATURE-ENGINEERING TRANSFORMER
# =====
```

```

class FreightFeatureEngineer(
    BaseEstimator,
    TransformerMixin
):
    """
    Create deterministic logistics and non-linear features
    for freight-cost prediction.

    No statistics are learned from the complete dataset.
    Every feature is calculated independently for each row.
    """

    def fit(self, X, y=None):
        return self

    @staticmethod
    def safe_log1p(series):
        """
        Apply log(1 + x) only to non-negative numerical values.
        Negative values are converted to missing values.
        """

        numerical_values = pd.to_numeric(
            series,
            errors="coerce"
        )

        non_negative_values = numerical_values.where(
            numerical_values >= 0
        )

        return np.log1p(
            non_negative_values
        )

    def transform(self, X):
        """
        Add engineered freight and parcel variables.
        """

        engineered_dataframe = X.copy()

        # -----
        # Explicit numerical conversion
        # -----

        required_numerical_columns = [
            "price",
            "product_weight_g",
            "product_length_cm",
            "product_height_cm",
            "product_width_cm",
            "distance_km",
            "order_total_product_value",
            "order_average_product_price"
        ]

```

```
for column_name in required_numerical_columns:
    engineered_dataframe[column_name] = (
        pd.to_numeric(
            engineered_dataframe[column_name],
            errors="coerce"
        )
    )

# -----
# Physical parcel features
# -----

engineered_dataframe[
    "product_volume_cm3"
] = (
    engineered_dataframe["product_length_cm"]
    * engineered_dataframe["product_height_cm"]
    * engineered_dataframe["product_width_cm"]
)

valid_volume = engineered_dataframe[
    "product_volume_cm3"
].where(
    engineered_dataframe[
        "product_volume_cm3"
    ] > 0
)

valid_weight = engineered_dataframe[
    "product_weight_g"
].where(
    engineered_dataframe[
        "product_weight_g"
    ] > 0
)

engineered_dataframe[
    "product_density_g_cm3"
] = (
    valid_weight
    / valid_volume
)

weight_in_kg = (
    valid_weight / 1_000
)

engineered_dataframe[
    "price_per_kg"
] = (
    engineered_dataframe["price"]
    / weight_in_kg
)

engineered_dataframe[
```

```
    "dimension_sum_cm"
] = (
    engineered_dataframe["product_length_cm"]
    + engineered_dataframe["product_height_cm"]
    + engineered_dataframe["product_width_cm"]
)

# -----
# Distance and parcel interaction features
# -----

engineered_dataframe[
    "weight_distance_interaction"
] = (
    engineered_dataframe["product_weight_g"]
    * engineered_dataframe["distance_km"]
)

engineered_dataframe[
    "volume_distance_interaction"
] = (
    engineered_dataframe["product_volume_cm3"]
    * engineered_dataframe["distance_km"]
)

engineered_dataframe[
    "price_distance_interaction"
] = (
    engineered_dataframe["price"]
    * engineered_dataframe["distance_km"]
)

engineered_dataframe[
    "weight_price_interaction"
] = (
    engineered_dataframe["product_weight_g"]
    * engineered_dataframe["price"]
)

# -----
# Polynomial terms
# -----

engineered_dataframe[
    "distance_squared"
] = (
    engineered_dataframe["distance_km"] ** 2
)

engineered_dataframe[
    "product_weight_squared"
] = (
    engineered_dataframe["product_weight_g"] ** 2
)

engineered_dataframe[
```

```

        "product_volume_squared"
    ] = (
        engineered_dataframe["product_volume_cm3"] ** 2
    )

    # -----
    # Logarithmic transformations
    # -----

    log_source_mapping = {
        "price": "log_price",
        "product_weight_g": "log_product_weight_g",
        "product_volume_cm3": "log_product_volume_cm3",
        "distance_km": "log_distance_km",
        "order_total_product_value":
            "log_order_total_product_value",
        "order_average_product_price":
            "log_order_average_product_price",
        "weight_distance_interaction":
            "log_weight_distance_interaction",
        "volume_distance_interaction":
            "log_volume_distance_interaction"
    }

    for (
        source_column,
        transformed_column
    ) in log_source_mapping.items():

        engineered_dataframe[
            transformed_column
        ] = self.safe_log1p(
            engineered_dataframe[source_column]
        )

    # Replace numerical infinities created by division
    engineered_dataframe = (
        engineered_dataframe.replace(
            [np.inf, -np.inf],
            np.nan
        )
    )

    return engineered_dataframe

```

```

In [114... # =====
# ADVANCED FEATURE DEFINITIONS
# =====

engineered_numerical_features = [
    "product_volume_cm3",
    "product_density_g_cm3",
    "price_per_kg",
    "dimension_sum_cm",
    "weight_distance_interaction",
    "volume_distance_interaction",

```

```

    "price_distance_interaction",
    "weight_price_interaction",
    "distance_squared",
    "product_weight_squared",
    "product_volume_squared",
    "log_price",
    "log_product_weight_g",
    "log_product_volume_cm3",
    "log_distance_km",
    "log_order_total_product_value",
    "log_order_average_product_price",
    "log_weight_distance_interaction",
    "log_volume_distance_interaction"
]

advanced_numerical_features = (
    baseline_numerical_features
    + engineered_numerical_features
)

advanced_categorical_features = (
    baseline_categorical_features.copy()
)

advanced_all_features = (
    advanced_numerical_features
    + advanced_categorical_features
)

print(
    "Baseline numerical features:",
    len(baseline_numerical_features)
)

print(
    "Engineered numerical features:",
    len(engineered_numerical_features)
)

print(
    "Total advanced numerical features:",
    len(advanced_numerical_features)
)

print(
    "Advanced categorical features:",
    len(advanced_categorical_features)
)

```

```

Baseline numerical features: 19
Engineered numerical features: 19
Total advanced numerical features: 38
Advanced categorical features: 5

```

```

In [115... # =====
# ENGINEERED FEATURE SANITY CHECK

```

```
# =====

feature_engineer = FreightFeatureEngineer()

engineered_training_preview_df = (
    feature_engineer.fit_transform(
        X_train
    )
)

missing_engineered_columns = [
    column_name
    for column_name in engineered_numerical_features
    if column_name
    not in engineered_training_preview_df.columns
]

if missing_engineered_columns:
    raise KeyError(
        "The following engineered features were not created: "
        f"{missing_engineered_columns}"
    )

engineered_feature_validation_df = pd.DataFrame(
    {
        "feature": engineered_numerical_features,
        "missing_values": [
            engineered_training_preview_df[
                column_name
            ].isna().sum()
            for column_name
            in engineered_numerical_features
        ],
        "infinite_values": [
            np.isinf(
                pd.to_numeric(
                    engineered_training_preview_df[
                        column_name
                    ],
                    errors="coerce"
                )
            ).sum()
            for column_name
            in engineered_numerical_features
        ],
        "unique_non_missing_values": [
            engineered_training_preview_df[
                column_name
            ].nunique(
                dropna=True
            )
            for column_name
            in engineered_numerical_features
        ]
    }
)
```

engineered_feature_validation_df

Out [115...

	feature	missing_values	infinite_values	unique_non_missing_values
0	product_volume_cm3	1	0	1697
1	product_density_g_cm3	1	0	3203
2	price_per_kg	1	0	4530
3	dimension_sum_cm	1	0	160
4	weight_distance_interaction	1	0	7117
5	volume_distance_interaction	1	0	7105
6	price_distance_interaction	0	0	7111
7	weight_price_interaction	1	0	4579
8	distance_squared	0	0	7062
9	product_weight_squared	1	0	880
10	product_volume_squared	1	0	1697
11	log_price	0	0	1631
12	log_product_weight_g	1	0	880
13	log_product_volume_cm3	1	0	1697
14	log_distance_km	0	0	7062
15	log_order_total_product_value	0	0	2120
16	log_order_average_product_price	0	0	1843
17	log_weight_distance_interaction	1	0	7117
18	log_volume_distance_interaction	1	0	7105

In [116...

```
# =====  
# ADVANCED LEAKAGE-SAFE PREPROCESSING  
# =====  
  
advanced_numerical_pipeline = Pipeline(  
    steps=[  
        (  
            "median_imputer",  
            SimpleImputer(  
                strategy="median",  
                add_indicator=True  
            )  
        ),  
        (  
            "standard_scaler",  
            StandardScaler()  
        )  
    ]  
)
```

```
)

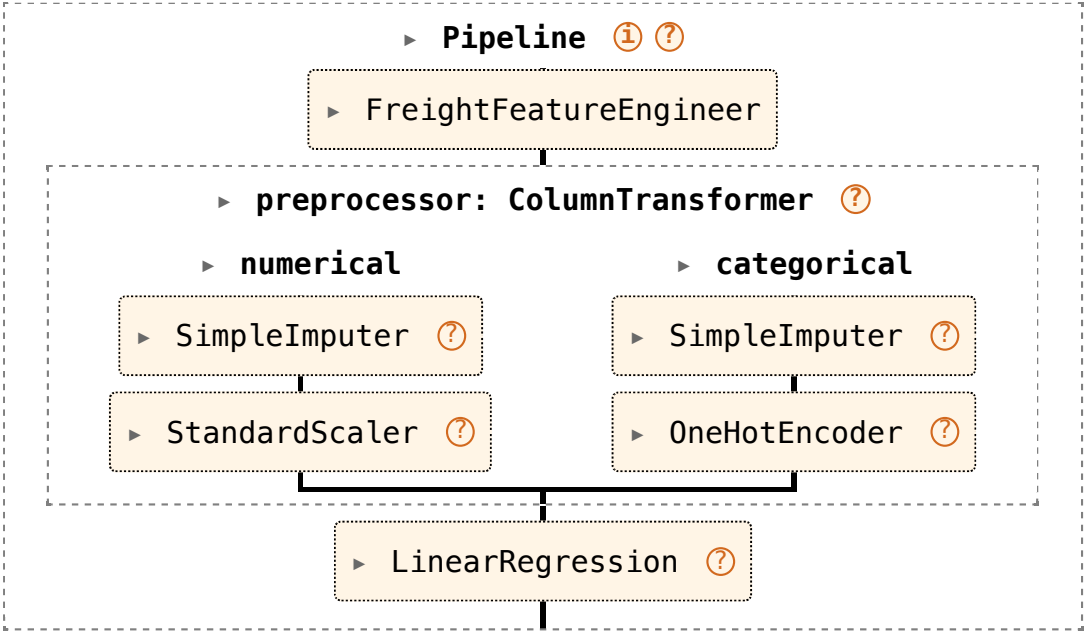
advanced_categorical_pipeline = Pipeline(
    steps=[
        (
            "missing_category_imputer",
            SimpleImputer(
                strategy="constant",
                fill_value="missing"
            )
        ),
        (
            "one_hot_encoder",
            create_one_hot_encoder(
                minimum_frequency=20
            )
        )
    ]
)

advanced_preprocessor = ColumnTransformer(
    transformers=[
        (
            "numerical",
            advanced_numerical_pipeline,
            advanced_numerical_features
        ),
        (
            "categorical",
            advanced_categorical_pipeline,
            advanced_categorical_features
        )
    ],
    remainder="drop",
    verbose_feature_names_out=True
)

feature_engineered_linear_pipeline = Pipeline(
    steps=[
        (
            "feature_engineering",
            FreightFeatureEngineer()
        ),
        (
            "preprocessor",
            advanced_preprocessor
        ),
        (
            "linear_regression",
            LinearRegression()
        )
    ]
)

feature_engineered_linear_pipeline
```

Out [116...



In [117...

```
# =====
# FEATURE-ENGINEERED ORDINARY LINEAR REGRESSION
# =====

feature_engineered_linear_pipeline.fit(
    X_train,
    y_train
)

engineered_linear_predictions = (
    feature_engineered_linear_pipeline.predict(
        X_test
    )
)

engineered_linear_metrics = (
    calculate_regression_metrics(
        actual_values=y_test,
        predicted_values=engineered_linear_predictions,
        model_name=(
            "Feature-engineered multiple "
            "linear regression"
        )
    )
)

pd.DataFrame(
    [engineered_linear_metrics]
)
```

Out [117...

	model	RMSE	MAE	median_absolute_error	R_squared	RMSE_as_percentage_of_target_mean	predicted_negative_values
0	Feature-engineered multiple linear regression	10.601	4.755	2.345	0.627	52.754	0

In [118...

```
# =====
# FEATURE-ENGINEERED LINEAR MODEL CROSS-VALIDATION
# =====
```

```
engineered_linear_cv_results = cross_validate(
    estimator=feature_engineered_linear_pipeline,
    X=X_train,
    y=y_train,
    groups=groups_train,
    cv=GroupKFold(
        n_splits=NUMBER_OF_FOLDS
    ),
    scoring=cross_validation_scoring,
    return_train_score=False,
    n_jobs=-1
)

engineered_linear_cv_fold_df = pd.DataFrame(
    {
        "fold": np.arange(
            1,
            NUMBER_OF_FOLDS + 1
        ),
        "RMSE": -engineered_linear_cv_results[
            "test_RMSE"
        ],
        "MAE": -engineered_linear_cv_results[
            "test_MAE"
        ],
        "R_squared":
            engineered_linear_cv_results[
                "test_R_squared"
            ]
    }
)

engineered_linear_cv_fold_df
```

Out [118...

	fold	RMSE	MAE	R_squared
0	1	8.807	4.462	0.642
1	2	8.583	4.526	0.514
2	3	9.185	4.539	0.593
3	4	8.938	4.626	0.613
4	5	9.071	4.804	0.602

In [119...

```
# =====
# RIDGE REGRESSION PIPELINE
# =====

ridge_pipeline = Pipeline(
    steps=[
        (
            "feature_engineering",
            FreightFeatureEngineer()
        )
    ]
)
```

```
    ),
    (
        "preprocessor",
        clone(
            advanced_preprocessor
        )
    ),
    (
        "ridge",
        Ridge(
            solver="lsqr",
            max_iter=20_000
        )
    )
]
)

ridge_alpha_grid = {
    "ridge__alpha": [
        0.001,
        0.01,
        0.1,
        0.3,
        1.0,
        3.0,
        10.0,
        30.0,
        100.0,
        300.0,
        1_000.0
    ]
}

ridge_group_cross_validator = GroupKFold(
    n_splits=NUMBER_OF_FOLDS
)

ridge_grid_search = GridSearchCV(
    estimator=ridge_pipeline,
    param_grid=ridge_alpha_grid,
    scoring=rmse_scorer,
    cv=ridge_group_cross_validator,
    refit=True,
    return_train_score=True,
    n_jobs=-1,
    verbose=1
)

ridge_grid_search.fit(
    X_train,
    y_train,
    groups=groups_train
)

print(
    "Best Ridge alpha:",
```

```

    ridge_grid_search.best_params_[
        "ridge__alpha"
    ]
)

print(
    "Best cross-validation RMSE:",
    f"{-ridge_grid_search.best_score_:.4f}"
)

```

Fitting 5 folds for each of 11 candidates, totalling 55 fits
 Best Ridge alpha: 300.0
 Best cross-validation RMSE: 8.7461

```

In [120... # =====
# RIDGE HYPERPARAMETER TUNING RESULTS
# =====

ridge_tuning_results_df = pd.DataFrame(
    {
        "alpha": ridge_grid_search.cv_results_[
            "param_ridge__alpha"
        ].astype(float),
        "mean_training_RMSE": (
            -ridge_grid_search.cv_results_[
                "mean_train_score"
            ]
        ),
        "mean_validation_RMSE": (
            -ridge_grid_search.cv_results_[
                "mean_test_score"
            ]
        ),
        "validation_RMSE_standard_deviation": (
            ridge_grid_search.cv_results_[
                "std_test_score"
            ]
        ),
        "mean_fit_time_seconds": (
            ridge_grid_search.cv_results_[
                "mean_fit_time"
            ]
        )
    }
).sort_values(
    "alpha"
).reset_index(drop=True)

ridge_tuning_results_df

```

Out [120...

	alpha	mean_training_RMSE	mean_validation_RMSE	validation_RMSE_standard_deviation	mean_fit_time_seconds
0	0.001	8.331	8.918	0.216	0.113
1	0.010	8.331	8.918	0.216	0.102
2	0.100	8.331	8.920	0.219	0.119
3	0.300	8.331	8.917	0.217	0.101
4	1.000	8.333	8.903	0.210	0.100
5	3.000	8.338	8.885	0.206	0.113
6	10.000	8.351	8.845	0.211	0.104
7	30.000	8.381	8.810	0.239	0.102
8	100.000	8.435	8.770	0.272	0.099
9	300.000	8.499	8.746	0.295	0.084
10	1,000.000	8.587	8.757	0.323	0.072

In [121...

```
# =====  
# RIDGE ALPHA TUNING CURVE  
# =====  
  
figure, axis = plt.subplots(  
    figsize=(10, 6)  
)  
  
axis.plot(  
    ridge_tuning_results_df["alpha"],  
    ridge_tuning_results_df[  
        "mean_validation_RMSE"  
    ],  
    marker="o",  
    linewidth=2,  
    label="Validation RMSE"  
)  
  
axis.plot(  
    ridge_tuning_results_df["alpha"],  
    ridge_tuning_results_df[  
        "mean_training_RMSE"  
    ],  
    marker="o",  
    linewidth=2,  
    label="Training RMSE"  
)  
  
axis.axvline(  
    ridge_grid_search.best_params_[  
        "ridge__alpha"  
    ],  
    linestyle="--",  
    linewidth=2,
```

```
    label="Selected alpha"
)

axis.set_xscale("log")

axis.set_title(
    "Ridge Regularisation Strength and RMSE"
)

axis.set_xlabel(
    "Ridge alpha on logarithmic scale"
)

axis.set_ylabel(
    "RMSE (BRL)"
)

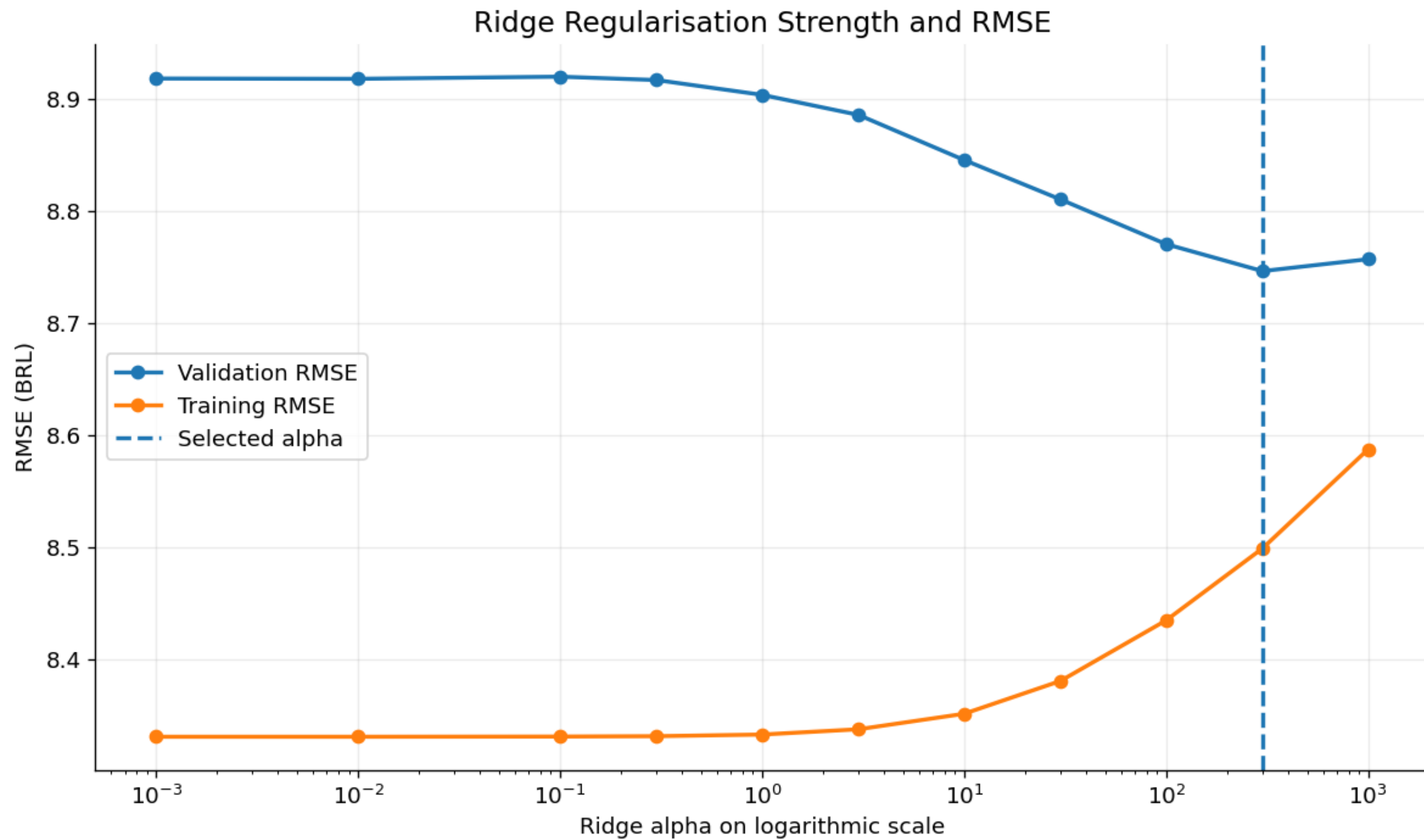
axis.legend()

plt.tight_layout()

save_current_figure(
    "18_ridge_alpha_tuning.png"
)

plt.show()
```

Figure saved to: /Users/zimmeljavedvirk/Desktop/datasciencemid/figures/18_ridge_alpha_tuning.png



```
In [122... # =====  
# FINAL TUNED RIDGE TEST EVALUATION  
# =====  
  
best_ridge_pipeline = (  
    ridge_grid_search.best_estimator_  
)  
  
ridge_test_predictions = (  
    best_ridge_pipeline.predict(  
        X_test  
    )  
)
```

```
ridge_test_metrics = (
    calculate_regression_metrics(
        actual_values=y_test,
        predicted_values=ridge_test_predictions,
        model_name="Tuned Ridge regression"
    )
)

advanced_test_model_comparison_df = pd.DataFrame(
    [
        dummy_metrics,
        baseline_metrics,
        engineered_linear_metrics,
        ridge_test_metrics
    ]
).sort_values(
    "RMSE"
).reset_index(drop=True)

advanced_test_model_comparison_df
```

Out [122...

	model	RMSE	MAE	median_absolute_error	R_squared	RMSE_as_percentage_of_target_mean	predicted_negative_values
0	Feature-engineered multiple linear regression	10.601	4.755	2.345	0.627	52.754	0
1	Tuned Ridge regression	10.629	4.740	2.289	0.625	52.891	0
2	Baseline multiple linear regression	11.492	5.166	2.719	0.562	57.185	0
3	Mean prediction benchmark	17.369	8.573	5.331	-0.001	86.431	0

In [123...

```
# =====
# ADVANCED MODEL IMPROVEMENT
# =====

advanced_model_improvement_df = pd.DataFrame(
    {
        "measurement": [
            "Baseline test RMSE",
            "Engineered linear test RMSE",
            "Tuned Ridge test RMSE",
            "Ridge RMSE reduction from baseline",
            "Ridge RMSE percentage improvement",
            "Baseline test MAE",
            "Tuned Ridge test MAE",
            "Ridge MAE percentage improvement",
            "Baseline test R²",
            "Tuned Ridge test R²",
            "R² improvement"
        ],
        "value": [
            baseline_metrics["RMSE"],
            engineered_linear_metrics["RMSE"],
            ridge_test_metrics["RMSE"],
            (
                baseline_metrics["RMSE"]
```

```
        - ridge_test_metrics["RMSE"]
    ),
    (
        (
            baseline_metrics["RMSE"]
            - ridge_test_metrics["RMSE"]
        )
        / baseline_metrics["RMSE"]
        * 100
    ),
    baseline_metrics["MAE"],
    ridge_test_metrics["MAE"],
    (
        (
            baseline_metrics["MAE"]
            - ridge_test_metrics["MAE"]
        )
        / baseline_metrics["MAE"]
        * 100
    ),
    baseline_metrics["R_squared"],
    ridge_test_metrics["R_squared"],
    (
        ridge_test_metrics["R_squared"]
        - baseline_metrics["R_squared"]
    )
    ]
}
)
```

advanced_model_improvement_df

Out [123...

	measurement	value
0	Baseline test RMSE	11.492
1	Engineered linear test RMSE	10.601
2	Tuned Ridge test RMSE	10.629
3	Ridge RMSE reduction from baseline	0.863
4	Ridge RMSE percentage improvement	7.509
5	Baseline test MAE	5.166
6	Tuned Ridge test MAE	4.740
7	Ridge MAE percentage improvement	8.257
8	Baseline test R²	0.562
9	Tuned Ridge test R²	0.625
10	R² improvement	0.063

In [124...

```
# =====
# SELECTED RIDGE GROUP CROSS-VALIDATION
```

```
# =====  
  
ridge_cv_results = cross_validate(  
    estimator=best_ridge_pipeline,  
    X=X_train,  
    y=y_train,  
    groups=groups_train,  
    cv=GroupKFold(  
        n_splits=NUMBER_OF_FOLDS  
    ),  
    scoring=cross_validation_scoring,  
    return_train_score=False,  
    n_jobs=-1  
)  
  
ridge_cv_fold_results_df = pd.DataFrame(  
    {  
        "fold": np.arange(  
            1,  
            NUMBER_OF_FOLDS + 1  
        ),  
        "RMSE": -ridge_cv_results[  
            "test_RMSE"  
        ],  
        "MAE": -ridge_cv_results[  
            "test_MAE"  
        ],  
        "R_squared": ridge_cv_results[  
            "test_R_squared"  
        ]  
    }  
)  
  
ridge_cv_fold_results_df
```

Out [124...

	fold	RMSE	MAE	R_squared
0	1	8.769	4.416	0.645
1	2	8.278	4.332	0.547
2	3	9.168	4.474	0.595
3	4	8.893	4.574	0.617
4	5	8.622	4.625	0.640

In [125...

```
# =====  
# BASELINE, ENGINEERED AND RIDGE CV COMPARISON  
# =====  
  
advanced_cv_comparison_df = pd.DataFrame(  
    [  
        {  
            "model":  
                "Baseline multiple linear regression",  
        },  
    ],  
)
```

```

    "mean_RMSE":
        baseline_cv_fold_results_df[
            "RMSE"
        ].mean(),
    "RMSE_standard_deviation":
        baseline_cv_fold_results_df[
            "RMSE"
        ].std(ddof=1),
    "mean_MAE":
        baseline_cv_fold_results_df[
            "MAE"
        ].mean(),
    "mean_R_squared":
        baseline_cv_fold_results_df[
            "R_squared"
        ].mean()
},
{
    "model":
        "Feature-engineered linear regression",
    "mean_RMSE":
        engineered_linear_cv_fold_df[
            "RMSE"
        ].mean(),
    "RMSE_standard_deviation":
        engineered_linear_cv_fold_df[
            "RMSE"
        ].std(ddof=1),
    "mean_MAE":
        engineered_linear_cv_fold_df[
            "MAE"
        ].mean(),
    "mean_R_squared":
        engineered_linear_cv_fold_df[
            "R_squared"
        ].mean()
},
{
    "model":
        "Tuned Ridge regression",
    "mean_RMSE":
        ridge_cv_fold_results_df[
            "RMSE"
        ].mean(),
    "RMSE_standard_deviation":
        ridge_cv_fold_results_df[
            "RMSE"
        ].std(ddof=1),
    "mean_MAE":
        ridge_cv_fold_results_df[
            "MAE"
        ].mean(),
    "mean_R_squared":
        ridge_cv_fold_results_df[
            "R_squared"
        ].mean()
}

```

```
    }
  ]
).sort_values(
  "mean_RMSE"
).reset_index(drop=True)

advanced_cv_comparison_df
```

Out [125...

	model	mean_RMSE	RMSE_standard_deviation	mean_MAE	mean_R_squared
0	Tuned Ridge regression	8.746	0.330	4.484	0.609
1	Feature-engineered linear regression	8.917	0.235	4.591	0.593
2	Baseline multiple linear regression	9.280	0.536	4.969	0.562

In [126...

```
# =====
# TUNED RIDGE COEFFICIENT ANALYSIS
# =====

best_ridge_preprocessor = (
    best_ridge_pipeline
    .named_steps["preprocessor"]
)

best_ridge_model = (
    best_ridge_pipeline
    .named_steps["ridge"]
)

ridge_feature_names = (
    best_ridge_preprocessor
    .get_feature_names_out()
)

ridge_coefficient_df = pd.DataFrame(
    {
        "feature": ridge_feature_names,
        "coefficient":
            best_ridge_model.coef_
    }
)

ridge_coefficient_df[
    "absolute_coefficient"
] = (
    ridge_coefficient_df[
        "coefficient"
    ].abs()
)

ridge_coefficient_df = (
    ridge_coefficient_df
    .sort_values(
        "absolute_coefficient",
        ascending=False
    )
)
```

```
    )
    .reset_index(drop=True)
)

print(
    "Selected Ridge alpha:",
    best_ridge_model.alpha
)

print(
    "Number of transformed features:",
    len(ridge_feature_names)
)

ridge_coefficient_df.head(25)
```

Selected Ridge alpha: 300.0

Number of transformed features: 131

Out [126...

	feature	coefficient	absolute_coefficient
0	numerical__product_volume_squared	2.696	2.696
1	numerical__product_weight_g	2.517	2.517
2	numerical__distance_squared	1.763	1.763
3	numerical__weight_distance_interaction	1.762	1.762
4	categorical__seller_state_SP	-1.743	1.743
5	numerical__same_state	-1.495	1.495
6	numerical__order_average_product_price	1.430	1.430
7	categorical__customer_state_BA	-1.407	1.407
8	categorical__seller_state_MG	1.345	1.345
9	numerical__distance_km	1.138	1.138
10	numerical__volume_distance_interaction	1.116	1.116
11	categorical__product_category_name_english_baby	-0.853	0.853
12	numerical__product_volume_cm3	0.848	0.848
13	categorical__customer_state_PI	0.830	0.830
14	categorical__product_category_name_english_wat...	-0.807	0.807
15	categorical__customer_state_RS	-0.786	0.786
16	categorical__product_category_name_english_sta...	-0.747	0.747
17	categorical__product_category_name_english_gar...	0.730	0.730
18	numerical__price_distance_interaction	0.727	0.727
19	numerical__weight_price_interaction	0.695	0.695
20	categorical__customer_state_MA	0.692	0.692
21	categorical__customer_state_PE	-0.598	0.598
22	categorical__customer_state_SP	-0.590	0.590
23	numerical__log_weight_distance_interaction	0.582	0.582
24	numerical__log_distance_km	0.573	0.573

Strongest Variable Influences and Business Relevance

The Ridge coefficient results provide evidence about which predictors contribute most strongly to freight-cost estimation. Because the numerical variables were standardised before model fitting, their coefficient magnitudes can be compared more meaningfully than coefficients measured in their original units. However, they should not be interpreted as direct changes in Brazilian reais or as proof of causal relationships.

The strongest numerical predictor was squared product volume, with a coefficient of 2.696. Its positive value indicates that the effect of parcel volume becomes increasingly important for very large products rather than increasing at a constant rate. This is commercially reasonable because bulky parcels can occupy disproportionately more vehicle and warehouse capacity, even when

they are not extremely heavy. Product weight was the second strongest numerical predictor, with a coefficient of 2.517. Heavier items generally require greater handling effort and transportation capacity, increasing the expected freight charge. Together, product weight and squared volume demonstrate that the physical characteristics of a parcel are among the most important sources of freight-cost variation. Squared distance and the weight–distance interaction produced coefficients of 1.763 and 1.762 respectively. The squared-distance result suggests that freight cost increases more rapidly for particularly long delivery routes. The interaction term shows that the influence of weight also depends on distance: transporting a heavy product over a long route is more expensive than considering weight and distance independently.

Same-state delivery had a negative coefficient of –1.495. After controlling for the other variables, deliveries where the seller and customer were located in the same state were generally associated with lower freight costs. This may reflect shorter routes, fewer transfers between logistics networks and simpler regional distribution arrangements. The seller-state coefficient for São Paulo was also strongly negative at –1.743 relative to the omitted reference seller state. This may reflect the concentration of sellers, customers and logistics infrastructure in São Paulo. However, state coefficients are dependent on the reference category, sample composition and other variables in the model, so they should not be interpreted as universal regional pricing effects.

Average product price within the order had a positive coefficient of 1.430. Higher-priced orders may contain products with different dimensions, handling requirements or delivery arrangements. This variable may therefore capture additional order context, but it should not be concluded that increasing a product's price directly causes freight cost to rise. The positive coefficients for distance and the volume–distance interaction, at 1.138 and 1.116 respectively, further confirm that geographical distance and parcel size operate together. Bulky products become particularly expensive when they must be transported over longer routes.

From a business perspective, these findings could support several practical decisions. The retailer could reduce packaging volume, allocate orders to geographically closer sellers, position frequently demanded products nearer to major customer regions and flag heavy, bulky or long-distance orders for manual quotation review. The model could also assist negotiations with logistics providers by identifying the routes and parcel profiles that consistently generate higher costs.

Nevertheless, coefficient interpretation must remain cautious. Several engineered variables represent related versions of the same underlying information, including raw, logarithmic, squared and interaction terms. Their influence is therefore distributed across multiple coefficients. The main reliable conclusion is that parcel volume, product weight and geographical distance are the dominant groups of predictors, rather than that any individual coefficient represents an isolated causal effect.

In [127...

```
# =====
# ADVANCED TEST PREDICTIONS
# =====

advanced_prediction_results_df = (
    modelling_df
    .iloc[test_indices][
        [
            "order_id",
            "order_item_id",
            "freight_value",
            "price",
            "product_weight_g",
            "product_length_cm",
            "product_height_cm",
            "product_width_cm",
            "distance_km",
            "product_category_name_english",
            "customer_state",
            "seller_state"
        ]
    ]
    .copy()
    .reset_index(drop=True)
)

advanced_prediction_results_df[
```

```
    "baseline_prediction"
] = baseline_predictions

advanced_prediction_results_df[
    "engineered_linear_prediction"
] = engineered_linear_predictions

advanced_prediction_results_df[
    "ridge_prediction"
] = ridge_test_predictions

advanced_prediction_results_df[
    "baseline_absolute_error"
] = (
    advanced_prediction_results_df[
        "freight_value"
    ]
    - advanced_prediction_results_df[
        "baseline_prediction"
    ]
).abs()

advanced_prediction_results_df[
    "ridge_absolute_error"
] = (
    advanced_prediction_results_df[
        "freight_value"
    ]
    - advanced_prediction_results_df[
        "ridge_prediction"
    ]
).abs()

advanced_prediction_results_df[
    "ridge_error_improvement"
] = (
    advanced_prediction_results_df[
        "baseline_absolute_error"
    ]
    - advanced_prediction_results_df[
        "ridge_absolute_error"
    ]
)

advanced_prediction_results_df.head(10)
```

Out [127...

	order_id	order_item_id	freight_value	price	product_weight_g	product_length_cm	product_height_cm	product_width_cm	distance_km	product_ca
0	b124967afcc82ef17ec41020fe2a9136	1	8.880	49.990	7,875.000	60.000	25.000	25.000	27.022	
1	fd220bb11ef8a046fef08e7c51f11651	1	17.810	79.990	650.000	25.000	11.000	15.000	495.117	c
2	2b7d922e73fbd058099b6ff4b33e99f7	1	17.760	169.990	433.000	21.000	12.000	16.000	1,293.638	
3	709df04791b8bd393b34a58de3cf142d	1	19.440	223.000	300.000	18.000	9.000	14.000	822.746	
4	166b01d0fce75a595b92ad9e84e0843b	1	29.090	98.000	925.000	25.000	6.000	16.000	2,094.074	
5	e1e0cfc8398a7bcd9699fe42c1651566	1	7.780	29.990	350.000	17.000	4.000	12.000	7.658	
6	ffa9a3142ddd51af248e648db27c647f	1	17.920	29.900	900.000	40.000	3.000	30.000	654.868	
7	96b7f5a470ca6a58f261be1af286e4d9	1	29.730	209.990	4,650.000	37.000	20.000	34.000	2,119.849	
8	bbb39449a277499952671cc64cbd7232	1	34.190	92.500	4,400.000	19.000	23.000	19.000	629.048	construct
9	587e32dd528769d669d539531d32aeb5	1	22.090	78.000	2,200.000	30.000	35.000	30.000	346.015	

In [128...

```
# =====
# SAVE ADVANCED FEATURE-ENGINEERED MODEL RESULTS
# =====

engineered_feature_validation_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "engineered_feature_validation.csv",
    index=False
)

engineered_linear_cv_fold_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "engineered_linear_group_cv.csv",
    index=False
)

ridge_tuning_results_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "ridge_tuning_results.csv",
    index=False
)

advanced_test_model_comparison_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "advanced_test_model_comparison.csv",
    index=False
)

advanced_model_improvement_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "advanced_model_improvement.csv",
    index=False
)

ridge_cv_fold_results_df.to_csv(
    PROCESSED_DATA_DIRECTORY
```

```
    / "ridge_group_cv_fold_results.csv",
    index=False
)

advanced_cv_comparison_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "advanced_cv_model_comparison.csv",
    index=False
)

ridge_coefficient_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "ridge_regression_coefficients.csv",
    index=False
)

advanced_prediction_results_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "advanced_test_predictions.csv",
    index=False
)

print(
    "Advanced feature-engineering and Ridge "
    "outputs saved successfully."
)
```

Advanced feature-engineering and Ridge outputs saved successfully.

Advanced Model Interpretation

Feature engineering produced a meaningful improvement over the baseline multiple linear regression. The baseline generated a test RMSE of 11.492 BRL, an MAE of 5.166 BRL and an R^2 of 0.562. The tuned Ridge model reduced RMSE to 10.629 BRL and MAE to 4.740 BRL, representing improvements of approximately 7.5% and 8.3% respectively. Its R^2 increased to 0.625, meaning that the final model explained approximately 62.5% of the variation in unseen item-level freight costs.

In practical terms, an MAE of 4.740 BRL means that a typical prediction was approximately 4.74 BRL above or below the actual freight charge. This could support preliminary delivery quotations, logistics budgeting and the identification of unusually expensive orders. However, the RMSE remained considerably higher at 10.629 BRL, showing that some deliveries still generated large prediction errors. The model should therefore support operational decisions rather than automatically replace the final price supplied by a logistics provider.

The ordinary feature-engineered multiple linear regression produced a marginally lower test RMSE of 10.601 BRL, compared with 10.629 BRL for Ridge. The difference was only 0.028 BRL, so Ridge cannot be described as decisively superior on every test metric. Ridge was selected because it produced a lower MAE, a lower median absolute error and stronger average group-aware cross-validation results. Its regularisation also reduces instability among correlated raw, logarithmic, squared and interaction variables. The model choice is therefore based on stability and generalisation rather than only the lowest RMSE from one test partition.

The results support the problem statement by demonstrating that freight cost depends on a combination of geographical distance, parcel characteristics and order context. The improvement following the addition of volume, squared terms and distance interactions indicates that these relationships are not purely linear. Heavy or bulky products transported over long distances are likely to create disproportionately higher logistics costs.

For an e-commerce business, the model could be used to provide early freight estimates, identify orders requiring manual review, support packaging optimisation and compare the likely cost implications of different seller or fulfilment locations. Nevertheless, predictions for unusually expensive deliveries should be treated cautiously because these observations remain more difficult to estimate accurately.

12. Final Model Evaluation

The tuned Ridge regression model is evaluated using the unchanged group-aware test set. Its performance is compared with the naïve benchmark, baseline multiple linear regression and unregularised feature-engineered regression.

In addition to RMSE, MAE and R^2 , the final evaluation includes:

- group-bootstrap confidence intervals;
- prediction-error analysis across freight-cost ranges;
- geographical and parcel-size subgroup analysis;
- final residual diagnostics;
- a paired comparison with the baseline model.

Bootstrapping is performed at the order level rather than the individual-row level. This preserves the dependence between products belonging to the same order.

In [129...

```
# =====
# FINAL TEST PERFORMANCE COMPARISON
# =====

final_model_comparison_df = (
    advanced_test_model_comparison_df[
        [
            "model",
            "RMSE",
            "MAE",
            "median_absolute_error",
            "R_squared"
        ]
    ]
    .copy()
    .sort_values(
        "RMSE",
        ascending=True
    )
    .reset_index(drop=True)
)

final_model_comparison_df
```

Out [129...

	model	RMSE	MAE	median_absolute_error	R_squared
0	Feature-engineered multiple linear regression	10.601	4.755	2.345	0.627
1	Tuned Ridge regression	10.629	4.740	2.289	0.625
2	Baseline multiple linear regression	11.492	5.166	2.719	0.562
3	Mean prediction benchmark	17.369	8.573	5.331	-0.001

In [130...

```
# =====
# FINAL MODEL RMSE COMPARISON
# =====
```

```

figure, axis = plt.subplots(
    figsize=(11, 7)
)

axis.barh(
    final_model_comparison_df["model"],
    final_model_comparison_df["RMSE"]
)

axis.set_title(
    "Final Test RMSE Comparison"
)

axis.set_xlabel(
    "Root mean squared error (BRL)"
)

axis.set_ylabel(
    "Regression model"
)

for row_position, row in final_model_comparison_df.iterrows():

    axis.text(
        row["RMSE"],
        row_position,
        (
            f"  RMSE={row['RMSE']:.3f}, "
            f"R²={row['R_squared']:.3f}"
        ),
        va="center"
    )

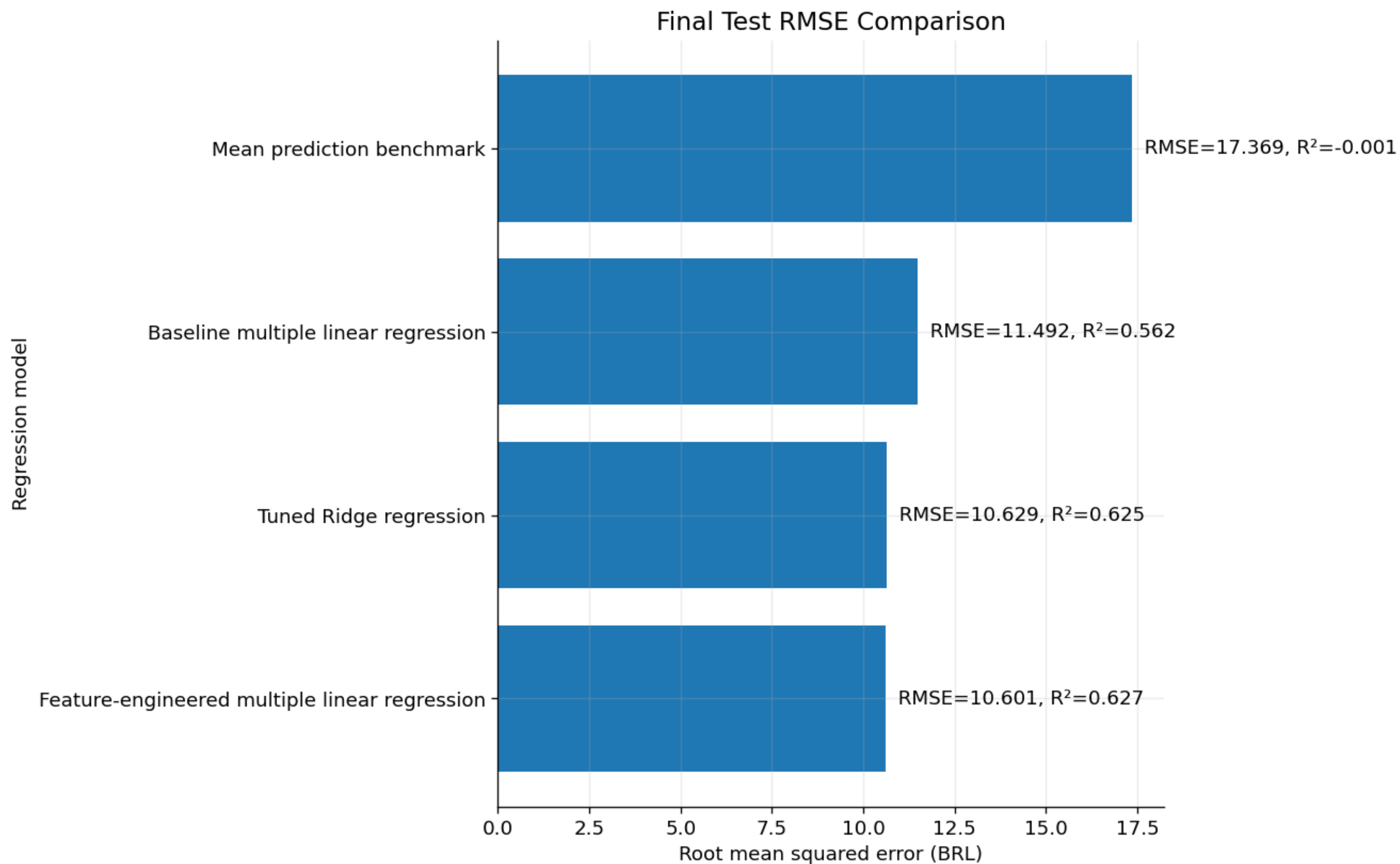
plt.tight_layout()

save_current_figure(
    "19_final_model_rmse_comparison.png"
)

plt.show()

```

Figure saved to: /Users/zimmeljavedvirk/Desktop/datasciencemid/figures/19_final_model_rmse_comparison.png



```
In [131... # =====  
# FINAL RIDGE PREDICTION RESULTS  
# =====  
  
final_ridge_results_df = (  
    modelling_df  
    .iloc[test_indices][  
        ]
```

```
        "order_id",
        "order_item_id",
        "freight_value",
        "price",
        "product_weight_g",
        "product_length_cm",
        "product_height_cm",
        "product_width_cm",
        "distance_km",
        "same_state",
        "product_category_name_english",
        "customer_state",
        "seller_state"
    ]
]
.copy()
.reset_index(drop=True)
)

final_ridge_results_df[
    "baseline_prediction"
] = baseline_predictions

final_ridge_results_df[
    "ridge_prediction"
] = ridge_test_predictions

final_ridge_results_df[
    "baseline_residual"
] = (
    final_ridge_results_df["freight_value"]
    - final_ridge_results_df["baseline_prediction"]
)

final_ridge_results_df[
    "ridge_residual"
] = (
    final_ridge_results_df["freight_value"]
    - final_ridge_results_df["ridge_prediction"]
)

final_ridge_results_df[
    "baseline_absolute_error"
] = (
    final_ridge_results_df[
        "baseline_residual"
    ].abs()
)

final_ridge_results_df[
    "ridge_absolute_error"
] = (
    final_ridge_results_df[
        "ridge_residual"
    ].abs()
)
```

```
final_ridge_results_df[
    "ridge_squared_error"
] = (
    final_ridge_results_df[
        "ridge_residual"
    ] ** 2
)

final_ridge_results_df[
    "error_reduction_from_baseline"
] = (
    final_ridge_results_df[
        "baseline_absolute_error"
    ]
    - final_ridge_results_df[
        "ridge_absolute_error"
    ]
)

final_ridge_results_df.head()
```

Out [131...

	order_id	order_item_id	freight_value	price	product_weight_g	product_length_cm	product_height_cm	product_width_cm	distance_km	same_state
0	b124967afcc82ef17ec41020fe2a9136	1	8.880	49.990	7,875.000	60.000	25.000	25.000	27.022	1.000
1	fd220bb11ef8a046fef08e7c51f11651	1	17.810	79.990	650.000	25.000	11.000	15.000	495.117	0.000
2	2b7d922e73fbd058099b6ff4b33e99f7	1	17.760	169.990	433.000	21.000	12.000	16.000	1,293.638	0.000
3	709df04791b8bd393b34a58de3cf142d	1	19.440	223.000	300.000	18.000	9.000	14.000	822.746	0.000
4	166b01d0fce75a595b92ad9e84e0843b	1	29.090	98.000	925.000	25.000	6.000	16.000	2,094.074	0.000

12.1 Final Residual Diagnostics

The final actual-versus-predicted and residual-versus-fitted plots are used to determine whether feature engineering and regularisation reduced systematic error.

Residuals above zero indicate underprediction, while residuals below zero indicate overprediction. A horizontal residual pattern centred around zero is preferable. Increasing spread at higher predictions would indicate that expensive shipments remain more difficult to estimate.

In [132...

```
# =====
# FINAL RIDGE ACTUAL VERSUS PREDICTED
# =====

minimum_ridge_plot_value = min(
    y_test.min(),
    ridge_test_predictions.min()
)

maximum_ridge_plot_value = max(
    y_test.max(),
    ridge_test_predictions.max()
)
```

```
figure, axis = plt.subplots(
    figsize=(9, 7)
)

axis.scatter(
    ridge_test_predictions,
    y_test,
    alpha=0.45,
    s=22
)

axis.plot(
    [
        minimum_ridge_plot_value,
        maximum_ridge_plot_value
    ],
    [
        minimum_ridge_plot_value,
        maximum_ridge_plot_value
    ],
    linestyle="--",
    linewidth=2,
    label="Ideal prediction line"
)

axis.set_title(
    "Final Ridge Model: Actual Versus Predicted Freight"
)

axis.set_xlabel(
    "Predicted freight value (BRL)"
)

axis.set_ylabel(
    "Actual freight value (BRL)"
)

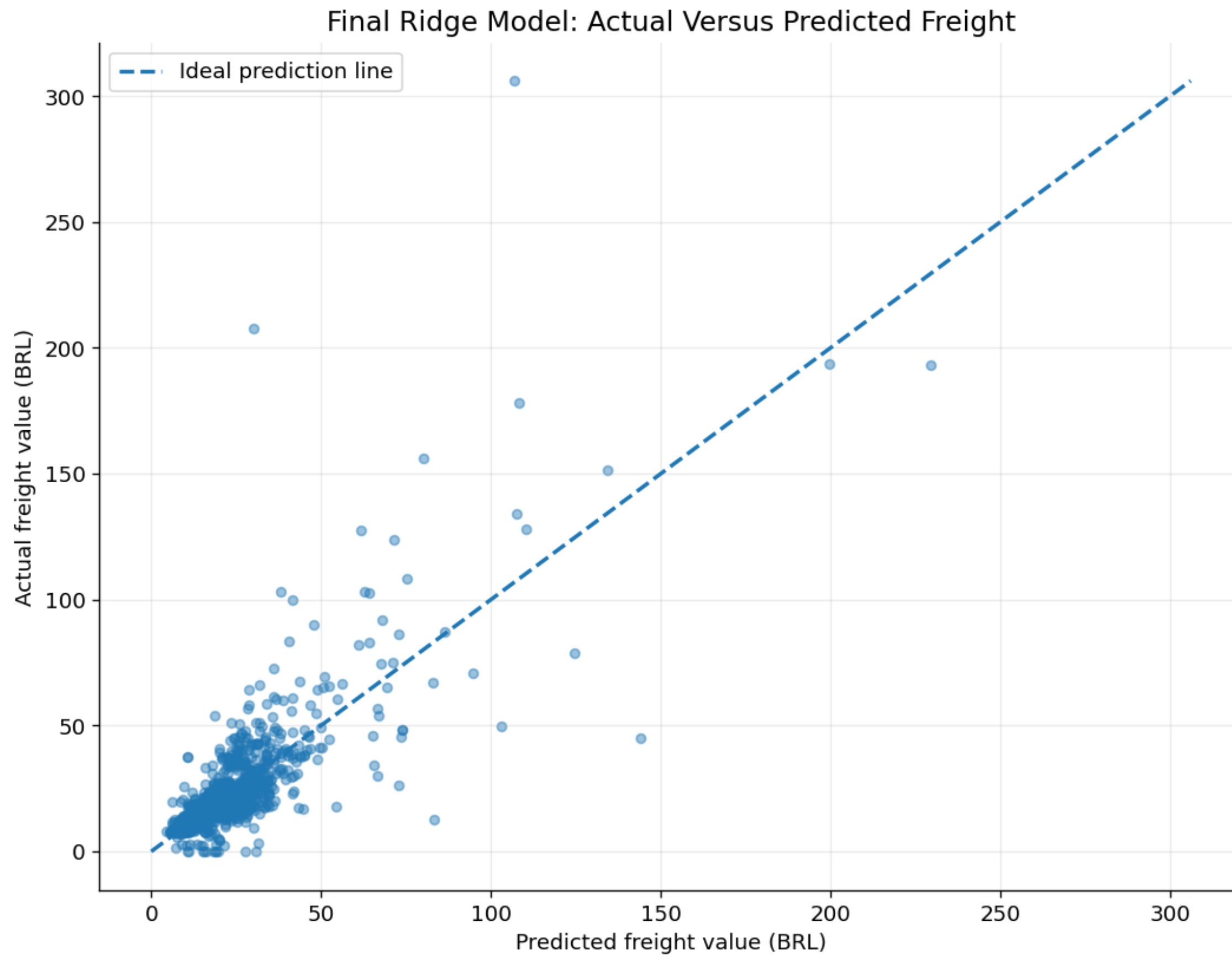
axis.legend()

plt.tight_layout()

save_current_figure(
    "20_ridge_actual_vs_predicted.png"
)

plt.show()
```

Figure saved to: /Users/zimmeljavedvirk/Desktop/datasciencemid/figures/20_ridge_actual_vs_predicted.png



In [133...]

```
# =====  
# FINAL RIDGE RESIDUALS VERSUS FITTED VALUES  
# =====  
  
ridge_residuals = (  
    y_test.to_numpy()
```

```
- ridge_test_predictions
)

ridge_residual_lowess = lowess(
    endog=ridge_residuais,
    exog=ridge_test_predictions,
    frac=0.30,
    return_sorted=True
)

figure, axis = plt.subplots(
    figsize=(10, 7)
)

axis.scatter(
    ridge_test_predictions,
    ridge_residuais,
    alpha=0.45,
    s=22
)

axis.plot(
    ridge_residual_lowess[:, 0],
    ridge_residual_lowess[:, 1],
    linewidth=2,
    label="LOWESS residual trend"
)

axis.axhline(
    y=0,
    linestyle="--",
    linewidth=1.5,
    label="Zero residual"
)

axis.set_title(
    "Final Ridge Residuals Versus Predicted Freight"
)

axis.set_xlabel(
    "Predicted freight value (BRL)"
)

axis.set_ylabel(
    "Residual: actual - predicted (BRL)"
)

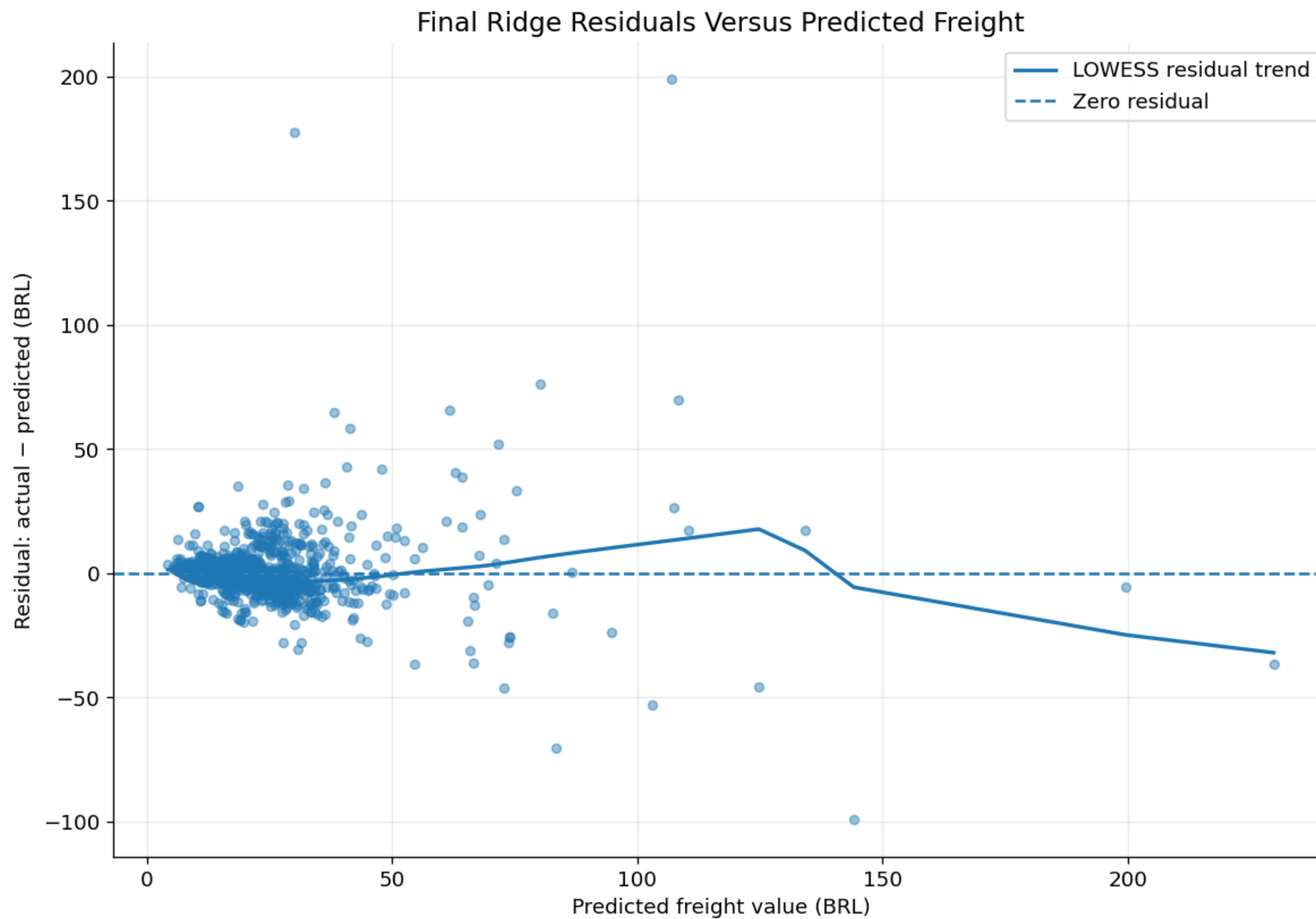
axis.legend()

plt.tight_layout()

save_current_figure(
    "21_ridge_residuais_vs_fitted.png"
)

plt.show()
```

Figure saved to: /Users/zimmeljavedvirk/Desktop/datasciencemid/figures/21_ridge_residuals_vs_fitted.png



```
In [134... # =====  
# BASELINE AND RIDGE RESIDUAL COMPARISON  
# =====  
  
final_residual_comparison_df = pd.DataFrame(  

```

```
[
    create_residual_summary(
        testing_residuals,
        "Baseline multiple linear regression"
    ),
    create_residual_summary(
        ridge_residuals,
        "Tuned Ridge regression"
    )
]

final_residual_comparison_df
```

Out [134...

	dataset	count	mean_residual	median_residual	standard_deviation	minimum	first_quartile	third_quartile	maximum	skewness	excess_kurtosis	mean_absolute_residual	percentage
0	Baseline multiple linear regression	1801	0.145	-0.183	11.494	-40.698	-3.138	2.189	229.903	8.228	129.884	5.166	
1	Tuned Ridge regression	1801	-0.047	-0.560	10.632	-99.238	-2.832	1.772	198.998	6.455	122.068	4.740	

12.2 Group-Bootstrap Confidence Intervals

Ordinary row-level bootstrapping would incorrectly treat every order item as independent. Instead, complete order groups are sampled with replacement.

This produces confidence intervals that respect the grouped structure used during model training and validation. The same bootstrap samples are applied to the baseline and Ridge predictions, allowing a paired comparison of their prediction errors.

In [135...

```
# =====
# ORDER-LEVEL CLUSTER BOOTSTRAP
# =====

def perform_group_bootstrap(
    actual_values,
    baseline_prediction_values,
    ridge_prediction_values,
    group_values,
    number_of_iterations=1_000,
    random_state=42
):
    """
    Estimate group-bootstrap confidence intervals for model
    performance and paired improvement.

    Complete order groups are sampled with replacement.
    """

    actual_array = np.asarray(
        actual_values,
        dtype=float
```

```
)

baseline_array = np.asarray(
    baseline_prediction_values,
    dtype=float
)

ridge_array = np.asarray(
    ridge_prediction_values,
    dtype=float
)

group_array = np.asarray(
    group_values
)

unique_groups = np.unique(
    group_array
)

group_position_mapping = {
    group_value: np.flatnonzero(
        group_array == group_value
    )
    for group_value in unique_groups
}

random_generator = np.random.default_rng(
    random_state
)

bootstrap_records = []

for bootstrap_iteration in range(
    number_of_iterations
):

    sampled_groups = random_generator.choice(
        unique_groups,
        size=len(unique_groups),
        replace=True
    )

    sampled_positions = np.concatenate(
        [
            group_position_mapping[
                group_value
            ]
            for group_value in sampled_groups
        ]
    )

    sampled_actual = actual_array[
        sampled_positions
    ]
```

```
sampled_baseline = baseline_array[
    sampled_positions
]

sampled_ridge = ridge_array[
    sampled_positions
]

baseline_rmse = calculate_rmse(
    sampled_actual,
    sampled_baseline
)

ridge_rmse = calculate_rmse(
    sampled_actual,
    sampled_ridge
)

baseline_mae = mean_absolute_error(
    sampled_actual,
    sampled_baseline
)

ridge_mae = mean_absolute_error(
    sampled_actual,
    sampled_ridge
)

ridge_r_squared = r2_score(
    sampled_actual,
    sampled_ridge
)

bootstrap_records.append(
    {
        "bootstrap_iteration":
            bootstrap_iteration + 1,
        "baseline_RMSE":
            baseline_rmse,
        "ridge_RMSE":
            ridge_rmse,
        "RMSE_reduction":
            baseline_rmse
            - ridge_rmse,
        "baseline_MAE":
            baseline_mae,
        "ridge_MAE":
            ridge_mae,
        "MAE_reduction":
            baseline_mae
            - ridge_mae,
        "ridge_R_squared":
            ridge_r_squared
    }
)
```

```
    return pd.DataFrame(
        bootstrap_records
    )

group_bootstrap_results_df = (
    perform_group_bootstrap(
        actual_values=y_test.to_numpy(),
        baseline_prediction_values=
            baseline_predictions,
        ridge_prediction_values=
            ridge_test_predictions,
        group_values=
            groups_test.to_numpy(),
        number_of_iterations=1_000,
        random_state=RANDOM_STATE
    )
)

group_bootstrap_results_df.head()
```

Out [135...

	bootstrap_iteration	baseline_RMSE	ridge_RMSE	RMSE_reduction	baseline_MAE	ridge_MAE	MAE_reduction	ridge_R_squared
0	1	10.034	9.658	0.376	4.893	4.562	0.331	0.593
1	2	12.760	11.676	1.083	5.394	4.829	0.565	0.619
2	3	8.854	8.469	0.385	4.848	4.497	0.351	0.687
3	4	10.259	9.117	1.142	4.760	4.234	0.526	0.651
4	5	11.170	10.066	1.103	5.052	4.496	0.556	0.624

In [136...

```
# =====
# GROUP-BOOTSTRAP CONFIDENCE INTERVAL SUMMARY
# =====

def bootstrap_confidence_interval(
    values,
    confidence_level=0.95
):
    """
    Return the bootstrap mean and percentile confidence interval.
    """

    lower_percentile = (
        (1 - confidence_level)
        / 2
        * 100
    )

    upper_percentile = (
        100 - lower_percentile
    )

    values = pd.Series(
```

```
        values
    ).dropna()

    return {
        "mean": values.mean(),
        "lower_confidence_limit": (
            np.percentile(
                values,
                lower_percentile
            )
        ),
        "upper_confidence_limit": (
            np.percentile(
                values,
                upper_percentile
            )
        )
    }

bootstrap_metrics = {
    "Ridge RMSE":
        group_bootstrap_results_df[
            "ridge_RMSE"
        ],
    "Ridge MAE":
        group_bootstrap_results_df[
            "ridge_MAE"
        ],
    "Ridge R2":
        group_bootstrap_results_df[
            "ridge_R_squared"
        ],
    "RMSE reduction from baseline":
        group_bootstrap_results_df[
            "RMSE_reduction"
        ],
    "MAE reduction from baseline":
        group_bootstrap_results_df[
            "MAE_reduction"
        ]
}

bootstrap_confidence_records = []

for metric_name, metric_values in (
    bootstrap_metrics.items()
):

    confidence_results = (
        bootstrap_confidence_interval(
            metric_values,
            confidence_level=0.95
        )
    )
```

```
bootstrap_confidence_records.append(
    {
        "metric": metric_name,
        "bootstrap_mean":
            confidence_results["mean"],
        "95_percent_lower_limit":
            confidence_results[
                "lower_confidence_limit"
            ],
        "95_percent_upper_limit":
            confidence_results[
                "upper_confidence_limit"
            ]
    }
)

bootstrap_confidence_interval_df = (
    pd.DataFrame(
        bootstrap_confidence_records
    )
)

bootstrap_confidence_interval_df
```

Out [136...

	metric	bootstrap_mean	95_percent_lower_limit	95_percent_upper_limit
0	Ridge RMSE	10.485	8.002	13.241
1	Ridge MAE	4.733	4.302	5.212
2	Ridge R²	0.629	0.514	0.738
3	RMSE reduction from baseline	0.876	-0.099	1.814
4	MAE reduction from baseline	0.430	0.223	0.633

In [137...

```
# =====
# BOOTSTRAP EVIDENCE OF RIDGE IMPROVEMENT
# =====

ridge_rmse_improvement_probability = (
    group_bootstrap_results_df[
        "RMSE_reduction"
    ] > 0
).mean()

ridge_mae_improvement_probability = (
    group_bootstrap_results_df[
        "MAE_reduction"
    ] > 0
).mean()

paired_bootstrap_conclusion_df = pd.DataFrame(
    {
```

```
        "measurement": [
            (
                "Proportion of bootstrap samples "
                "where Ridge reduced RMSE"
            ),
            (
                "Proportion of bootstrap samples "
                "where Ridge reduced MAE"
            ),
            (
                "Median bootstrap RMSE reduction"
            ),
            (
                "Median bootstrap MAE reduction"
            )
        ],
        "value": [
            ridge_rmse_improvement_probability,
            ridge_mae_improvement_probability,
            group_bootstrap_results_df[
                "RMSE_reduction"
            ].median(),
            group_bootstrap_results_df[
                "MAE_reduction"
            ].median()
        ]
    }
)

paired_bootstrap_conclusion_df
```

Out [137...

	measurement	value
0	Proportion of bootstrap samples where Ridge re...	0.964
1	Proportion of bootstrap samples where Ridge re...	1.000
2	Median bootstrap RMSE reduction	0.901
3	Median bootstrap MAE reduction	0.427

12.3 Error Analysis Across Freight-Cost Bands

Overall metrics can hide poor performance for unusual or expensive deliveries. Test observations are therefore divided into quartiles using their actual freight values.

This analysis determines whether the final model performs consistently across low-, medium- and high-cost shipments.

In [138...

```
# =====
# PERFORMANCE BY ACTUAL FREIGHT QUARTILE
# =====

final_ridge_results_df[
    "actual_freight_quartile"
] = pd.qcut(
    final_ridge_results_df[
```

```
        "freight_value"
    ],
    q=4,
    labels=[
        "Q1: Lowest freight",
        "Q2: Lower-middle freight",
        "Q3: Upper-middle freight",
        "Q4: Highest freight"
    ],
    duplicates="drop"
)

def summarise_prediction_performance(
    dataframe,
    grouping_column,
    actual_column="freight_value",
    prediction_column="ridge_prediction"
):
    """
    Calculate model performance within each subgroup.
    """

    performance_records = []

    for (
        group_name,
        group_dataframe
    ) in dataframe.groupby(
        grouping_column,
        observed=True,
        dropna=False
    ):

        actual_values = group_dataframe[
            actual_column
        ]

        predicted_values = group_dataframe[
            prediction_column
        ]

        performance_records.append(
            {
                "group": str(group_name),
                "observation_count":
                    len(group_dataframe),
                "mean_actual_freight":
                    actual_values.mean(),
                "mean_predicted_freight":
                    predicted_values.mean(),
                "mean_residual": (
                    actual_values
                    - predicted_values
                ).mean(),
                "MAE": mean_absolute_error(
```

```
        actual_values,
        predicted_values
    ),
    "RMSE": calculate_rmse(
        actual_values,
        predicted_values
    ),
    "R_squared": (
        r2_score(
            actual_values,
            predicted_values
        )
        if actual_values.nunique() > 1
        else np.nan
    )
}

)

return pd.DataFrame(
    performance_records
)

freight_quartile_performance_df = (
    summarise_prediction_performance(
        dataframe=final_ridge_results_df,
        grouping_column=
            "actual_freight_quartile"
    )
)

freight_quartile_performance_df
```

Out [138...

	group	observation_count	mean_actual_freight	mean_predicted_freight	mean_residual	MAE	RMSE	R_squared
0	Q1: Lowest freight	451	9.485	11.101	-1.617	2.784	5.893	-3.865
1	Q2: Lower-middle freight	450	14.866	16.304	-1.437	2.351	3.415	-12.271
2	Q3: Upper-middle freight	451	18.426	20.193	-1.766	3.710	5.316	-20.390
3	Q4: Highest freight	449	37.672	33.022	4.651	10.132	19.447	0.494

The negative R^2 values within some freight-cost quartiles do not mean that the complete Ridge model performs worse than the overall benchmark. Dividing test observations according to the actual target creates groups with a deliberately restricted range of freight values. Because R^2 depends on the amount of target variation within a group, it becomes unstable and less informative inside these narrow freight bands. RMSE, MAE and mean residual are therefore treated as the principal measures for subgroup performance.

In [139...

```
# =====
# RIDGE RMSE BY ACTUAL FREIGHT QUARTILE
# =====

figure, axis = plt.subplots(
    figsize=(11, 6)
)
```

```
axis.bar(
    freight_quartile_performance_df[
        "group"
    ],
    freight_quartile_performance_df[
        "RMSE"
    ]
)

axis.set_title(
    "Final Ridge RMSE by Actual Freight-Cost Quartile"
)

axis.set_xlabel(
    "Actual freight-cost quartile"
)

axis.set_ylabel(
    "RMSE (BRL)"
)

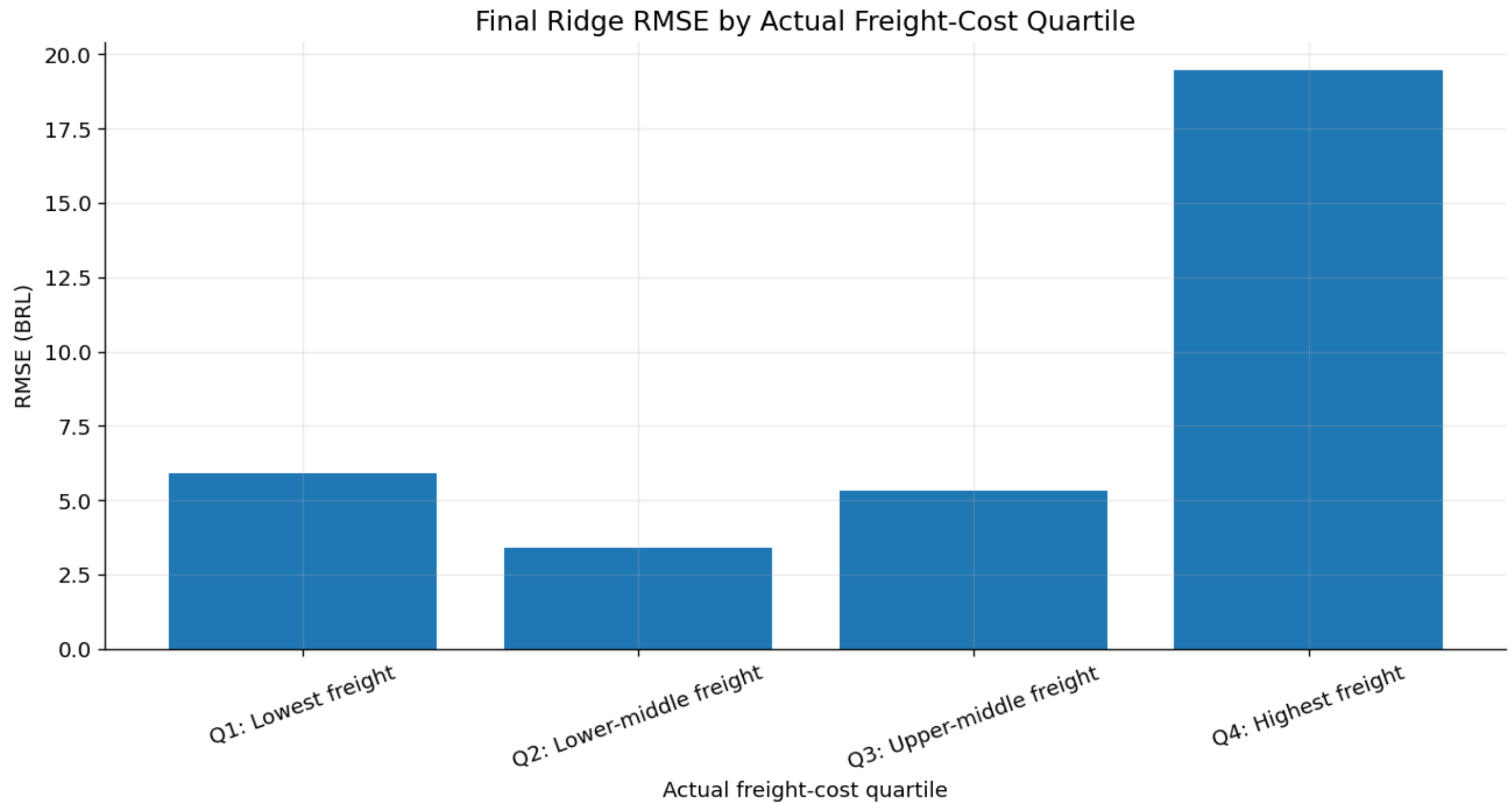
axis.tick_params(
    axis="x",
    rotation=20
)

plt.tight_layout()

save_current_figure(
    "22_ridge_rmse_by_freight_quartile.png"
)

plt.show()
```

Figure saved to: /Users/zimmeljavedvirk/Desktop/datasciencemid/figures/22_ridge_rmse_by_freight_quartile.png



```
In [140... # =====  
# PERFORMANCE BY DISTANCE QUARTILE  
# =====  
  
final_ridge_results_df[  
    "distance_quartile"  
] = pd.qcut(  
    final_ridge_results_df[  
        "distance_km"  
    ],  
    q=4,  
    labels=[  
        "Q1: Shortest distance",  
        "Q2: Lower-middle distance",
```

```
        "Q3: Upper-middle distance",
        "Q4: Longest distance"
    ],
    duplicates="drop"
)

distance_quartile_performance_df = (
    summarise_prediction_performance(
        dataframe=final_ridge_results_df,
        grouping_column="distance_quartile"
    )
)

distance_quartile_performance_df
```

Out [140...

	group	observation_count	mean_actual_freight	mean_predicted_freight	mean_residual	MAE	RMSE	R_squared
0	Q1: Shortest distance	451	12.867	13.011	-0.144	3.005	6.037	0.515
1	Q2: Lower-middle distance	450	19.047	17.831	1.217	4.392	14.052	0.444
2	Q3: Upper-middle distance	450	19.922	20.956	-1.034	4.213	7.654	0.669
3	Q4: Longest distance	450	28.562	28.788	-0.226	7.352	12.632	0.663

```
In [141... # =====
# PERFORMANCE FOR SAME-STATE AND INTERSTATE DELIVERIES
# =====

final_ridge_results_df[
    "route_type"
] = final_ridge_results_df[
    "same_state"
].map(
    {
        1: "Same-state delivery",
        0: "Interstate delivery"
    }
).fillna(
    "Unknown route type"
)

route_performance_df = (
    summarise_prediction_performance(
        dataframe=final_ridge_results_df,
        grouping_column="route_type"
    )
)

route_performance_df
```

Out [141...

	group	observation_count	mean_actual_freight	mean_predicted_freight	mean_residual	MAE	RMSE	R_squared
0	Interstate delivery	1151	23.821	23.845	-0.024	5.739	12.585	0.600
1	Same-state delivery	650	13.499	13.586	-0.087	2.969	5.706	0.502

In [142...

```
# =====
# PERFORMANCE BY PRODUCT-WEIGHT QUARTILE
# =====

weight_available_mask = (
    final_ridge_results_df[
        "product_weight_g"
    ].notna()
)

weight_performance_source_df = (
    final_ridge_results_df.loc[
        weight_available_mask
    ].copy()
)

weight_performance_source_df[
    "weight_quartile"
] = pd.qcut(
    weight_performance_source_df[
        "product_weight_g"
    ],
    q=4,
    labels=[
        "Q1: Lightest products",
        "Q2: Lower-middle weight",
        "Q3: Upper-middle weight",
        "Q4: Heaviest products"
    ],
    duplicates="drop"
)

weight_quartile_performance_df = (
    summarise_prediction_performance(
        dataframe=
            weight_performance_source_df,
        grouping_column="weight_quartile"
    )
)

weight_quartile_performance_df
```

Out [142...

	group	observation_count	mean_actual_freight	mean_predicted_freight	mean_residual	MAE	RMSE	R_squared
0	Q1: Lightest products	451	14.911	14.557	0.354	2.645	4.263	0.568
1	Q2: Lower-middle weight	473	15.754	15.731	0.023	2.749	4.648	0.574
2	Q3: Upper-middle weight	430	17.865	18.729	-0.864	3.862	7.080	0.303
3	Q4: Heaviest products	447	32.068	31.806	0.261	9.804	19.125	0.572

In [143...

```
# =====
# FINAL SELECTED MODEL SUMMARY
# =====
```

```

final_selected_model_summary_df = pd.DataFrame(
{
    "measurement": [
        "Selected model",
        "Selected Ridge alpha",
        "Test RMSE",
        "Test MAE",
        "Test median absolute error",
        "Test R2",
        "Mean group CV RMSE",
        "Group CV RMSE standard deviation",
        "Mean group CV MAE",
        "Mean group CV R2",
        "RMSE improvement over baseline",
        "MAE improvement over baseline",
        "Number of transformed features",
        "Training observations",
        "Testing observations",
        "Overlapping train-test orders"
    ],
    "value": [
        "Feature-engineered Ridge regression",
        best_ridge_model.alpha,
        ridge_test_metrics["RMSE"],
        ridge_test_metrics["MAE"],
        ridge_test_metrics[
            "median_absolute_error"
        ],
        ridge_test_metrics["R_squared"],
        ridge_cv_fold_results_df[
            "RMSE"
        ].mean(),
        ridge_cv_fold_results_df[
            "RMSE"
        ].std(ddof=1),
        ridge_cv_fold_results_df[
            "MAE"
        ].mean(),
        ridge_cv_fold_results_df[
            "R_squared"
        ].mean(),
        (
            baseline_metrics["RMSE"]
            - ridge_test_metrics["RMSE"]
        ),
        (
            baseline_metrics["MAE"]
            - ridge_test_metrics["MAE"]
        ),
        len(ridge_feature_names),
        len(X_train),
        len(X_test),
        len(overlapping_orders)
    ]
}

```

```
)

final_selected_model_summary_df
```

Out [143...

	measurement	value
0	Selected model	Feature-engineered Ridge regression
1	Selected Ridge alpha	300.000
2	Test RMSE	10.629
3	Test MAE	4.740
4	Test median absolute error	2.289
5	Test R²	0.625
6	Mean group CV RMSE	8.746
7	Group CV RMSE standard deviation	0.330
8	Mean group CV MAE	4.484
9	Mean group CV R²	0.609
10	RMSE improvement over baseline	0.863
11	MAE improvement over baseline	0.427
12	Number of transformed features	131
13	Training observations	7199
14	Testing observations	1801
15	Overlapping train-test orders	0

In [144...

```
# =====
# SAVE FINAL EVALUATION OUTPUTS
# =====

final_model_comparison_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "final_model_comparison.csv",
    index=False
)

final_ridge_results_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "final_ridge_test_results.csv",
    index=False
)

final_residual_comparison_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "final_residual_comparison.csv",
    index=False
)
```

```
group_bootstrap_results_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "group_bootstrap_results.csv",
    index=False
)

bootstrap_confidence_interval_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "bootstrap_confidence_intervals.csv",
    index=False
)

paired_bootstrap_conclusion_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "paired_bootstrap_conclusions.csv",
    index=False
)

freight_quartile_performance_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "freight_quartile_performance.csv",
    index=False
)

distance_quartile_performance_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "distance_quartile_performance.csv",
    index=False
)

route_performance_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "route_performance.csv",
    index=False
)

weight_quartile_performance_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "weight_quartile_performance.csv",
    index=False
)

final_selected_model_summary_df.to_csv(
    PROCESSED_DATA_DIRECTORY
    / "final_selected_model_summary.csv",
    index=False
)

print(
    "Final model-evaluation outputs saved successfully."
)
```

Final model-evaluation outputs saved successfully.

Programming Style and Reproducibility

The notebook follows a consistent and reproducible programming structure. Variables and functions use descriptive names, repeated operations are implemented through reusable functions and major processing stages are documented using comments and Markdown explanations. Configuration values, including the random state, sample size, test proportion and number of cross-validation folds, are defined centrally rather than repeatedly entered as unnamed constants.

Raw datasets are preserved separately from cleaned working copies, while processed datasets, audit tables and figures are stored in dedicated output directories. Assertions and merge-validation arguments are used to detect duplicate keys, unexpected row multiplication, missing targets and train-test group overlap. Scikit-learn pipelines ensure that imputation, scaling and categorical encoding are fitted using training data only. A fixed random state supports reproducible sampling, splitting and bootstrap analysis.

13. Evaluation of the Model and Project

Three regression approaches were evaluated: baseline multiple linear regression, feature-engineered ordinary least-squares regression and tuned Ridge regression. Performance was assessed using RMSE, MAE and R^2 . RMSE was selected as the primary measure because it expresses prediction error in BRL and penalises large errors more heavily. This is appropriate for freight-cost estimation, where a substantial underprediction may create greater financial risk than several small errors. MAE was also reported because it provides an easily interpretable average error, while R^2 indicates the proportion of freight-cost variation explained by the model.

The baseline model achieved an RMSE of 11.492 BRL, an MAE of 5.166 BRL and an R^2 of 0.562. It provided a transparent benchmark and confirmed that product, geographical and order-level variables contain useful predictive information. However, its higher errors and structured residual patterns showed that simple additive relationships could not fully capture unusually heavy, bulky or long-distance deliveries.

Feature-engineered ordinary least squares improved RMSE to 10.601 BRL, MAE to 4.755 BRL and R^2 to 0.627. It achieved the lowest RMSE and highest R^2 , demonstrating that parcel volume, logarithmic transformations, squared terms and distance interactions captured important non-linear freight relationships. Nevertheless, several engineered predictors represented correlated transformations of the same underlying variables, which may reduce coefficient stability and make interpretation less reliable.

The tuned Ridge model achieved an RMSE of 10.629 BRL, an MAE of 4.740 BRL and an R^2 of 0.625. Although its RMSE was 0.028 BRL higher than engineered ordinary least squares, Ridge achieved the lowest MAE, the lowest median absolute error and stronger group-aware cross-validation performance. It was therefore selected as the preferred model because regularisation provides a better overall balance between predictive accuracy, coefficient stability and generalisation.

Bootstrap analysis showed that Ridge reduced RMSE relative to the baseline in 96.4% of order-level samples and reduced MAE in every sample. However, performance remained weaker for expensive deliveries: in the highest freight quartile, RMSE increased to 19.447 BRL and average underprediction was 4.651 BRL. The model should therefore support preliminary quotations, budgeting and identification of high-risk orders rather than fully automated pricing.

The project contributes to e-commerce logistics by providing an interpretable and reproducible freight-cost estimation workflow. It is transferable to courier pricing, last-mile delivery planning, warehouse cost estimation and supply-chain budgeting, but each application would require local validation, relevant operational variables and periodic retraining.

References

Gevaers, R., Van de Voorde, E. and Vanelslender, T. (2014) 'Cost modelling and simulation of last-mile characteristics in an innovative B2C supply chain environment with implications on urban areas and cities', *Procedia - Social and Behavioral Sciences*, 125, pp. 398–411. doi: 10.1016/j.sbspro.2014.01.1483. Available at: <https://doi.org/10.1016/j.sbspro.2014.01.1483>

James, G., Witten, D., Hastie, T., Tibshirani, R. and Taylor, J. (2023) *An Introduction to Statistical Learning: with Applications in Python*. Cham: Springer. doi: 10.1007/978-3-031-38747-0. Available at: <https://doi.org/10.1007/978-3-031-38747-0>

Lim, S.F.W.T., Jin, X. and Srail, J.S. (2018) 'Consumer-driven e-commerce: A literature review, design framework, and research agenda on last-mile logistics models', *International Journal of Physical Distribution & Logistics Management*, 48(3), pp. 308–332. doi: 10.1108/IJPDLM-02-2017-0081. Available at: <https://doi.org/10.1108/IJPDLM-02-2017-0081>

Olist (n.d.) *Brazilian E-Commerce Public Dataset by Olist*. Kaggle. Available at: <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

In []:

In []: